

Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

КУРСОВА РОБОТА
з дисципліни **"Об'єктно-орієнтоване програмування"**
на тему: «РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ ДЛЯ МОДЕЛЮВАННЯ
ОБ'ЄКТІВ "Roman Conquest. Макрооб'єкти: Джерело руди, База, Башта. Мікрооб'єкти: Воїн,
Центуріон, Преторіанець"
З ВИКОРИСТАННЯМ UML ТА МОВИ ПРОГРАМУВАННЯ JAVA»

Виконав: студент 1-го курсу групи 2ПІ-256
спеціальності F2 Інженерія програмного
забезпечення
Олександр КОВАЛЬ

Signature

Керівник доцент кафедри ПЗ
к.т.н., доц. Олександр РЕШЕТНІК

Кількість балів: 90
Оцінка: ЄКТС A

Члени комісії:

(підпис)

Решетнік О.О.

(підпис)

Кательніков Д.І.

(підпис)

(прізвище та ініціали)

м. Вінниця - 2026 рік

Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення
Рівень вищої освіти перший (бакалаврський)
Спеціальність F2 Інженерія програмного забезпечення
Освітньо-професійна програма «Інженерія програмного забезпечення»



ЗАТВЕРДЖУЮ

Завідувач кафедри ПЗ

д.т.н., проф., О.Н. Романюк

«10» лютого 2026 року

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ

на курсову роботу з дисципліни "Об'єктно-орієнтоване програмування"

Ковалю Олександрю Дмитровичу, студенту гр. 2ПІ-256

Тема роботи: РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ ДЛЯ МОДЕЛЮВАННЯ ОБ'ЄКТІВ "Roman Conquest.

Макрооб'єкти: Джерело руди, База, Башта. Мікрооб'єкти: Воїн, Центуріон, Преторіанець" З ВИКОРИСТАННЯМ UML ТА МОВИ ПРОГРАМУВАННЯ JAVA.

Завдання: розробити програмну систему і пояснювальну записку до процесу розробки.

Вхідні дані до виконання проєкту:

Мова програмування: Java.

Середовище розробки: NetBeans IDE, IntelliJ IDEA, Eclipse або Visual Studio Code.

Графічний інтерфейс: платформа JavaFX, Swing або AWT.

Зображення мікрооб'єкта: мінімум три графічних примітива (один з яких – текст). Зображення кожного макрооб'єкту повинно складатись не менше чим з трьох графічних примітивів (лінія, прямокутник, коло, арка, полігон, текст, графічний образ та інших), при цьому обов'язково один з примітивів повинен бути текст. Додати до класу мікрооб'єкта в якості елемента посилальний тип, щоб зробити необхідним глибинне копіювання. Для класу

мікрооб'єкта слід реалізувати глибинне копіювання. Макроб'єкти повинні мати можливість містити кількість мікрооб'єктів більше двох. Повина бути реалізована ієрархія класів мікрооб'єктів, яка містить як мінімум три рівня наслідування, які можуть інстанціюватись і відображатись на екрані. Повинен бути реалізований наступний запит: знайти мікрооб'єкт з вказаними параметрами. Програма шукає об'єкт з введеними параметрами і видає на екран: його місце розташування, приналежність до макроб'єктів (чи зазначає, що мікрооб'єкт не належить жодному макроб'єкту). Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які належать вказаному макроб'єкту. Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які не належать жодному макроб'єкту. Повинен бути реалізований наступний запит: встановити критерій сортування мікрооб'єктів у запитах, які виводять переліки. Критеріїв сортування повинно бути не менше трьох, наприклад: Ім'я_студента, Середній_бал, Кількість_Оцінок_4_та_вище. Повинен бути реалізований автоматичний рух деяких мікрооб'єктів: 1) при русі деякі мікрооб'єкти взаємодіють між собою; 2) деякі мікрооб'єкти заходять в макроб'єкти та деякі виходять; 3) характер руху мікрооб'єктів повинен змінюватись при натискуванні певної клавіші клавіатури або при виборі команди меню. Необхідно реалізувати щонайменше три запити на підрахунок мікрооб'єктів. Наприклад: а) кількість активних мікрооб'єктів; б) кількість об'єктів із рівнем життя понад 50%; в) кількість об'єктів, що розташовані у правій частині екрана тощо. У вікні програми відображається лише певна частина ігрового світу, яка не повинна перевищувати 25% його загального розміру. Реалізувати можливість змінювати частину ігрового світу, яку спостерігає користувач. Повинна бути реалізована мінікарта, яка дозволяє прискорену навігацію: 1) на мінікарті повинна бути позначена видима область універсального об'єкта; 2) рух видимої області універсального об'єкта відповідно змінює мінікарту; 3) макроб'єкти та мікрооб'єкти позначені на мінікарті; 4) рух та взаємодія мікрооб'єктів призводить до відповідних змін на мінікарті; 5) натискування лівої кнопки миші на мінікарті призводить до переміщення видимої області універсального об'єкта. Повинна бути запрограмована серіалізація/де-серіалізація всіх об'єктів у текстовий/бінарний/XML файл. Серіалізація/де-серіалізація обов'язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об'єктів, але й про їх позицію на екрані.

Зміст пояснювальної записки:

АНОТАЦІЯ

ЗМІСТ

ВСТУП

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ
ЗАВДАННЯ НА РОБОТУ

2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ

4 КЕРІВНИЦТВО КОРИСТУВАЧА

ВИСНОВКИ

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

ДОДАТОК А (обов'язковий) Лістинг програми

Дата видачі завдання: "11" лютого 2026 р.

Строк подання завершеної курсової роботи: 18 червня 2026 р.

Керівник _____

Олександр РЕШЕТНІК

Завдання отримав _____

Олександр КОВАЛЬ

Signature
(підпис)

Signature
(підпис)

АНОТАЦІЯ

Коваль О.Д. Розробка програмної системи для моделювання об'єктів "Roman Conquest. Макрооб'єкти Джерело руди, База, Башта. Мікрооб'єкти: Воїн, Центуріон, Преторіанець." з використанням UML та мови програмування Java. : курсова робота з дисципліни «Об'єктно-орієнтоване програмування»

У курсовій роботі здійснена розробка програмного застосунку для ОС Windows/Linux/Mac OS, виконаний за допомогою JavaFX. Програмний застосунок моделює функціонування об'єктів "воїни, будівлі" з використанням UML та мови програмування Java. Використано усі основні принципи об'єктно-орієнтованого програмування, а саме: абстракція, інкапсуляція, успадкування та поліморфізм. Проаналізовано проблему зменшення зацікавленості в історії стародавнього Риму та інфраструктуру його армії. Створено готовий програмний продукт.

Ключові слова: мікрооб'єкти, успадкування, макрооб'єкти, агрегація, універсальний об'єкт, композиція.

ЗМІСТ

ВСТУП	4
1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ	7
1.1 Предметна область	7
1.2 Існуючі реалізації	10
1.3 Розробка технічного завдання на роботу	12
1.4 Обґрунтування вибору мови програмування	16
1.5 Висновки	19
2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ	20
2.1 Модель графічного відображення	20
2.2 Графічні процедури підсистеми графічного відображення	23
3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ	25
3.1 Діаграма наслідування	25
3.2 Діаграма агрегації	27
3.3 Діаграма композиції	30
3.4 Діаграма асоціації	30
3.5 Діаграма кооперації	31
3.6 Діаграма послідовності	33
3.7 Діаграма стану	34
4 КЕРІВНИЦТВО КОРИСТУВАЧА	37
ВИСНОВКИ	41
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	42
ДОДАТОК А (обов'язковий) Лістинг програми	43

ВСТУП

Сучасний етап розвитку людської цивілізації нерозривно пов'язаний із глобальною цифровізацією та стрімким впровадженням інноваційних технологій у всі сфери нашої життєдіяльності. Досягнення сучасних інформаційних технологій (ІТ) здійснили справжню революцію з точки зору збереження інформації, її передачі на великі відстані, математичного та логічного обрахунку у реальному часі, а також високоякісної візуалізації, в тому числі тривимірної. Людство здійснило колосальний стрибок від розрізнених паперових архівів до багаторівневих реляційних та об'єктно-орієнтованих баз даних, хмарних сховищ, які здатні надійно акумулювати терабайти історичних, статистичних та структурних відомостей, гарантуючи їхню стійкість до втрат. Розвиток глобальних оптоволоконних та бездротових мереж дозволяє обмінюватися цими масивами даних практично миттєво, стираючи будь-які географічні межі. Проте одним із найважливіших досягнень є здатність сучасних обчислювальних систем та парадигм програмування (зокрема об'єктно-орієнтованого підходу) обробляти складні алгоритми у реальному часі [1]. Використання багатопотоковості дозволяє системі одночасно керувати станами тисяч незалежних програмних сутностей. У поєднанні з передовими графічними рушіями, це дає змогу відтворювати гіперреалістичні симуляційні світи з деталізованою фізикою, де кожен елемент функціонує за наперед заданими правилами.

Незважаючи на цей безпрецедентний технологічний прогрес, сучасне суспільство стикається з глибокою соціокультурною та освітньою кризою, що набуває ознак глобальної проблеми. Складається враження, що люди починають все менше звертати увагу на історію Давнього Риму, його фундаментальну культуру та неоціненний внесок у формування сучасної європейської та світової цивілізації. Світ охоплюють своєрідні інформаційні жахи сьогодення: молоде покоління стрімко втрачає історичну пам'ять, страждає від так званого «кліпового мислення» та повністю розсіює власну увагу в нескінченних потоках низькоякісного, швидкого розважального контенту. Суспільство поступово

забуває, що Давній Рим славився не тільки видатними філософами, талановитими архітекторами, правознавцями і науковцями, а й, на свій час, найбільш розвинутою та ефективною мілітаризованою сферою. Це була система з надзвичайно чіткою, бездоганно налагодженою військовою та суспільною ієрархією, залізною дисципліною та передовим стратегічним плануванням, яке дозволило побудувати одну з найвеличніших імперій в історії. Державні інституції та традиційні освітні заклади продовжують нести величезні витрати коштів на друк консервативних підручників та підтримку застарілих методик викладання. Проте ці колосальні фінансові вливання не дають бажаного результату, викликаючи у сучасній аудиторії лише нудьгу та відторгнення. Це реальна проблема, яку неможливо подолати класичними інструментами, але яку можна успішно вирішити шляхом інтерактивного комп'ютерного моделювання і прогнозування, адаптувавши подачу інформації до вимог цифрової епохи.

Можливість застосування сучасних ІТ у проблемі, що розглядається, відкриває абсолютно нові, безмежні горизонти для інноваційних освітніх підходів. Замість пасивного споживання сухого текстового матеріалу, ми маємо унікальну можливість використати всю потужність об'єктно-орієнтованого програмування для створення інтерактивної форми ознайомлення. Розробка динамічної стратегічної гри або навчального симулятора дозволяє перевести складний історичний контекст у зрозумілу та захопливу площину. Програмне середовище надає інструментарій для детального моделювання архітектури, економіки та життєдіяльності римського військового табору. З інфраструктурної точки зору, така програмна система органічно спирається на глобальні макрооб'єкти: «Джерело руди» забезпечуватиме економічне підґрунтя та критично важливу ресурсну базу для розвитку, центральна «База» відповідатиме за логістику, управління та генерацію нових одиниць, а «Башта» формуватиме неприступний оборонний периметр та надаватиме тактичну перевагу. З іншого боку, деталізація ігрових процесів реалізовуватиметься через мікрооб'єкти, які ідеально репрезентують римську військову ієрархію: від базового «Воїна» (легіонера), через тактично підготовленого «Центуріона», і аж до елітного «Преторіанця». Користувач отримує

можливість безпосередньо взаємодіяти з цією багаторівневою системою, керувати розподілом ресурсів, розробляти власні стратегії та аналізувати наслідки прийнятих рішень. Через такий захопливий ігровий процес молодь підсвідомо засвоює складні принципи римської організації, що робить вивчення історії максимально ефективним та живим.

З огляду на нагальну необхідність подолання глибокої кризи історичної необізнаності, а також враховуючи колосальний потенціал сучасних інструментів об'єктно-орієнтованого програмування для створення інтерактивних навчально-ігрових симуляцій, саме тому в якості теми курсової роботи було обрано розробку програмної системи моделювання об'єктів гри Roman Conquest.

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ

1.1 Предметна область

Давній Рим залишається однією з найвпливовіших цивілізацій в історії людства, чий досягнення у військовій справі, державному управлінні та культурі заклали фундамент сучасної європейської цивілізації. Римська армія вважається одним із найдосконаліших військових механізмів античності, що забезпечило створення і підтримку однієї з найбільших імперій у світовій історії [2]. Проте сучасне покоління все менше цікавиться історією, особливо її військовими аспектами, віддаючи перевагу швидкому розважальному контенту замість глибокого вивчення історичних подій. Сучасні технології відкривають нові можливості для вивчення та популяризації історії через інтерактивні графічні програми, які дозволяють у захоплюючій формі демонструвати взаємодію макро- та мікрооб'єктів один з одним.

Військова організація Давнього Риму базувалася на чіткій ієрархічній структурі, де кожен воїн мав своє місце та функції. Кожна війна у античному світі не відбувалась без **воїнів (Warrior)**, які складали основу легіонів. Згідно з відомим висловом Отто фон Бісмарка, війни виграють не лише генерали, а й звичайні люди, які беруть у них участь. У римській армії воїни часто приходили на військову службу не з власного бажання, а через необхідність захищати інтереси Римської імперії на численних фронтах. Ці люди були змушені піти на війну і битися за Римську Імперію, формуючи хребет римської військової машини. Воїни брали участь у кампаніях на Заході та Сході, воювали з галами в глибинах Іспанії, переправлялися через Середземне море для боротьби з Карфагеном. Саме вони виконували накази командирів та брали участь у найважчих битвах.

Після багатьох років служби та участі в численних кампаніях, коли воїн провів достатньо військових операцій і здобув необхідний досвід та гроші, найздібніші з них могли отримати підвищення у ієрархії римського легіону і стати

центуріонами (Centurio). Це звання походить від латинського слова, що означає десяток. Ця людина отримувала у свою владу вже десяток людей і повагу, краще спорядження та титул. Центуріони відмітилися в історії Риму як вправні воїни та мудрі командувачі, вони є надважливою ланкою у ході бою, на плечах яких лежав величезний обсяг завдань. Центуріони відігравали ключову роль у римській армії, адже на них лежала відповідальність за підготовку солдатів, підтримання дисципліни та прийняття тактичних рішень безпосередньо на полі бою. Вони отримували більшу повагу з боку товаришів та вищу оплату. Їхня майстерність володіння мечем та пілумом, стратегічне мислення та лідерські якості робили їх незамінними у військовій ієрархії.

Як тільки центуріон відзначився у бою, зробив героїчний вчинок, став найкращим майстром меча та пілума у своєму легіоні або врятував своїх людей від безвихідного оточення, йому пропонували стати **преторіанцем (Praetorian)**. Найвищою ланкою в ієрархії римських воїнів були преторіанці. Цих елітних воїнів відбирали серед найкращих солдатів з усієї імперії, і їхня задача була найскладнішою – захищати консулів та імператора. Преторіанці – це воїни при імператорі, це його особиста армія, розташована безпосередньо в Римі. Бути преторіанцем означало мати значний титул і перебувати у відносній безпеці навіть протягом довгих воєн, тому в програмі на тему Roman Conquest вони є найвищою ланкою ієрархії. Преторіанці отримували значно вищу платню порівняно зі звичайними легіонерами та мали особливі привілеї. Також, преторіанці були особливими воїнами, захисниками імператора, останнім етапом опору, котрий відділяв і обороняв сенаторів від населення.

Проте успіх будь-якої імперії залежить не лише від сили її армії, а й від економічної основи. Кожна імперія була побудована насиллям, проте для існування імперії, утримання регулярної армії та побудови нових колоній необхідні гроші. Римська імперія була побудована на системі ресурсного забезпечення, де ключову роль відігравали рудники. **Джерело руди (Source)** є спорудою та макрооб'єктом, що приносить руду до команди гравця. Воно може бути захоплене будь-якою командою, захопити його може будь-яка ланка із ієрархії мікрооб'єктів, але в

залежності від навичок конкретного воїна час захоплення може змінюватися. Контроль над джерелами руди означав контроль над виробництвом зброї, інструментів та монет, необхідних для функціонування держави. Ці стратегічні об'єкти часто ставали предметом конфліктів між різними фракціями.

На шляху до бази імперії зустрічаються перешкоди. Тому в якості інтерпретації цього в програмі було обрано **башту (Tower)** як макрооб'єкт, що являє собою перешкоду для ворожої армії, яку необхідно зруйнувати щоб отримати шлях до бази супротивника. Оборонні башти виконували кілька важливих функцій. По-перше, вони забезпечували захищену позицію для оборонців. По-друге, башти слугували пунктами спостереження, звідки можна було виявити наближення ворога. По-третє, вони використовувалися як бази для постачання. Башти – це споруди, де мікрооб'єкти-союзники можуть отримати допомогу у вигляді поповнення очок здоров'я, також вони атакуватимуть найближчого ворожого мікрооб'єкта в певній області. Водночас ворожі башти становили загрозу, оскільки їхні захисники могли атакувати римських солдатів. Важливою особливістю башт є можливість полагодити їх, але зробити це може тільки центуріон завдяки набутому досвіду в ході кампаній та застосуванню його в реаліях битви.

Центральним елементом військової організації була **база (Base)** або головний табір. База є головною спорудою гравця, її треба захищати будь-якою ціною, оскільки база є початковою точкою всіх мікрооб'єктів будь-якої команди. Римські військові табори були справжніми шедеврами інженерної думки, де кожен елемент мав своє призначення. База служила відправною точкою для всіх військових операцій, місцем підготовки новобранців та центром командування. Втрата головної бази означала катастрофу для римського легіону, тому її захист був пріоритетним завданням. У той же час знищення ворожої бази було головною стратегічною метою в будь-якій військовій кампанії. Знищити базу – головна мета ворожої команди в програмі на тему Roman Conquest.

Ефективність римської армії значною мірою базувалася на взаємодії між різними рангами військовослужбовців. Як і в реальному житті, на війні панує співпраця між воїнами. Звичайні воїни не мають жодних особливих здібностей,

проте вони, перебуваючи поруч із досвідченими преторіанцями, дивлячись на них, можуть отримати натхнення до битви, бути мотивованими, отримуючи підвищення своїх характеристик поки перебувають у певній близькості з преторіанцем. Такий психологічний фактор відігравав важливу роль у підтриманні бойового духу. Центуріони, як зазначалось вище, можуть лагодити башти, щоб ті ще трохи допомогли у ході конфлікту. Ця система наставництва працювала природним чином та давала відчутні результати у бою. Сучасні технології дають змогу моделювати ці взаємодії через об'єктно-орієнтоване програмування, поєднуючи вивчення історії з демонстрацією принципів ООП.

1.2 Існуючі реалізації

На сьогодні існує чимало програмних продуктів, що присвячені тематиці Давнього Риму та його військовій історії. На цю тему є багато програмних реалізацій, проте кожна з них має свої особливості, переваги та недоліки. Кожна гра відображає різні аспекти стародавньої держави, проте всі ігри мають перелік проблем, що не дозволяють вважати їх вдалим вибором для популяризації ідеї вивчення історію Риму. Однією з найвідоміших реалізацій є гра Total War Rome 2, розроблена студією Creative Assembly [3], де було реалізовано поєднання стратегії в реальному часі та покрокової стратегії із сетингом Давнього Риму. Цей проект поєднує елементи стратегії в реальному часі з покроковою стратегічною картою, де гравець може обирати одну з багатьох фракцій на основі історичних сценаріїв. Гра пропонує масштабні битви з тисячами воїнів на екрані, детальну систему управління провінціями та дипломатії, а також історичні кампанії, засновані на реальних подіях.

Іншим цікавим прикладом є гра Ryse Son of Rome, створена компанією Crytek [4], де гравець бере під своє керування простого легіонера, котрий з часом проходить підвищення до командувача. На відміну від стратегічного жанру попереднього проекту, ця гра належить до категорії action-adventure з видом від третьої особи. Гравець керує римським легіонером, який проходить шлях від

звичайного солдата до командира високого рангу. Геймплей зосереджений на динамічних бойових сценах та системі швидкого реагування на дії противників. Гра відрізняється кінематографічною подачею та вражаючою графікою.

Аналізуючи ці реалізації, можна виділити як їхні сильні сторони, так і суттєві недоліки. Total War Rome 2 пропонує глибоку стратегічну складову та масштабність, проте висока складність механік може бути серйозним бар'єром для початківців, які лише знайомляться з історією Риму. Система управління вимагає значного часу на освоєння, що може відлякати користувачів. Великі битви потребують потужного апаратного забезпечення для комфортної гри, що робить цей продукт недоступним для користувачів із звичайними комп'ютерами. Крім того, історична достовірність іноді приноситься в жертву ігровому балансу. Гра також є комерційним продуктом з високою ціною.

Ryse Son of Rome зосереджується на індивідуальному досвіді одного воїна, що дозволяє краще відчувати атмосферу. Однак лінійність сюжету та обмежена варіативність геймплею значно знижують реіграбельність проекту. Гра також критикується за спрощену бойову систему, де після освоєння базових механік процес стає досить повторюваним. Освітній аспект присутній переважно у формі довідкової інформації, яка не інтегрована органічно в ігровий процес.

Крім того, дана реалізація підлягає обмеженням PG через присутність моментів жорстокості, неприхованої крові та насильства, що є перешкодою для розповсюдження серед молодшого покоління. Обидва проекти мають високі системні вимоги та є платними продуктами, що обмежує доступ широкої аудиторії студентів.

З точки зору освітньої цінності, існуючі реалізації не завжди дотримуються оптимального балансу між розважальною та навчальною складовою. Також варто відзначити відсутність проектів, орієнтованих на демонстрацію принципів об'єктно-орієнтованого програмування через історичну тематику. Більшість навчальних програм з ООП використовують абстрактні приклади, упускаючи можливість поєднати програмування з історією. Така комбінація могла б підвищити мотивацію студентів до вивчення обох дисциплін одночасно.

Враховуючи вищезазначені обмеження, моя реалізація є актуальною, оскільки пропонує безкоштовний та доступний інструмент для вивчення історії, який не вимагає потужного обладнання. Програма дозволить студентам бачити, як абстрактні класи відображають реальну ієрархію римського війська. Використання Java забезпечить платформонезалежність.

1.3 Розробка технічного завдання на роботу

В програмі на мові програмування Java повинно бути створено застосунок з графічним інтерфейсом. Клас мікрооб'єкта повинен містити не менше 4 елементів змінних і не менше 4 методів (окрім геттерів та сеттерів). Як мінімум одна змінна повинна бути типу `int`, одна – типу `double` і як мінімум одна – рядок. Зображення мікрооб'єкта: мінімум три графічних примітива, один з яких – текст (навіть в неактивному стані). Білий (=невидимий) графічний примітив не рахується. Додати до класу мікрооб'єкта в якості елемента посилальний тип, щоб зробити необхідним глибинне копіювання. Для класу мікрооб'єкта слід реалізувати глибинне копіювання. Зображення мікрооб'єкта повинно містити графічні примітиви, які показують стан елемента посилального типу, який був доданий до класу мікрооб'єкта, щоб зробити необхідним глибинне копіювання при клонуванні. При натискуванні лівої кнопки миші на мікрооб'єкті він повинен ставати активним/неактивним. В програмі повинна бути реалізована активація декількох об'єктів одночасно. Зображення активного мікрооб'єкта повинно відрізнятись від зображення неактивного хоча б одним графічним примітивом. Інтерфейс програми повинен сигналізувати про те, який саме мікрооб'єкт активував користувач. По вибору студента інформація про активний мікрооб'єкт/мікрооб'єкти може виводитись у рядку статусу або робочій області програми. Якщо користувач активував декілька мікрооб'єктів, то інформація про це також має якимось чином відображатися. При натискуванні клавіш-стрілок активні об'єкти/об'єкт повинні рухатись у вікні програми. При натискуванні клавіші `Delete` активні об'єкти повинні знищуватись. Якщо активного об'єкта нема – клавіша, то клавіша `Delete`

ігнорується. Клавiша Esc повинна відмінати активацію об'єкта. По певній комбінації клавiш (наприклад Ctrl-C) повинно бути реалізоване копіювання активного мікрооб'єкта. При натискуванні правої кнопки миші на мікрооб'єкті повинно з'являтися діалогове вікно, яке дозволяє змінювати параметри вказаного мікрооб'єкта. Додати до проекту три або більше макрооб'єкта. Макроб'єкти повинні мати можливість містити більше двох мікрооб'єктів одночасно. Додати команди рисування макрооб'єктів в програму. Не обов'язково мати можливості додавати/видаляти макрооб'єкти в програму. Цілком достатньо, якщо програма буде мати деяку кількість макрооб'єктів згенеровану при запуску програми і яка буде незмінною протягом роботи програми. Макроб'єкти повинні мати певні позиції в універсальному об'єкті і повинні відображатись на екрані. Позиції макрооб'єктів всередині універсального об'єкта можуть залишатись незмінними протягом роботи програми. В універсальному об'єкті повинно міститись як мінімум один екземпляр кожного оголошеного в темі макрооб'єкта. Зображення кожного макрооб'єкту повинно складатись не менше чим з трьох графічних примітивів (лінія, прямокутник, коло, арка, полігон, текст, графічний образ та інших), при цьому обов'язково один з примітивів повинен бути текст. Білий (=невидимий) графічний примітив не рахується. Мікрооб'єкти можуть або належати одному або більшій кількості макрооб'єктів або не належати жодному. Глядачу має бути зрозуміло в чому полягає приналежність мікрооб'єкта макрооб'єкту, тобто чим вигляд (або поведінка) такого мікрооб'єкта відрізняється від поведінки мікрооб'єкта, який не належить жодному макрооб'єкту. Повинна обов'язково бути реалізована можливість вилучити мікрооб'єкт з всіх макрооб'єктів, в результаті чого він не буде належати жодному макрооб'єкту. Повинна обов'язково бути реалізована можливість зробити так, щоб активний мікрооб'єкт став належати одному з наявних макрооб'єктів. В зображенні макрооб'єкта обов'язково повинно виводитись кількість мікрооб'єктів, які йому належать. Повина бути побудована ієрархія класів мікрооб'єктів, яка містить як мінімум три рівня наслідування, які можуть інстанціюватись і зображатись на екрані. Зображення кожного рівня має відрізнятись від всіх інших хоча б одним

графічним примітивом. В побудованій ієрархії класів повинні бути продемонстровані: а) виклик конструктора базового класу; б) виклик метода базового класу; в) використання динамічного поліморфізму; г) використання статичного поліморфізму. Повинно бути продемонстровано використання ключового слова `instanceof`. Повинен бути реалізований автоматичний рух деяких мікрооб'єктів: 1) при русі деякі мікрооб'єкти взаємодіють між собою (це має бути помітно візуально в тому сенсі, що або щось змінюється у їх зображенні або у характері їх руху); 2) деякі мікрооб'єкти заходять в макрооб'єкти та деякі виходять; 3) характер руху мікрооб'єктів повинен змінюватись при натискуванні певної клавіші клавіатури або при виборі команди меню. Під характером руху мається на увазі не просто траєкторія, а те, в який спосіб вони взаємодіють з макрооб'єктами та іншими мікрооб'єктами. У вікні програми відображається лише певна частина ігрового світу, яка не повинна перевищувати 25% його загального розміру. Реалізувати можливість змінювати частину ігрового світу, яку спостерігає користувач (будь яким способом). Повинна бути реалізована мінікарта, яка дозволяє прискорену навігацію: 1) на мінікарті повинна бути позначена видима область універсального об'єкта; 2) рух видимої області універсального об'єкта відповідно змінює мінікарту; 3) макрооб'єкти та мікрооб'єкти позначені на мінікарті; 4) рух та взаємодія мікрооб'єктів призводить до відповідних змін на мінікарті; 5) натискування лівої кнопки миші на мінікарті призводить до переміщення видимої області універсального об'єкта. Повинна бути запрограмована серіалізація/де-серіалізація всіх об'єктів у текстовий/бінарний/XML файл. Серіалізація/де-серіалізація обов'язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об'єктів, але й про їх позицію на карті. При серіалізації/де-серіалізації обов'язково повинні використовуватись діалогові вікна, щоб запитати у користувача ім'я файлу та його розташування на диску (наприклад можна використовувати системне діалогове вікно `FileChooser`). Продемонструвати використання лямбда-виразу. Продемонструвати використання операція з потоком `Stream`. При натискуванні клавіші `Insert` повинно з'являтися діалогове вікно, яке повинно визначати

параметри створюваного мікрооб'єкта. В нього повинна бути присутня можливість обирати до якого з класів нащадків у ієрархії наслідування він належить. Крім керуючого елемента Button у діалоговому вікні також повинні бути використані як мінімум три з наступного списку: TextField, CheckBox, {ComboBox || ListView || ChoiceBox}, RadioButton .

Повинен бути реалізований наступний запит: знайти мікрооб'єкт з вказаними користувачем параметрами, в результаті якого програма шукає мікрооб'єкт і видає на екран: його місце розташування, приналежність цього мікрооб'єкта до макрооб'єктів (чи зазначає, що мікрооб'єкт не належить жодному макрооб'єкту). Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які належать вказаному макрооб'єкту. Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які не належать жодному макрооб'єкту. Повинен бути реалізований наступний запит: встановити критерій сортування мікрооб'єктів у запитах, які виводять переліки. Критеріїв сортування повинно бути не менше трьох, наприклад: Ім'я_студента, Середній_бал, Кількість_Оцінок_4_та_вище.

Повинні бути реалізовані як мінімум три запити по підрахуванню мікрооб'єктів. Наприклад а) кількість активних мікрооб'єктів, кількість мікрооб'єктів з рівнем життя більше половини, кількість мікрооб'єктів, які знаходяться в правій половині екрану тощо.

Для нормального виконання програми необхідно наступне апаратне забезпечення:

1. Комп'ютер серії IBM PC з частотою 3200 МГц і вище.
2. Операційні системи: Windows Vista/7/10/11; Linux; Mac OS.
3. 32 Gb оперативної пам'яті DDR4.
4. Графічний адаптер SVGA (Super Video Graphic Adapter).
5. Відео-карта об'ємом пам'яті не менше 8 Gb.
6. Клавіатура IBM, миша.

Розмір дискового простору, який потрібен для коректної роботи програми: 30 Mb.

Розмір оперативної пам'яті, що займає програма: 300 Mb.

1.4 Обґрунтування вибору мови програмування

Вибір мови програмування є критично важливим рішенням на етапі проектування будь-якого програмного продукту. Для реалізації проекту на тему Roman Conquest необхідно порівняти три основні мови програмування: C++, C# та Java, і довести, що для даної програми Java є оптимальним вибором. Кожна з цих мов має свої особливості, переваги та недоліки, які необхідно детально проаналізувати.

C++ є потужною мовою програмування, яка розвивалася з мови C шляхом додавання об'єктно-орієнтованих можливостей. Ця мова надає програмісту максимальний контроль над апаратними ресурсами, дозволяє працювати безпосередньо з пам'яттю через покажчики та пропонує високу продуктивність. Програми на C++ компілюються в машинний код для конкретної платформи, що забезпечує швидкість виконання, але вимагає окремої компіляції для кожної операційної системи.

C# розроблена компанією Microsoft як частина платформи .NET Framework та має багато спільного з Java за синтаксисом та концепціями. Ця мова тісно інтегрована з екосистемою Microsoft, що робить її відмінним вибором для розробки додатків під Windows. Протягом останніх років підтримка інших платформ покращилася завдяки проекту .NET Core, але історично C# мала обмеження щодо кросплатформенності.

Java є об'єктно-орієнтованою мовою програмування, розробленою компанією Sun Microsystems у середині дев'яностих років. Головною філософією Java є принцип «напиши один раз, запускай скрізь», що досягається завдяки використанню віртуальної машини Java. Програма, написана на Java, компілюється в байт-код, який потім інтерпретується віртуальною машиною незалежно від операційної системи [5]. Це означає, що один і той же виконуваний файл може працювати на Windows, macOS, Linux без перекомпіляції.

Для детального порівняння цих мов доцільно розглянути їхні характеристики, наведені в таблиці 1.1.

Таблиця 1.1 – Порівняння мов програмування C++, C# та Java

Характеристика	C++	C#	Java
Платформонезалежність	Потребує окремої компіляції для кожної платформи	Обмежена, покращена в .NET Core	Повна підтримка через JVM
Управління пам'яттю	Ручне через покажчики	Автоматичне збирання сміття	Автоматичне збирання сміття
Швидкість розробки	Середня через складність синтаксису	Висока завдяки інструментам Microsoft	Висока завдяки бібліотекам
Складність вивчення	Висока через низькорівневі концепції	Середня, схожа на Java	Середня, зрозумілий синтаксис
Графічні бібліотеки	Qt, wxWidgets, SFML	WPF, Windows Forms	JavaFX, Swing
Багатопотоковість	Складніша реалізація	Вбудована підтримка	Вбудована підтримка

Аналізуючи специфіку проекту на тему Roman Conquest, можна виділити кілька ключових вимог. По-перше, програма має бути доступною для максимально широкої аудиторії, включаючи користувачів різних операційних систем. Студенти можуть працювати на комп'ютерах з Windows, macOS або Linux, тому важливо забезпечити однакову функціональність на всіх платформах. По-друге, для

навчального проекту важлива простота та зрозумілість коду. Java пропонує чистий синтаксис, де кожен клас зберігається в окремому файлі, що полегшує навігацію. C++ має складнішу структуру з заголовковими файлами, що може заплутати початківців.

По-третє, автоматичне управління пам'яттю є важливим фактором для навчального проекту. У Java та C# програмісту не потрібно вручну виділяти та звільняти пам'ять, оскільки це робить вбудований збирач сміття. У C++ необхідно самостійно контролювати час життя об'єктів, що вимагає досвіду. Для студентів, які починають вивчати об'єктно-орієнтоване програмування, автоматичне управління пам'яттю дозволяє зосередитися на логіці програми та принципах ООП. По-четверте, наявність потужної стандартної бібліотеки значно прискорює розробку. Java надає величезний набір готових класів для роботи з колекціями, файлами, графікою. Бібліотека JavaFX пропонує сучасний підхід до створення графічних інтерфейсів з підтримкою анімації.

По-п'яте, підтримка багатопотоковості є важливою для створення динамічної симуляції, де різні об'єкти повинні функціонувати одночасно. Java має вбудовану підтримку багатопотокового програмування з класами Thread, Runnable та пакетом java.util.concurrent. Це дозволяє реалізувати паралельне виконання логіки різних ігрових об'єктів без ускладнення коду. Що стосується C#, хоча ця мова має багато спільного з Java, вона традиційно орієнтована на екосистему Windows. Java залишається більш перевіреним рішенням для створення додатків, які працюють на різних платформах. Також варто враховувати, що багато навчальних закладів використовують Java як основну мову для викладання об'єктно-орієнтованого програмування.

Популярність мови також відіграє роль у виборі. Згідно з рейтингом ТЮВЕ, який відстежує популярність мов програмування на основі кількості пошукових запитів, Java стабільно входить до трійки найпопулярніших мов програмування у світі [6]. Це означає наявність великої спільноти розробників, величезної кількості навчальних матеріалів, готових бібліотек та фреймворків, а також активну підтримку мови. Враховуючи всі вищезазначені фактори, Java є оптимальним

вибором для реалізації проекту на тему Roman Conquest. Ця мова забезпечує необхідну платформонезалежність, має зрозумілий синтаксис для навчальних цілей, пропонує автоматичне управління пам'яттю, містить потужну бібліотеку JavaFX для створення графічного інтерфейсу, надає засоби для багатопотокового програмування та підтримується великою спільнотою. Усі ці переваги роблять Java ідеальним інструментом для створення освітнього програмного продукту.

1.5 Висновки

У першому розділі було проаналізовано предметну область проекту, що охоплює військову організацію Давнього Риму з її ієрархічною структурою. Розглянуто мікрооб'єкти (воїни, центуріони, преторіанці) та макрооб'єкти (джерела руди, башти, бази). Комплексний аналіз існуючих реалізацій виявив їхні недоліки: високі системні вимоги та значну вартість. Порівняння мов програмування C++, C# та Java дозволило обґрунтувати вибір Java як оптимального інструменту завдяки платформонезалежності, зрозумілому синтаксису та потужній бібліотеці JavaFX.

2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

2.1 Модель графічного відображення

Кожен мікрооб'єкт складається з власної PNG-картинки, прямокутника, що відображає рівень очок здоров'я мікрооб'єкта, назва мікрооб'єкта та маркер команди мікрооб'єкта [7].

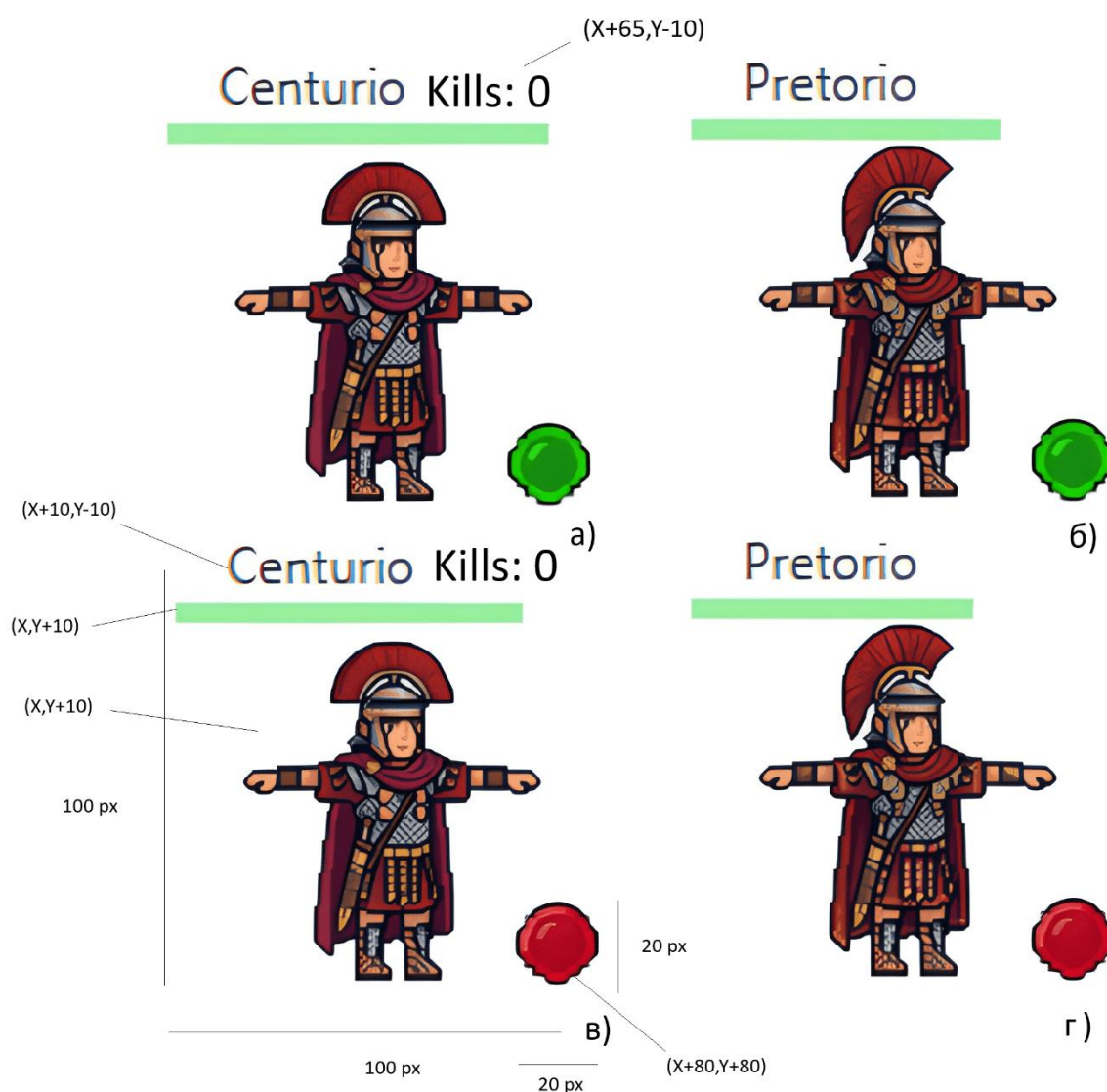


Рисунок 2.1 – Зображення мікрооб'єктів: а) Центуріон за команду союзників; б) Преторіанець за команду союзників; в) Центуріон за команду противників; г) Преторіанець за команду противників

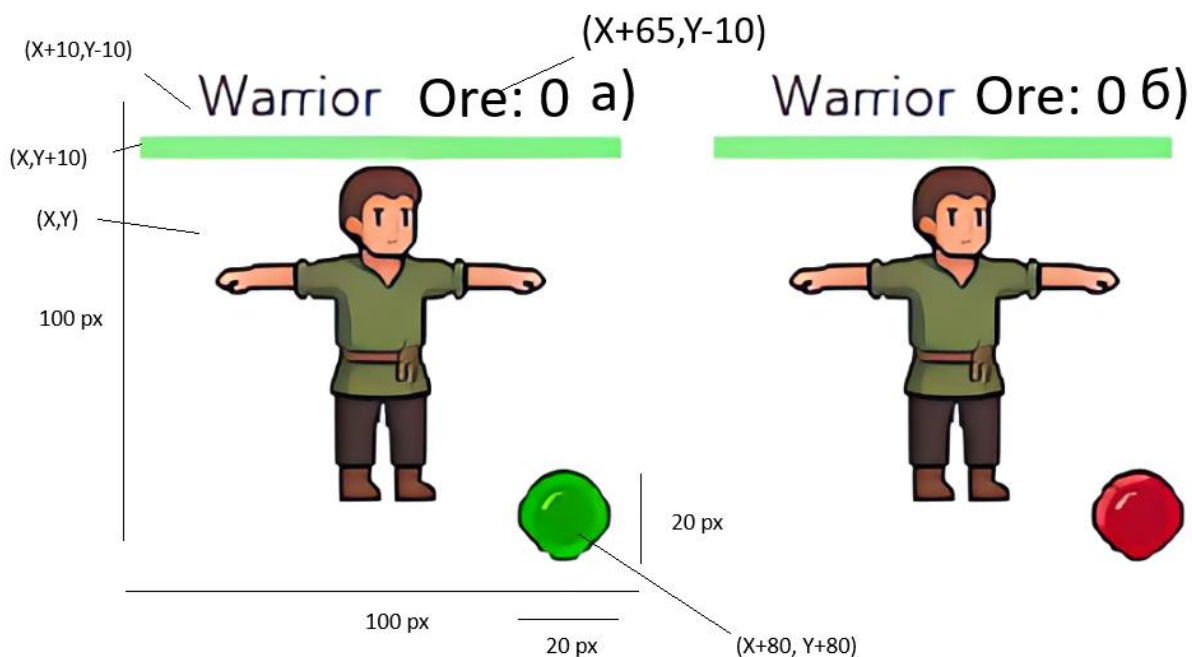


Рисунок 2.2 – Зображення мікрооб’єктів: а) Воїн за команду союзників; б) Воїн за команду противників

Для вирахування мікрооб’єктів за основу було взято початкове положення зображення X та Y (див. рисунок 2.1 ,2.2), які є приватними полями у батьківському класі мікрооб’єкта. Коефіцієнти були підібрані так, щоб було чітко видно кожен елемент мікрооб’єкта.

Кожен макрооб’єкт складається з власної PNG-картинки, прямокутника, що відображає рівень міцності споруди, назва макрооб’єкта, лічильник мікрооб’єктів у реальному часі, що знаходяться в даному макрооб’єкті та маркер, що показують команду до якої належить макрооб’єкт. Всі примітивні графічні елементи зображенні стосовно початкових координат (рисунок 2.3 – 2.5).

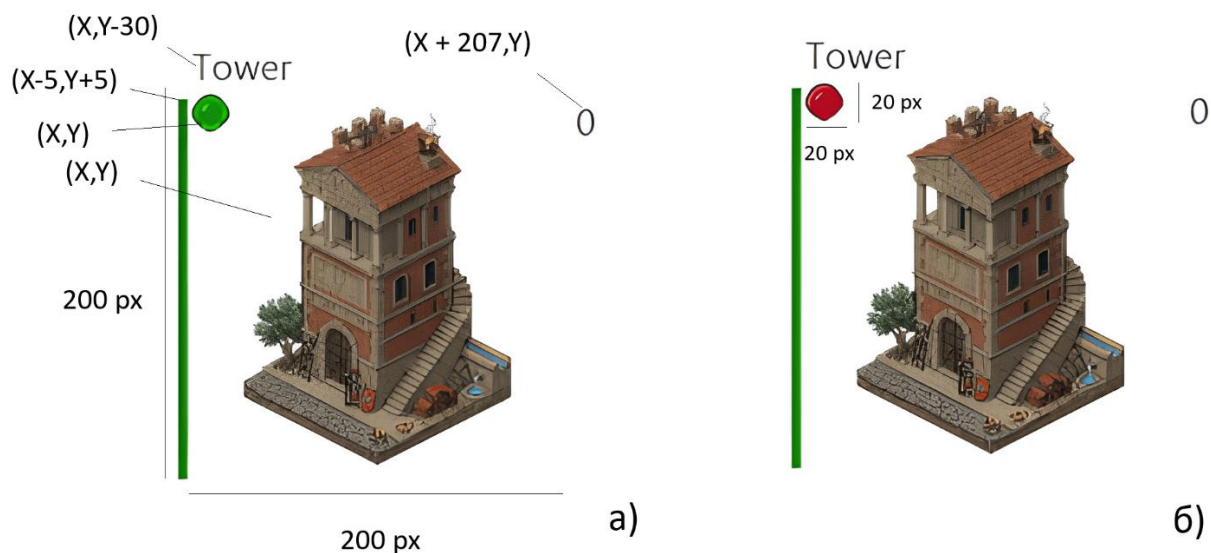


Рисунок 2.3 – Зображення макрооб’єктів: а) Башта за команду союзників; б) Башта за команду противників

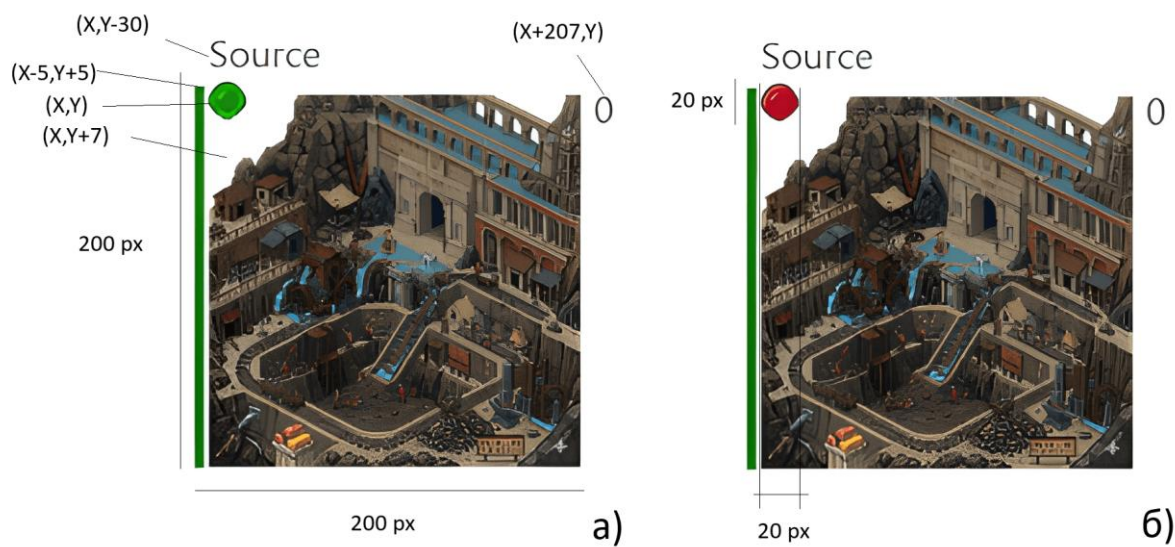


Рисунок 2.4 – Зображення макрооб’єктів: а) Джерело руди за команду союзників; б) Джерело руди за команду противників

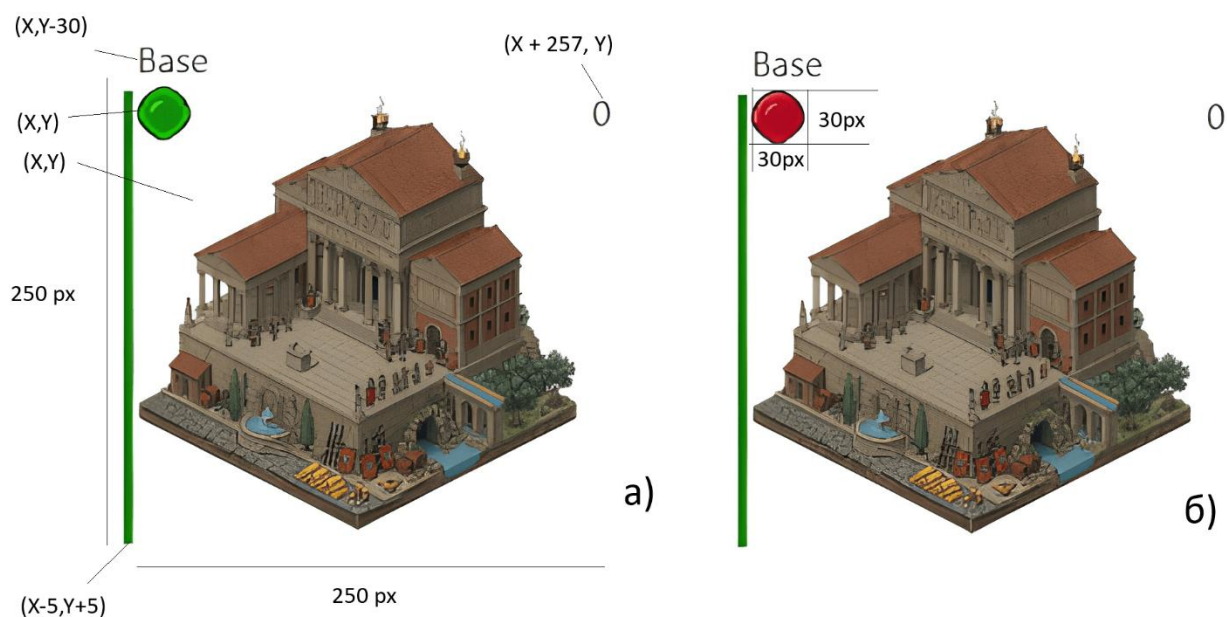


Рисунок 2.5 – Зображення макрооб’єктів: а) База за команду союзників; б) База за команду противників

Мікрооб’єкти складаються з трьох основних графічних примітивів, а також з 1 динамічного графічного примітива для класу Warrior та Centurio, що відображає кількість необхідної руди або кількість необхідних вбивств щоб отримати наступний рівень ієрархії для поточного мікрооб’єкта.

2.2 Графічні процедури підсистеми графічного відображення

Для створення зображень мікро- та макрооб’єктів у програмі було використано графічні засоби бібліотеки JavaFX [8]. Основними використаними графічними примітивами були:

`Group()` – конструктор, що створює групу, яка може містити у собі деяку кількість елементів класу `Node` для їх спільного розміщення на головний екран програми.

`Scene(Parent root, double width, double height)` – створює основну сцену розміром 1080×700 пікселів, яка містить `Group` з усіма графічними об’єктами.

`Image(String url)` – завантажує картинку із переданого URL-посилання або шляху у пам'яті комп'ютера.

`ImageView(Image image)` – конструктор елементу-картинки, який є наслідником класу `Node`, що дозволяє розміщати його на екрані.

`Label(String text)` – створює текстову мітку із заданим текстом.

`Line()` – створює лінійний елемент для відображення полоски здоров'я.

`Rectangle(double x, double y, double width, double height)` – створює прямокутник з координатами верхнього лівого кута (x, y), шириною та висотою.

`setCoordinates()` – переписаний методу класу `Unit`, який синхронізує всі графічні елементи (`Label`, `Line`, `ImageView`, `Rectangle`) з логічними координатами юнита.

`setPosition(double newX, double newY)` – встановлює нову позицію юнита на екрані.

`getBoundsInParent().contains(double x, double y)` – метод `ImageView`, який перевіряє, чи знаходиться точка з координатами (x, y) всередині меж зображення.

`AnimationTimer()` – конструктор елементу, який містить методи `start()`, `stop()` та переписаний метод `handle(long now)`.

3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ

Діагра́ма класів (англ. class diagram) — статичне представлення структури моделі [9]. Відображає статичні (декларативні) елементи, такі як: класи, типи даних, їх зміст та відношення.

3.1 Діаграма наслідування

Наслідування – це один з принципів об'єктно-орієнтованого програмування, який дозволяє описати новий клас на основі існуючого. При цьому властивості і функції базового класу успадковуються нащадком (рисунок 3.1).

Базовий рівень (Unit) є універсальним класом для всіх класів мікрооб'єктів у програмі. Він реалізує інтерфейс Cloneable та містить основні поля та методи, які характеризують загальні властивості мікрооб'єкта, такі як: health – рівень здоров'я; isSpawned – флаг спавну; isDead – флаг смерті; damage – урон; team – приналежність до команди; inventor – контейнер інвентарю; baseHealth – базова кількість здоров'я; baseDamage – базовий урон; numObjects – статичний лічильник об'єктів класу; labelName – надпис з ім'ям (перший графічний примітив); life: Line – смужка здоров'я (другий графічний примітив); image: ImageView – png картинка юніта (третій графічний примітив); rectActive: Rectangle – область активності мікрооб'єкта; imageMark – графічний показник команди; mainWeaponImage – графічне представлення головної зброї (четвертий графічний примітив); x, y – координати мікрооб'єкта на екрані; moveSpeed = 0.005 – швидкість руху; lastAttackTime – час останньої атаки; ATTACK_COOLDOWN = 1000 – час перезавантаження атаки; а також деякі інші технічні змінні для графіки та логіки.

«V» перед функціями означає: «віртуальна». Основні методи Unit включають: V void takeDamage() – обробляє отримання шкоди; V void setPosition(double x, double y) – встановлює позицію юніта; V void resurrect() – додає графічні елементи юніта на екран; V void logic() – встановлює поведінку конкретного типу юніта; V void attack() – обробляє атаку юніта на ворогів; V void

`moveTo(double newX, double newY)` – метод переміщення юніта та всіх його графічних примітивів; `V void removeUnitFromGame()` – видалення юніта з гри та очищення ресурсів.

У програмі реалізовано наслідування чотирьох рівнів (рисунок 3.1). Перший рівень (`Warrior`) наслідує `Unit` і розширює функціональність полями та методами для збору ресурсів (руди). Основні атрибути: `name = "Warrior"` – ім'я (статична константа); `MAX_HEALTH = 100.0` – максимальне здоров'я (константа); `oreAmount` – поточна кількість зібраної руди; `activeOre` – активна руда під час збирання (до 10); `ORE_COOLDOWN = 1000` – час перезагрузки збору руди; `lastOreTime` – час останнього збору; `collectingOre` – флаг процесу збору руди; `deliveringOre` – флаг процесу доставки руди; `COMBAT_PRIORITY_RADIUS = 180.0` – радіус пріоритету боїв з ворогами; `oreCountLabel` – графічний показник активної кількості руди (п'ятий графічний примітив); а також деякі інші технічні змінні. Основні методи `Warrior`: `V void logic()` – логіка руху, пошуку ворогів, збору та доставки руди; `V void doOre()` – обробляє цикл збору руди зі `Source` та доставки до `Base`; `V void promotion()` – промоція `Warrior` до `Centurio` при наборі 50 одиниць руди; `protected void setOreCount(int oreCount)` – встановлює активну кількість руди; `public double getOre()` – повертає поточну кількість накопленої руди.

Другий рівень (`Centurio`) наслідує `Warrior` і розширює можливості додаванням обліку вбивств та лікування союзних будівель. Основні атрибути: `name = "Centurio"` – ім'я (статична константа); `MAX_HEALTH = 120.0` – максимальне здоров'я (константа); `healNum = 10.0` – величина лікування будівлі за один раз; `HEAL_COOLDOWN = 2000` – час перезагрузки лікування; `lastHealTime` – час останнього лікування; `killCount` – лічильник вбитих ворожих юнітів; `killCountLabel` – графічний показник кількості вбивств (шостий графічний примітив); `countedKills: HashSet` – множина відслідкованих вбитих юнітів для уникнення подвійного рахунку; а також деякі інші технічні змінні. Основні методи `Centurio`: `V void attack()` – атака з обліком вбитих ворогів та збільшенням лічильника `killCount`; `V void logic()` – розширена логіка вибору цілей (враги, будівлі, союзні будівлі для лікування, з пріоритезацією союзних будівель що потребують лікування); `V void promotion()` –

промоція Centurio до Pretorio при наборі 5 вбивств; `private void heal(World target)` – лікування союзної будівлі якщо знаходиться в зоні 100 пікселів; `public int getKillCount()` – повертає кількість вбивств.

Третій рівень (Pretorio) наслідує Centurio і додає здатність групового лікування для союзних юнітів в радіусі ефекту. Основні атрибути: `name = "Pretorio"` – ім'я (статична константа); `MAX_HEALTH = 150.0` – максимальне здоров'я (константа); `healNum = 5.0` – величина лікування кожного союзного юніта за один раз; `areaOfEffect: Circle` – графічне представлення зони радіусу 150 пікселів для групового лікування (сьомий графічний примітив); `HEALAREA_COOLDOWN = 1000` – час перезагрузки групового лікування; `lastHealAreaTime` – час останнього групового лікування; а також деякі інші технічні змінні. Основні методи Pretorio: `V void logic()` – розширена логіка вибору цілей та групового лікування; `V void healInArea()` – лікування всіх союзних юнітів (крім себе) в радіусі 150 пікселів, що мають здоров'я > 0 ; `V void flipActivation()` – перемикання активності юніта з показом/приховуванням зони ефекту; `V void resurrect()` – додає зону ефекту на екран; `V void setCoordinates()` – оновлює позицію зони ефекту. Рівні при такому розширюються – мікрооб'єкт отримує максимальне здоров'я 150 та нові здатності групового лікування союзних юнітів.

3.2 Діаграма агрегації

Відношення агрегації описує зв'язок між об'єктом-контейнером та його елементами, які можна динамічно додавати або вилучати. Як правило, таке співвідношення має формат 1:N. Суть агрегації полягає в тому, що один об'єкт зберігає посилання на інший, проте їхні життєві цикли не пов'язані — обидва можуть існувати як самостійні сутності незалежно один від одного. У даній програмі прикладами агрегації слугують зв'язки `Base – Unit`, `Tower – Unit`, `Source – Unit` та `Unit – inventor` (рисунок 3.2). Кожна із зазначених сутностей виступає контейнером для іншої, вміщуючи в собі елементи певного класу чи компонента [1].

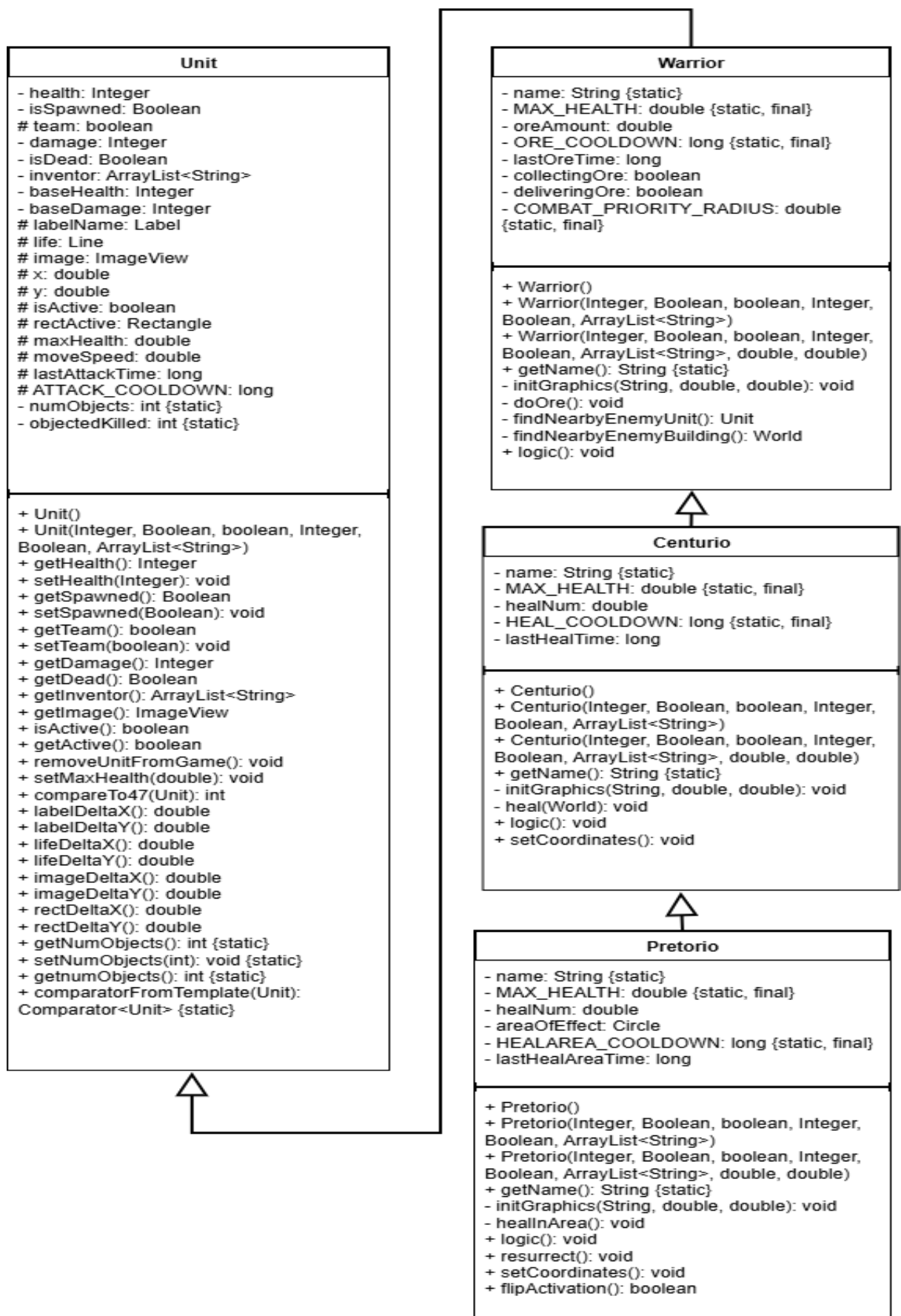


Рисунок 3.1 – Діаграма наслідування

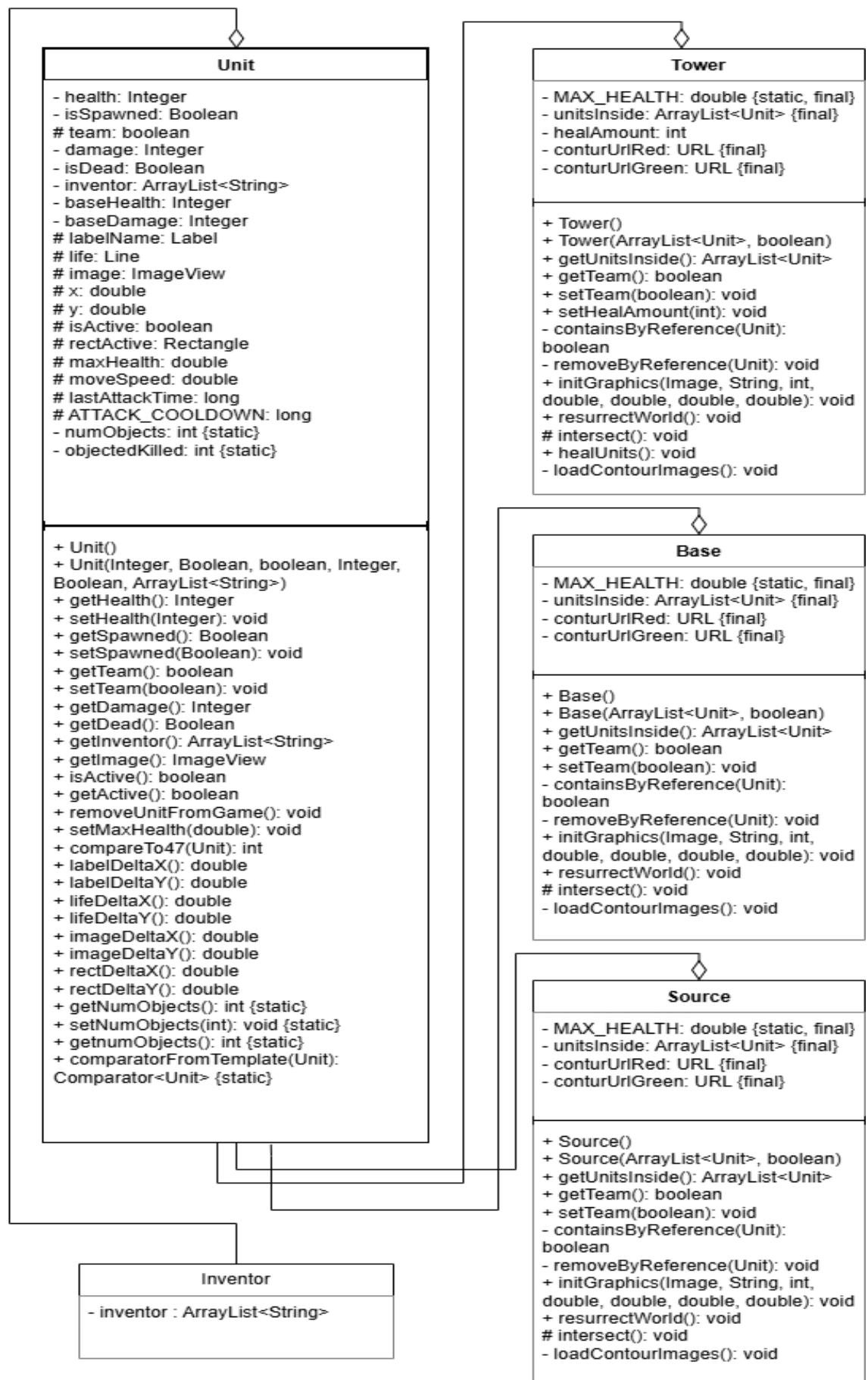


Рисунок 3.2 – Діаграма агрегації

В програмі співвідношення між класом мікрооб'єкта та макрооб'єктів є динамічними, мікрооб'єкти не можуть належати декільком мікрооб'єктам одночасно.

3.3 Діаграма композиції

Суть композиції абсолютно протилежна суті агрегації. Вона полягає у тому, що певний об'єкт є невід'ємною частиною іншого, який в свою чергу, не може існувати без попереднього. У програмі композиція реалізована у зв'язку мікрооб'єкт та його інвентар; мікрооб'єкт та його `labelName`; мікрооб'єкт та його показник здоров'я; мікрооб'єкт та його `image`; мікрооб'єкт та його `rectActive` (`Rectangle`) (рисунки 3.3). Неможливо створити мікрооб'єкт без перелічених параметрів.

3.4 Діаграма асоціації

Асоціація являє собою вид взаємозв'язку, при якому об'єкт містить у собі посилання на менший, службовий об'єкт. Це дозволяє одному об'єкту звертатися до іншого та використовувати його для досягнення визначених цілей.

В даній програмі прикладом асоціації є взаємодія між класом башти (`Tower`) та мікрооб'єкта (`Unit`) (рисунки 3.4). Клас макрооб'єкта `Tower` взаємодіє з кожним мікрооб'єктом, що належить до макрооб'єкта, шляхом зміни здоров'я в залежності від команди макрооб'єкта. При належності мікрооб'єкта до протилежної команди, здоров'я мікрооб'єкта буде зменшуватися протягом часу перебування в макрооб'єкті. Також, схожу функцію взаємодії має мікрооб'єкт класу `Pretorio`, він має визначену на певній площі (радіус) взаємодії, що поповнює очки здоров'я всім мікрооб'єктам що перебувають в цій області за умови, що належність мікрооб'єкта класу `Pretorio` співпадає із зазначеним мікрооб'єктом, якщо в радіусі дії буде знаходитися мікрооб'єкт іншої команди, нічого не відбуватиметься.

3.5 Діаграма кооперації

Кооперація (collaboration) — це специфікація групи об'єктів, які спільно взаємодіють для реалізації певних варіантів використання в загальному контексті системи, що моделюється. Її головна мета полягає у деталізації особливостей реалізації цих варіантів використання або найбільш значущих системних операцій. Структура поведінки, застосована в розробленій програмі, визначає процес обробки повідомлень під час її виконання (рисунок 3.5).

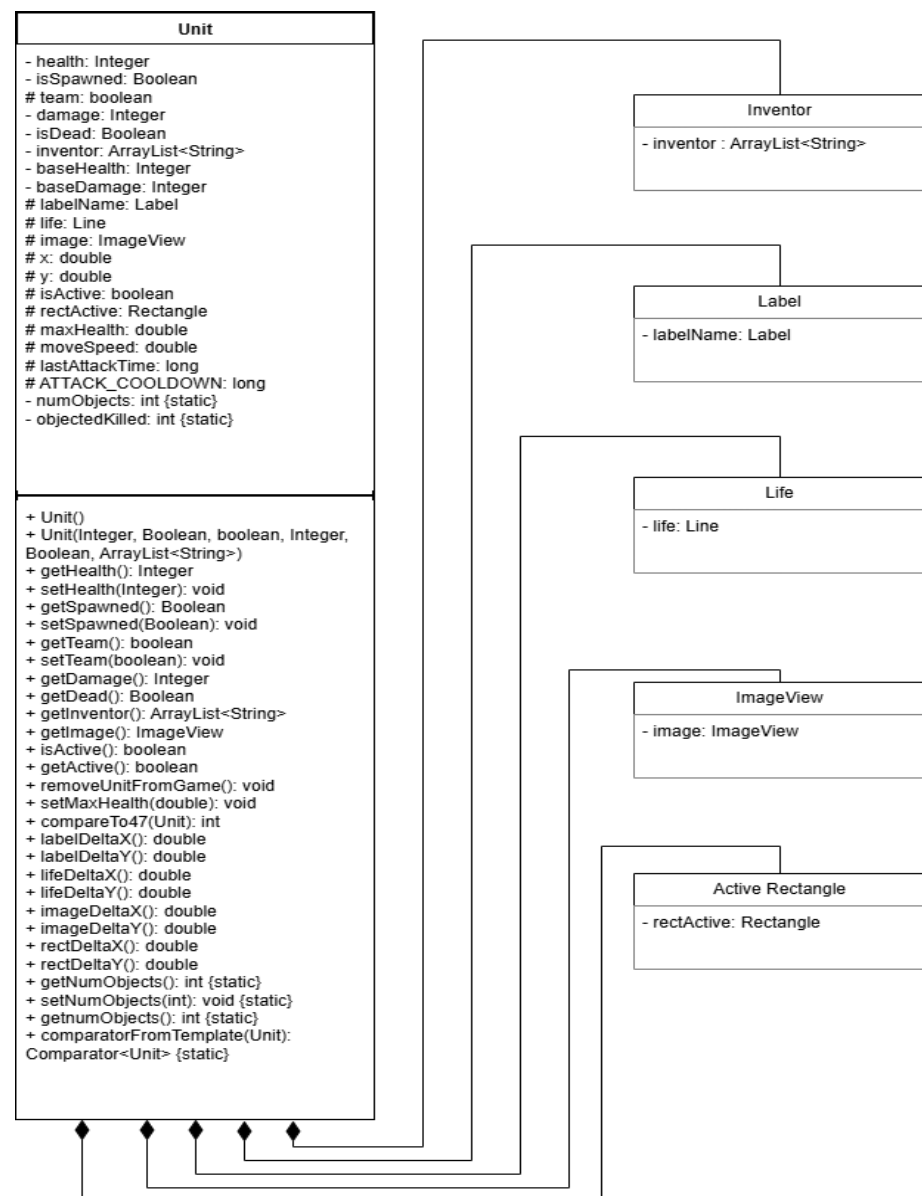


Рисунок 3.3 – Діаграма композиції

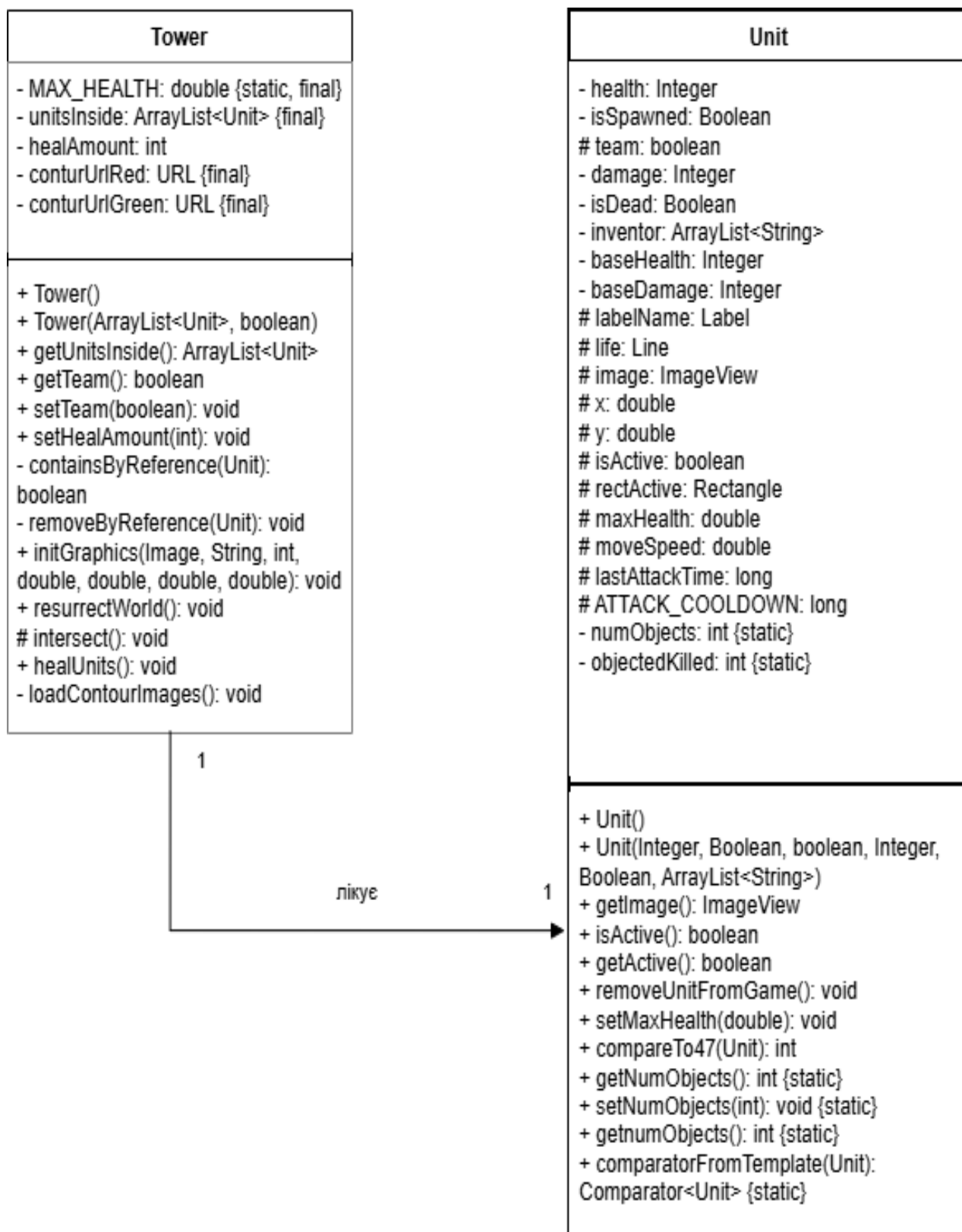


Рисунок 3.4 – Діаграма асоціації

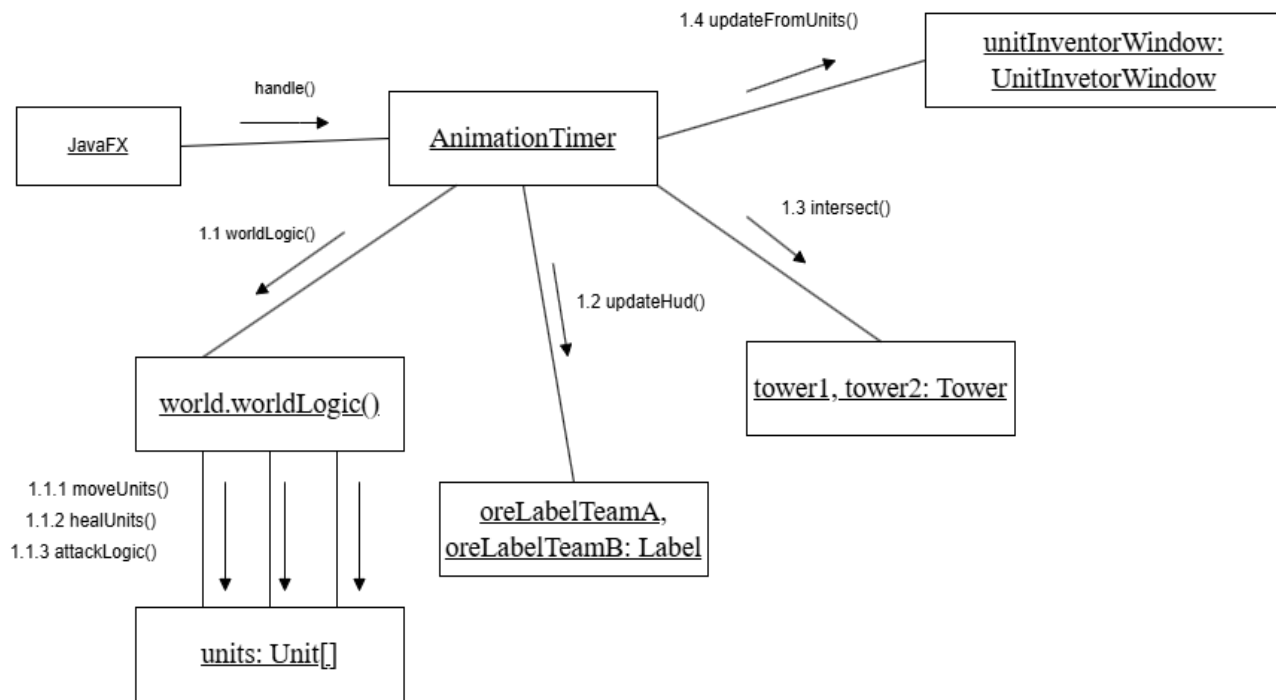


Рисунок 3.5 – Діаграма кооперації

Діаграма кооперації (див. рисунок 3.5) ілюструє перелік методів та класів що входять до кооперації протягом використання програми.

3.6 Діаграма послідовності

Діаграма послідовності (Sequence Diagram) — це різновид поведінкової діаграми UML, що відображає хронологічну послідовність взаємодії об'єктів системи в межах певного заданого сценарію. Вона наочно демонструє екземпляри об'єктів та процес їхньої комунікації один з одним через обмін повідомленнями в часі. Діаграми послідовності активно використовуються для візуалізації складної бізнес-логіки, життєвих циклів та механізмів обміну даними між компонентами.

Зокрема, розроблена діаграма дозволяє зрозуміти, як ігрові мікрооб'єкти взаємодіють один з одним та з ігровим середовищем, і що є безпосереднім наслідком їхньої взаємодії (рисунок 3.6). Вона ілюструє багатокроковий сценарій еволюції (покращення) бойових одиниць.

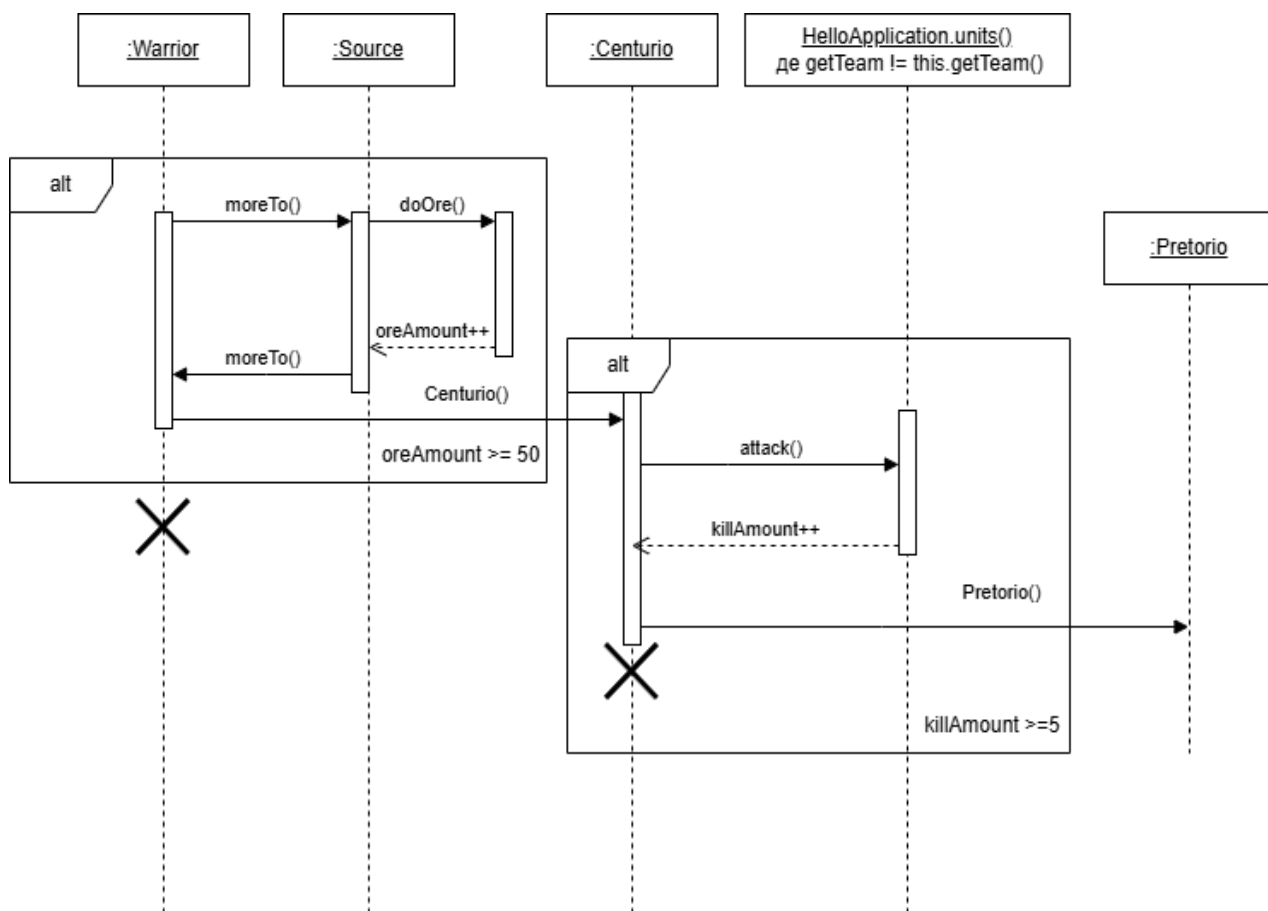


Рисунок 3.6 – Діаграма послідовності

Продемонстрована взаємодія мікрооб'єкта Warrior (див. рисунок 3.6), який протягом програми при певних умовах переходить до наступного в ієрархії класу мікрооб'єкта виконуючи певні дії, що дозволяють йому підвищити свою кваліфікацію.

3.7 Діаграма стану

Діаграма станів (State Diagram) — це поведінкова діаграма UML, що візуалізує динамічну поведінку об'єкта, його можливі стани та переходи між ними під впливом певних умов [10]. Вона дозволяє детально простежити реакцію системи на різноманітні чинники під час її життєвого циклу (рисунок 3.7).

Життєвий цикл розпочинається зі створення базового юніта класу Warrior, головним завданням якого є накопичення ресурсів. Об'єкт циклічно виконує метод doOre(), доки кількість зібраної руди не досягне необхідного мінімуму (oreAmount

≥ 50). Після успішного виконання цієї умови генерується подія `promotion()`, яка ініціює перехід об'єкта до вдосконаленого класу `Centurio`. Відтепер його ціль змінюється на здобуття бойового досвіду: юніт циклічно викликає метод `attack()` та взаємодіє з об'єктами колекції `HelloApplication.units()`, поки не здійснить п'ять знищень ворогів (`killAmount  $\geq 5$`).

Досягнення цієї бойової цілі викликає новий тригер `promotion()`, що трансформує об'єкт у фінальний елітний клас — `Pretorio`. Цей юніт продовжує вести бій аж до знищення бази супротивника, після чого система переходить до кінцевого успішного стану «Перемога!».

Водночас важливою умовою є те, що на будь-якому етапі еволюції (`Warrior`, `Centurio` чи `Pretorio`) система безперервно перевіряє рівень здоров'я юніта. Якщо цей показник падає до критичної позначки (`health  $\leq 0$`), відбувається негайний перехід до термінального стану, що символізує загибель та вилучення мікрооб'єкта з гри. Таким чином, діаграма комплексно описує як поетапний розвиток бойових одиниць, так і всі можливі сценарії завершення їхнього життєвого циклу.

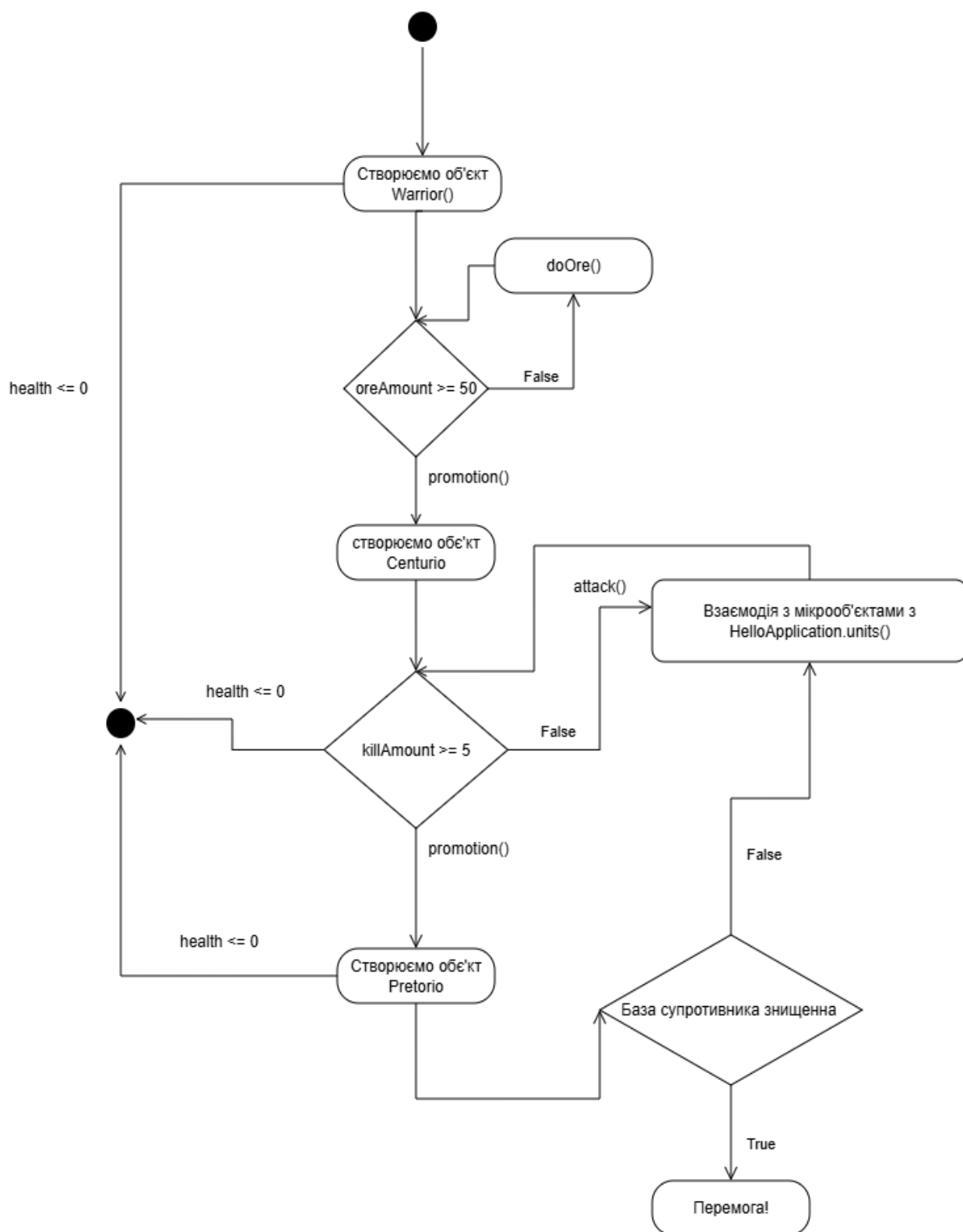


Рисунок 3.7 – Діаграма станів

Програма закінчується при умові, що база супротивника або мікрооб'єкту буде знищена мікрооб'єктом ворожої команди.

4 КЕРІВНИЦТВО КОРИСТУВАЧА\

Створена програма не потребує окремої інсталяції проекту: достатньо скопіювати JAR-архів або зібрані файли програми на комп'ютер. Запуск здійснюється подвійним натисненням лівої кнопки миші по файлу JAR або через консоль командою `java -jar Cursova.jar` за наявності встановленої актуальної версії Java 21+. Під час розробки проект також можна запускати командою `mvnw.cmd javafx:run` у каталозі проекту. Програма є JavaFX-стратегією з великою картою світу, мінікартою, вікнами керування юнітами та підтримкою збереження і завантаження стану гри у форматі XML [11].

Одразу після запуску програми ми можемо спостерігати, що вона запустилася у вікі із розширенням 1920x1080 (FHD). На екрані можна спостерігати розташування об'єктів згідно рисунку (рисунок 4.1).



Рисунок 4.1 – Вид програми при коректному запуску

Щоб відкрити вікно для додавання нових мікрооб'єктів (рисунок 4.2), слід натиснути гарячу клавішу «INSERT». У цьому вікні користувач може створювати

нові бойові одиниці, обираючи їхній тип, параметри та належність до команди, кожен новостворений мікрооб'єкт з'явиться на базі відповідно до команди.

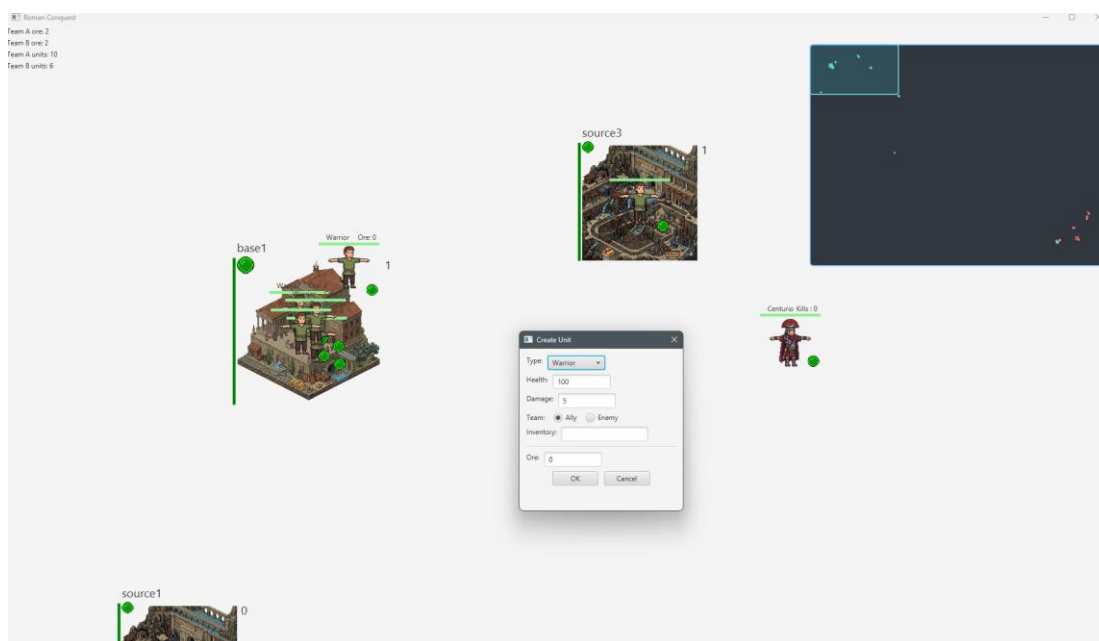


Рисунок 4.2 – Інтерфейс створення нового мікрооб'єкта

Рухати активними мікрооб'єктами можна за допомогою клавіш «W», «S», «A», «D», щоб активувати мікрооб'єкт потрібно натиснути лівою клавішою миші. Для взаємодії із макрооб'єктами слід підійти до найближчого ворожого макрооб'єкта та натиснути клавішу «SPACE», у результаті взаємодії макрооб'єкт втратить частинку очок міцності, також, деякі макрооб'єкти автоматично взаємодіють із мікрооб'єктами відповідно до належності до команди мікрооб'єкта.

Для відкриття вікна інвентарю мікрооб'єкта (рисунок 4.3) треба натиснути клавішу «I», в ньому можна спостерігати наявні предмети у мікрооб'єкта.

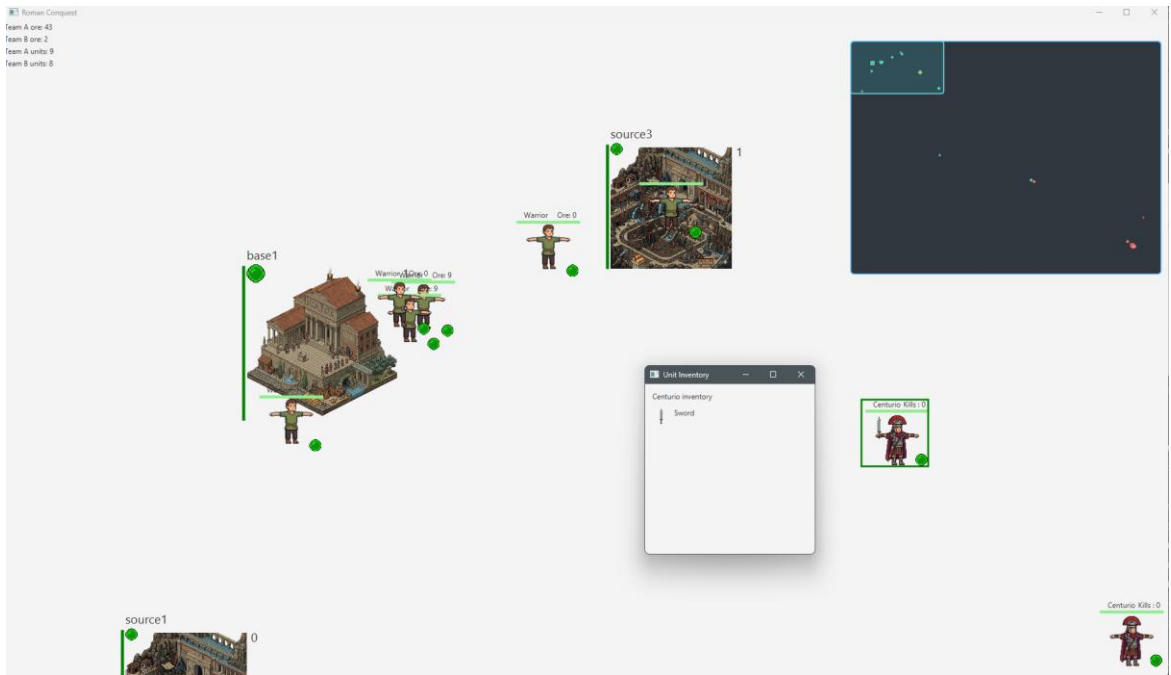


Рисунок 4.3– Інтерфейс інвентарю активного мікрооб’єкта

Для того щоб відкрити панель керування мікрооб’єктами (рисунок 4.4) слід натиснути клавішу «F», на цій панелі можна побачити всіх мікрооб’єктів, інформацію про них, їх розташування, і належність до макрооб’єкта, також можна встановити фільтри, згідно яким можна побачити всі мікрооб’єкти відповідно встановленим критеріям.

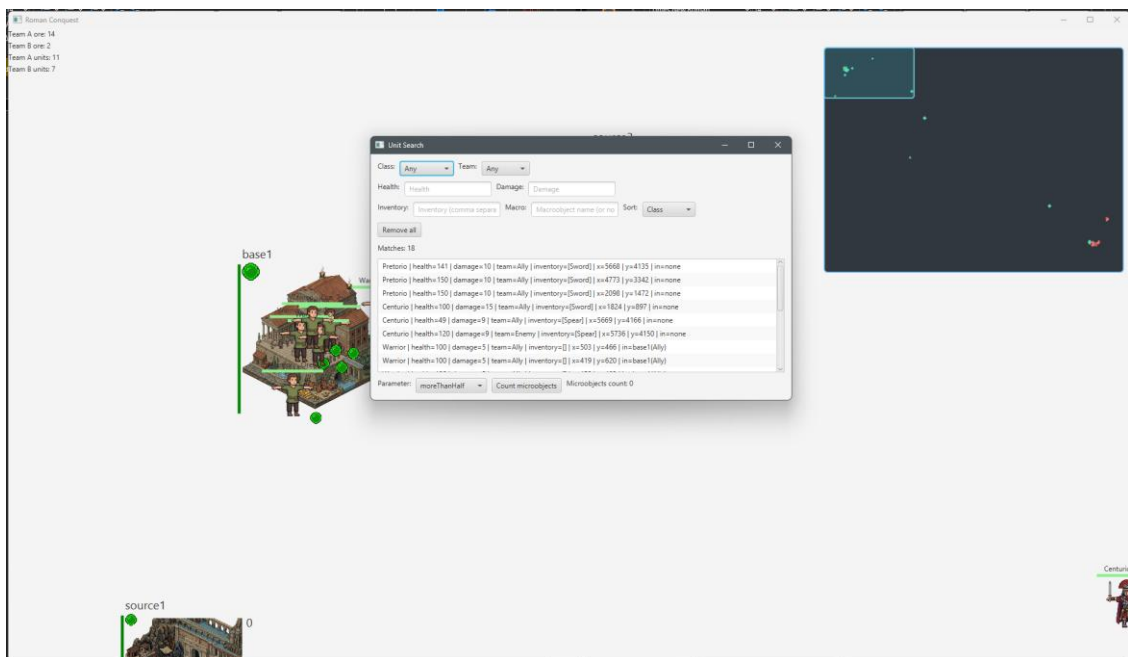


Рисунок 4.4– Інтерфейс інвентарю активного мікрооб’єкта

Щоб відкрити опцію серіалізації/де-серіалізації (рисунок 4.5) слід натиснути «Z», на ній можна зберегти актуальну конфігурацію всіх об'єктів програми, обравши розташування XML файлу на комп'ютері, аналогічно встановити конфігурацію на поточний запуск програми.

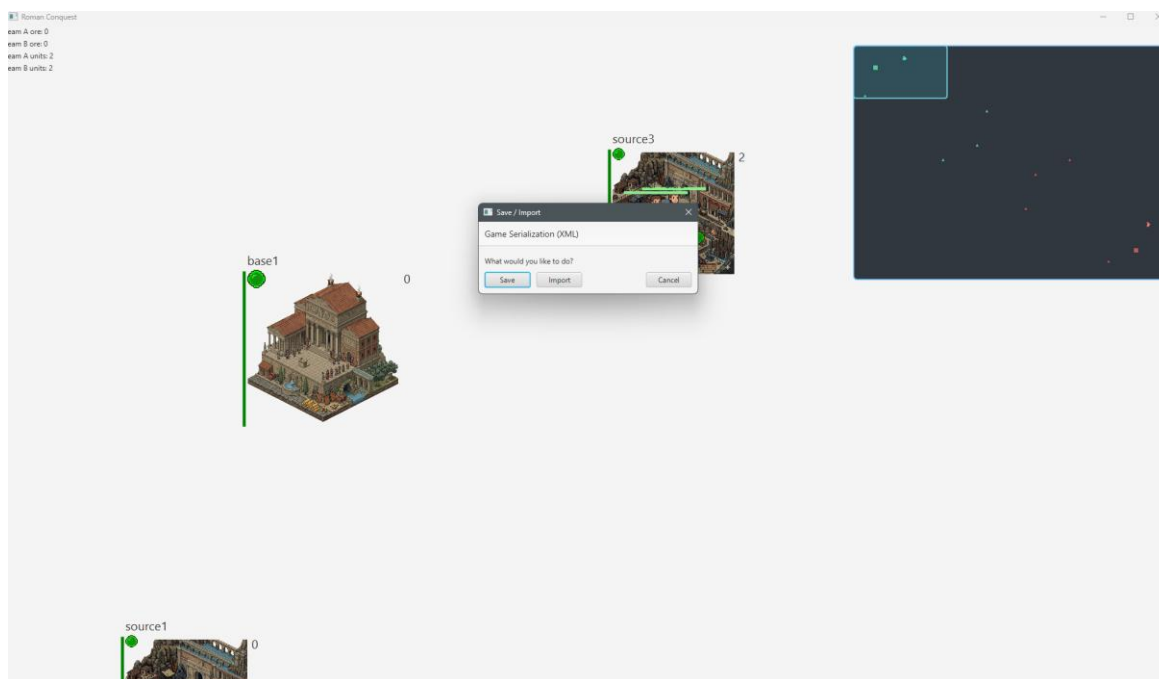


Рисунок 4.5— Інтерфейс керування серіалізації/де-серіалізації

Програма містить ще перелік можливих функцій таких як: копіювання мікрооб'єкта, приховування міні мапи, видалення мікрооб'єкта.

ВИСНОВКИ

Була розроблена програма з графічним інтерфейсом на тему «Roman Conquest», яка симулює і відображає войовничий народ римської імперії, різноманіття ієрархічної системи тогочасної імперії. В ході розробки курсової роботи було проаналізовано проблему зменшення зацікавленості у вивченні історії римської імперії, її здобутків і надзвичайно прогресивною на свій час системою військової справи. Були розглянуті найпопулярніші об'єктно-орієнтовані мови програмування, такі як: Java, C++, C#, в результаті чого було обрано мову Java. Вивчені основні принципи об'єктно-орієнтованого програмування, та способи їх використання мовою Java. Була детально досліджена та вивчена бібліотека JavaFX для створення графічного інтерфейсу користувача.

В процесі роботи над курсовою роботою було отримано навички роботи у середовищі розробки «IntelliJ IDEA», «VSCode» вивчено основні засоби програмування мовою Java. Були набуті навички побудови та відображення основних програмних моментів за допомогою UML-діаграм. Отримано цінний досвід, необхідний при роботі з JavaFX та бібліотеками Java.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Буч Г. Об'єктно-орієнтований аналіз і проектування з прикладами застосування / Г. Буч, Р. Максимчук, М. Енгл. 3-тє вид. Київ : Діалектика, 2019. 720 с.
2. Копейчиков В. В. Історія держави і права зарубіжних країн : навч. посіб. Київ : Юрінком Інтер, 2015. 416 с.
3. Total War: Rome II : official website. URL: <https://www.totalwar.com/games/> (дата звернення: 11.04.2026).
4. Ryse: Son of Rome. Steam : website. URL: https://store.steampowered.com/app/302510/Ryse_Son_of_Rome/ (дата звернення: 30.05.2026).
5. Шилдт Г. Java. Повне керівництво. 11-те вид. Київ : Діалектика, 2020. 1296 с.
6. TIOBE Index for May 2026 : website. TIOBE Software. URL: <https://www.tiobe.com/tiobe-index/> (дата звернення: 30.05.2026).
7. Chin S. Pro JavaFX 9: A Definitive Guide to Building Desktop, Mobile, and Embedded Java Applications / S. Chin, J. Weaver, J. Vos, S. Gao. New York : Apress, 2017. 408 p.
8. OpenJFX. JavaFX Documentation : website. URL: <https://openjfx.io/> (дата звернення: 30.05.2026).
9. Фаулер М. UML. Основи. 3-тє вид. Київ : Креативна книга, 2018. 192 с.
10. Nystrom R. Game Programming Patterns. [б. м.] : Genever Benning, 2014. 356 p.
11. Java XML Technology. Oracle Help Center : website. URL: <https://docs.oracle.com/javase/tutorial/jaxp/> (дата звернення: 29.05.2026).

ДОДАТОК А

Лістинг програми

(обов'язковий)

```
package org.example.laba5;

import java.io.IOException;
import java.net.URL;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.Stream;
import javafx.animation.Animation;
import javafx.animation.KeyFrame;
import javafx.animation.Timeline;
import javafx.application.Application;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.input.KeyCode;
import javafx.scene.input.MouseButton;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;
import javafx.util.Duration;
import org.example.laba5.Unit.Unit;
import org.example.laba5.Unit.Warrior;
import org.example.laba5.Unit.Centurio;
import org.example.laba5.Unit.Pretorio;

public class HelloApplication extends Application {

    private static final String APP_TITLE = "Roman Conquest";

    public static final double WORLD_WIDTH = 6400.0;
    public static final double WORLD_HEIGHT = 4800.0;
    private static final double VIEWPORT_WIDTH = 1920.0;
    private static final double VIEWPORT_HEIGHT = 1080.0;

    public static Group group;
    public static Scene scene;

    public static Image imgWarrior;
    public static Image imgCenturio;
    public static Image imgPretorio;
    public static Image imgBase;
    public static Image imgTower;
    public static Image imgSource;
    public static ArrayList<Warrior> warriors;
    public static ArrayList<Centurio> centurios;
    public static ArrayList<Pretorio> pretorios;
    public static ArrayList<Unit> units;

    private final Map<KeyCode, Double> keysPresses = new HashMap<>();
    private final Map<KeyCode, Boolean> keysPressed = new HashMap<>();
    private final Set<KeyCode> handledActionKeys = new HashSet<>();

    public static double keyStepX = 4.5;
    public static double keyStepY = 4.5;

    public static ArrayList<Tower> towersA = new ArrayList<>();
    public static ArrayList<Source> sourcesA = new ArrayList<>();
    public static ArrayList<Base> basesA = new ArrayList<>();
    public static ArrayList<Tower> towersB = new ArrayList<>();
    public static ArrayList<Source> sourcesB = new ArrayList<>();
    public static ArrayList<Base> basesB = new ArrayList<>();
```

```

public static ArrayList<World> buildings = new ArrayList<>();
private Pane cameraViewport;
private Pane worldLayer;
private Pane overlayPane;
private Label oreLabelTeamA;
private Label oreLabelTeamB;
private UnitInventorWindow unitInventorWindow;
private UnitSearchWindow unitSearchWindow;
private MiniMapOverlay miniMapOverlay;
private double cameraX;
private double cameraY;
private Stage primaryStage;
public static Label numUnitsTeamA;
public static Label numUnitsTeamB;

@Override
public void start(Stage stage) throws IOException {
    group = new Group();
    worldLayer = new Pane();
    worldLayer.setPrefSize(WORLD_WIDTH, WORLD_HEIGHT);
    Rectangle worldBounds = new Rectangle(WORLD_WIDTH, WORLD_HEIGHT);
    worldBounds.setFill(Color.TRANSPARENT);
    worldLayer.getChildren().add(worldBounds);
    group.getChildren().add(worldLayer);

    cameraViewport = new Pane();
    cameraViewport.setPrefSize(VIEWPORT_WIDTH, VIEWPORT_HEIGHT);
    cameraViewport.setMinSize(VIEWPORT_WIDTH, VIEWPORT_HEIGHT);
    cameraViewport.setMaxSize(VIEWPORT_WIDTH, VIEWPORT_HEIGHT);
    cameraViewport.setStyle("-fx-background-color: transparent;");
    cameraViewport.setClip(new Rectangle(VIEWPORT_WIDTH, VIEWPORT_HEIGHT));
    cameraViewport.getChildren().add(group);

    overlayPane = new Pane();
    overlayPane.setPickOnBounds(false);

    Pane root = new Pane(cameraViewport, overlayPane);
    scene = new Scene(root, 1920, 1080);
    unitInventorWindow = new UnitInventorWindow();
    unitSearchWindow = new UnitSearchWindow();

    oreLabelTeamA = new Label();
    oreLabelTeamB = new Label();
    numUnitsTeamA = new Label();
    numUnitsTeamB = new Label();
    oreLabelTeamA.setLayoutX(0);
    oreLabelTeamA.setLayoutY(0);
    oreLabelTeamB.setLayoutX(0);
    oreLabelTeamB.setLayoutY(20);
    numUnitsTeamA.setLayoutX(0);
    numUnitsTeamA.setLayoutY(40);
    numUnitsTeamB.setLayoutX(0);
    numUnitsTeamB.setLayoutY(60);
    overlayPane.getChildren().addAll(oreLabelTeamA, oreLabelTeamB, numUnitsTeamA, numUnitsTeamB);
    keysPresses.put(KeyCode.W, -keyStepY);
    keysPresses.put(KeyCode.S, keyStepY);
    keysPresses.put(KeyCode.A, -keyStepX);
    keysPresses.put(KeyCode.D, keyStepX);
    keysPresses.put(KeyCode.UP, -keyStepY);
    keysPresses.put(KeyCode.DOWN, keyStepY);
    keysPresses.put(KeyCode.LEFT, -keyStepX);
    keysPresses.put(KeyCode.RIGHT, keyStepX);

    cameraX = 0.0;
    cameraY = 0.0;

    scene.setOnKeyPressed(e -> {
        KeyCode code = e.getCode();
        if (code == KeyCode.M) {
            if (miniMapOverlay != null) {
                miniMapOverlay.toggleVisible();
            }
            return;
        }
        if (code == KeyCode.Z && !e.isControlDown()) {
            if (!handledActionKeys.contains(KeyCode.Z)) {
                handledActionKeys.add(KeyCode.Z);
                javafx.application.Platform.runLater(() -> {

```

```

        SerializationDialog.showDialog(primaryStage);
        handledActionKeys.remove(KeyCode.Z);
    });
}
return;
}
if (code == KeyCode.V && e.isControlDown()) {
    if (handledActionKeys.contains(KeyCode.V)) {
        return;
    }
    handledActionKeys.add(KeyCode.V);
    cloneActiveUnit();
    return;
}

if (code == KeyCode.DELETE || code == KeyCode.INSERT || code == KeyCode.ESCAPE || code == KeyCode.SPACE || code == KeyCode.I ||
code == KeyCode.V || code == KeyCode.F) {
    if (handledActionKeys.contains(code)) {
        return;
    }
    handledActionKeys.add(code);
}
keysPressed.put(code, true);
});
scene.setOnKeyReleased(e -> {
    KeyCode code = e.getCode();
    keysPressed.remove(code);
    handledActionKeys.remove(code);
});

URL warriorUrl = HelloApplication.class.getResource("/warrior.png");
URL CenturioUrl = HelloApplication.class.getResource("/centurio.png");
URL PretorioUrl = HelloApplication.class.getResource("/pretorio.png");
URL baseUrl = HelloApplication.class.getResource("/base2.png");
URL towerUrl = HelloApplication.class.getResource("/tower2.png");
URL sourceUrl = HelloApplication.class.getResource("/source2.png");
if (warriorUrl == null) {
    throw new IllegalStateException("Resource not found: /warrior.png");
}
if (CenturioUrl == null) {
    throw new IllegalStateException("Resource not found: /centurio.png");
}
if (PretorioUrl == null) {
    throw new IllegalStateException("Resource not found: /pretorio.png");
}
if (baseUrl == null) {
    throw new IllegalStateException("Resource not found: /base.png");
}
if (towerUrl == null) {
    throw new IllegalStateException("Resource not found: /tower.png");
}
if (sourceUrl == null) {
    throw new IllegalStateException("Resource not found: /source.png");
}

imgBase = new Image(baseUrl.toExternalForm(), 250, 250, false, false);
imgTower = new Image(towerUrl.toExternalForm(), 200, 200, false, false);
imgSource = new Image(sourceUrl.toExternalForm(), 200, 200, false, false);
imgWarrior = new Image(warriorUrl.toExternalForm(), 100, 100, false, false);
imgCenturio = new Image(CenturioUrl.toExternalForm(), 100, 100, false, false);
imgPretorio = new Image(PretorioUrl.toExternalForm(), 100, 100, false, false);
warriors = new ArrayList<>();
centurios = new ArrayList<>();
pretorios = new ArrayList<>();
units = new ArrayList<>();

Warrior warrior1 = new Warrior();
warrior1.setTeam(true);
warriors.add(warrior1);
Warrior warrior2 = new Warrior();
warrior2.setTeam(false);
warriors.add(warrior2);
Warrior warrior3 = new Warrior();
warrior3.setTeam(true);
warriors.add(warrior3);
Warrior warrior4 = new Warrior();
warrior4.setTeam(false);
warriors.add(warrior4);

```

```

units = Stream.of(warriors, centurios, pretorios)
    .filter(Objects::nonNull)
    .flatMap(list -> list.stream())
    .collect(Collectors.toCollection(ArrayList::new));

Base base1 = new Base();
base1.setTeam(true);
base1.initGraphics(imgBase, "base1", 0, 400, 400, 200.0, 200);
base1.resurrectWorld();
basesA.add(base1);
buildings.add(base1);

Source source1 = new Source();
source1.setTeam(true);
source1.initGraphics(imgSource, "source1", 0, 200, 1000, 200.0, 200);
source1.resurrectWorld();
sourcesA.add(source1);
buildings.add(source1);

Source source3 = new Source();
source3.setTeam(true);
source3.initGraphics(imgSource, "source3", 0, 1000, 200, 200.0, 200);
source3.resurrectWorld();
sourcesA.add(source3);
buildings.add(source3);

Tower tower1 = new Tower();
tower1.setTeam(true);
tower1.initGraphics(imgTower, "tower1", 0, 2500, 2000, 300.0, 300);
tower1.resurrectWorld();
towersA.add(tower1);
buildings.add(tower1);
Timeline healTimer1 = new Timeline(new KeyFrame(Duration.seconds(1), e -> tower1.healUnits()));
healTimer1.setCycleCount(Animation.INDEFINITE);
healTimer1.play();

Tower tower3 = new Tower();
tower3.setTeam(true);
tower3.initGraphics(imgTower, "tower3", 0, 1800, 2300, 300.0, 300);
tower3.resurrectWorld();
towersA.add(tower3);
buildings.add(tower3);
Timeline healTimer3 = new Timeline(new KeyFrame(Duration.seconds(1), e -> tower3.healUnits()));
healTimer3.setCycleCount(Animation.INDEFINITE);
healTimer3.play();

Tower tower5 = new Tower();
tower5.setTeam(true);
tower5.initGraphics(imgTower, "tower5", 0, 2700, 1300, 300.0, 300);
tower5.resurrectWorld();
towersA.add(tower5);
buildings.add(tower5);
Timeline healTimer5 = new Timeline(new KeyFrame(Duration.seconds(1), e -> tower5.healUnits()));
healTimer5.setCycleCount(Animation.INDEFINITE);
healTimer5.play();

Base base2 = new Base();
base2.setTeam(false);
base2.initGraphics(imgBase, "base2", 0, 5750, 4150, 200.0, 200);
base2.resurrectWorld();
basesB.add(base2);
buildings.add(base2);

Source source2 = new Source();
source2.setTeam(false);
source2.initGraphics(imgSource, "source2", 0, 6000, 3600, 200.0, 200);
source2.resurrectWorld();
sourcesB.add(source2);
buildings.add(source2);

Source source4 = new Source();
source4.setTeam(false);
source4.initGraphics(imgSource, "source4", 0, 5200, 4400, 200.0, 200);
source4.resurrectWorld();
sourcesB.add(source4);
buildings.add(source4);

Tower tower2 = new Tower();
tower2.setTeam(false);

```

```

tower2.initGraphics(imgTower, "tower2", 0, 3700, 2600, 300.0, 300);
tower2.resurrectWorld();
towersB.add(tower2);
buildings.add(tower2);
Timeline healTimer2 = new Timeline(new KeyFrame(Duration.seconds(1), e -> tower2.healUnits()));
healTimer2.setCycleCount(Animation.INDEFINITE);
healTimer2.play();

Tower tower4 = new Tower();
tower4.setTeam(false);
tower4.initGraphics(imgTower, "tower4", 0, 4400, 2300, 300.0, 300);
tower4.resurrectWorld();
towersB.add(tower4);
buildings.add(tower4);
Timeline healTimer4 = new Timeline(new KeyFrame(Duration.seconds(1), e -> tower4.healUnits()));
healTimer4.setCycleCount(Animation.INDEFINITE);
healTimer4.play();

Tower tower6 = new Tower();
tower6.setTeam(false);
tower6.initGraphics(imgTower, "tower6", 0, 3500, 3300, 300.0, 300);
tower6.resurrectWorld();
towersB.add(tower6);
buildings.add(tower6);
Timeline healTimer6 = new Timeline(new KeyFrame(Duration.seconds(1), e -> tower6.healUnits()));
healTimer6.setCycleCount(Animation.INDEFINITE);
healTimer6.play();

units.stream().filter(Objects::nonNull).forEach(unit -> {
    if (unit.getTeam()) {
        unit.resurrect();
        unit.setPosition(basesA.get(0).x, basesA.get(0).y);
    } else {
        unit.resurrect();
        unit.setPosition(basesB.get(0).x, basesB.get(0).y);
    }
});

World world = new World(units);
double initialW = scene != null && scene.getWidth() > 0 ? scene.getWidth() : VIEWPORT_WIDTH;
double initialH = scene != null && scene.getHeight() > 0 ? scene.getHeight() : VIEWPORT_HEIGHT;
miniMapOverlay = new MiniMapOverlay(WORLD_WIDTH, WORLD_HEIGHT, initialW, initialH, target -> {
    double currentViewportWidth = scene != null && scene.getWidth() > 0 ? scene.getWidth() : VIEWPORT_WIDTH;
    double currentViewportHeight = scene != null && scene.getHeight() > 0 ? scene.getHeight() : VIEWPORT_HEIGHT;
    updateCameraPosition(target[0] - currentViewportWidth / 2.0, target[1] - currentViewportHeight / 2.0);
});
miniMapOverlay.bindToScene(scene);
overlayPane.getChildren().add(miniMapOverlay.getPane());

scene.setOnMouseClicked(event -> {
    if (miniMapOverlay != null) {
        double miniMapX = miniMapOverlay.getPane().getLayoutX();
        double miniMapY = miniMapOverlay.getPane().getLayoutY();
        double miniMapWidth = miniMapOverlay.getPane().getPrefWidth();
        double miniMapHeight = miniMapOverlay.getPane().getPrefHeight();
        if (event.getX() >= miniMapX && event.getX() <= miniMapX + miniMapWidth && event.getY() >= miniMapY && event.getY() <=
miniMapY + miniMapHeight) {
            return;
        }
    }
});

double worldMouseX = event.getX() + cameraX;
double worldMouseY = event.getY() + cameraY;

if (event.getButton() == MouseButton.PRIMARY) {
    for (int i = units.size() - 1; i >= 0; i--) {
        Unit unit = units.get(i);
        if (unit.tryActivate(worldMouseX, worldMouseY)) {
            break;
        }
    }
} else if (event.getButton() == MouseButton.SECONDARY) {
    for (int i = units.size() - 1; i >= 0; i--) {
        Unit unit = units.get(i);
        if (unit.getImage().getBoundsInParent().contains(worldMouseX, worldMouseY)) {
            javafx.application.Platform.runLater() -> {
                UnitEditDialog editDialog = new UnitEditDialog(unit);
                editDialog.showAndWait();
            };
        }
    }
}

```

```

        break;
    }
}
});

javafx.animation.AnimationTimer gameLoop = new javafx.animation.AnimationTimer() {
    @Override
    public void handle(long now) {
        double dx = 0, dy = 0;
        double camDx = 0, camDy = 0;
        double camSpeedMultiplier = 4.5;

        for (KeyCode code : new ArrayList<>(keysPressed.keySet())) {
            if (keysPresses.containsKey(code)) {
                double step = keysPresses.get(code);
                if (code == KeyCode.W) {
                    dy += step;
                } else if (code == KeyCode.S) {
                    dy -= step;
                } else if (code == KeyCode.A) {
                    dx += step;
                } else if (code == KeyCode.D) {
                    dx -= step;
                } else if (code == KeyCode.UP) {
                    camDy += step * camSpeedMultiplier;
                } else if (code == KeyCode.DOWN) {
                    camDy -= step * camSpeedMultiplier;
                } else if (code == KeyCode.LEFT) {
                    camDx += step * camSpeedMultiplier;
                } else if (code == KeyCode.RIGHT) {
                    camDx -= step * camSpeedMultiplier;
                }
            } else if (code == KeyCode.DELETE) {
                List<Unit> activeUnits = units.stream().filter(Unit::isActive).collect(Collectors.toList());
                activeUnits.forEach(Unit::removeUnitFromGame);
                keysPressed.remove(KeyCode.DELETE);
            } else if (code == KeyCode.INSERT) {
                javafx.application.Platform.runLater(() -> {
                    UnitCreationDialog dialog = new UnitCreationDialog();
                    Unit newUnit = dialog.showAndWait();
                    double x;
                    double y;
                    if (newUnit != null) {
                        units.add(newUnit);
                        boolean team = newUnit.getTeam();
                        if (team) {
                            x = basesA.get(0).x;
                            y = basesA.get(0).y;
                        } else {
                            x = basesB.get(0).x;
                            y = basesB.get(0).y;
                        }
                        newUnit.setPosition(x, y);
                        newUnit.resurrect();
                        newUnit.spawnAtTeamBase();
                    }
                });
                keysPressed.remove(KeyCode.INSERT);
            } else if (code == KeyCode.ESCAPE) {
                for (Unit unit : units) {
                    if (unit.isActive()) {
                        unit.flipActivation();
                    }
                }
                keysPressed.remove(KeyCode.ESCAPE);
            } else if (code == KeyCode.SPACE) {
                for (Unit unit : units) {
                    if (unit.isActive()) {
                        unit.attack();
                    }
                }
                keysPressed.remove(KeyCode.SPACE);
            } else if (code == KeyCode.I) {
                if (unitInventorWindow.isVisible()) {
                    unitInventorWindow.setVisible(false);
                } else {
                    unitInventorWindow.showForActiveUnit(units);
                }
            }
        }
    }
};

```

```

        keysPressed.remove(KeyCode.I);
    } else if (code == KeyCode.F) {
        if (unitSearchWindow.isVisible()) {
            unitSearchWindow.setVisible(false);
        } else {
            unitSearchWindow.show();
        }
        keysPressed.remove(KeyCode.F);
    } else if (code == KeyCode.V) {
        for (Unit unit : units) {
            if (unit.isActive() && unit.getClass() == Warrior.class) {
                Warrior w = (Warrior) unit;
                w.setInverseMode(!w.isInverseMode());
            }
        }
        keysPressed.remove(KeyCode.V);
    }
}

if (camDx != 0 || camDy != 0) {
    updateCameraPosition(cameraX + camDx, cameraY + camDy);
}

for (World world : new ArrayList<>(buildings)) {
    if (world.getHealth() <= 0) {
        world.removeBuildingFromGame();
    }
}

updateHud();
world.worldLogic();
miniMapOverlay.update(units, buildings);

for (World building : new ArrayList<>(buildings)) {
    building.intersect();
}

for (Unit unit : new ArrayList<>(units)) {
    if (unit.isActive()) {
        unit.move(dx, dy);
    }
    if (unit instanceof Warrior) {
        Warrior w = (Warrior) unit;
        if (w.isInverseMode()) {
            w.logicInverse();
        } else {
            w.logic();
        }
    } else {
        unit.logic();
    }
}

unitInventorWindow.updateFromUnits(units);
}
};
gameLoop.start();

primaryStage = stage;
stage.setTitle(APP_TITLE);
stage.setScene(scene);
stage.show();

scene.widthProperty().addListener((obs, oldVal, newVal) -> {
    double w = newVal.doubleValue();
    cameraViewport.setPrefWidth(w);
    cameraViewport.setMinWidth(w);
    cameraViewport.setMaxWidth(w);
    if (cameraViewport.getClip() instanceof Rectangle) {
        ((Rectangle) cameraViewport.getClip()).setWidth(w);
    }
    if (miniMapOverlay != null) {
        miniMapOverlay.setViewportSize(w, scene.getHeight());
    }
    updateCameraPosition(cameraX, cameraY);
});
scene.heightProperty().addListener((obs, oldVal, newVal) -> {
    double h = newVal.doubleValue();
    cameraViewport.setPrefHeight(h);

```

```

        cameraViewport.setMinHeight(h);
        cameraViewport.setMaxHeight(h);
        if (cameraViewport.getClip() instanceof Rectangle) {
            ((Rectangle) cameraViewport.getClip()).setHeight(h);
        }
        if (miniMapOverlay != null) {
            miniMapOverlay.setViewportSize(scene.getWidth(), h);
        }
        updateCameraPosition(cameraX, cameraY);
    });
}

private void cloneActiveUnit() {
    Unit sourceUnit = null;
    for (int i = units.size() - 1; i >= 0; i--) {
        Unit candidate = units.get(i);
        if (candidate != null && candidate.isActive()) {
            sourceUnit = candidate;
            break;
        }
    }

    if (sourceUnit == null) {
        handledActionKeys.remove(KeyCode.V);
        return;
    }

    try {
        Unit clonedUnit = (Unit) sourceUnit.clone();
        units.add(clonedUnit);
        clonedUnit.setPosition(sourceUnit.x + 20, sourceUnit.y + 20);
        clonedUnit.resurrect();
    } catch (CloneNotSupportedException ex) {
    }
}

private void updateHud() {
    double teamAOre = basesA.stream().filter(Objects::nonNull).mapToDouble(Base::getOre).sum();
    double teamBOre = basesB.stream().filter(Objects::nonNull).mapToDouble(Base::getOre).sum();
    oreLabelTeamA.setText("Team A ore: " + (int) teamAOre);
    oreLabelTeamB.setText("Team B ore: " + (int) teamBOre);
}

private void updateCameraPosition(double newCameraX, double newCameraY) {
    double currentViewportWidth = scene != null && scene.getWidth() > 0 ? scene.getWidth() : VIEWPORT_WIDTH;
    double currentViewportHeight = scene != null && scene.getHeight() > 0 ? scene.getHeight() : VIEWPORT_HEIGHT;
    double maxCameraX = Math.max(0.0, WORLD_WIDTH - currentViewportWidth);
    double maxCameraY = Math.max(0.0, WORLD_HEIGHT - currentViewportHeight);
    cameraX = Math.max(0.0, Math.min(newCameraX, maxCameraX));
    cameraY = Math.max(0.0, Math.min(newCameraY, maxCameraY));
    group.setLayoutX(-cameraX);
    group.setLayoutY(-cameraY);
    if (miniMapOverlay != null) {
        miniMapOverlay.setCameraPosition(cameraX, cameraY);
    }
}
}

```

Лістинг модуля Unit.java

```

package org.example.laba5.Unit;

import java.net.URL;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.Objects;
import javafx.scene.shape.Rectangle;
import javafx.scene.shape.Line;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import org.example.laba5.HelloApplication;
import org.example.laba5.World;

public class Unit implements Cloneable {
    private Integer health;
    private Boolean isSpawned;
    protected boolean team;
    private Integer damage;
    private Boolean isDead;
    private ArrayList<String> inventor;

    private Integer baseHealth;
    private Integer baseDamage;

    private static int numObjects = 0;
    private static int objectedKilled = 0;

    public Label labelName;
    public Line life;
    public ImageView image;
    public double x, y;
    public boolean isActive;
    public Rectangle rectActive;
    public double maxHealth;
    public ImageView imageMarkRed;
    public ImageView imageMarkGreen;
    public ImageView imageMark;
    public ImageView swordImage;
    public ImageView knifeImage;
    public ImageView spearImage;
    public ImageView bowImage;
    public ImageView mainWeaponImage;

    protected double moveSpeed = 1;
    private static boolean isPushing = false;

    protected long lastAttackTime = 0;
    protected final long ATTACK_COOLDOWN = 1000;

    URL swordUrl = getClass().getResource("/sword.png");
    URL knifeUrl = getClass().getResource("/knife.png");
    URL spearUrl = getClass().getResource("/spear.png");
    URL bowUrl = getClass().getResource("/bow.png");

    public static void plusObjectedKilled() {
        objectedKilled++;
    }

    public void setMaxHealth(double maxHealth) {
        this.maxHealth = maxHealth;
    }

    public double getMaxHealth() {
        return this.maxHealth;
    }

    protected double labelDeltaX() {
        return 10.0;
    }

    protected double labelDeltaY() {
        return -10.0;
    }
}

```

```

protected double lifeDeltaX() {
    return 0.0;
}

protected double lifeDeltaY() {
    return 10.0;
}

protected double imageDeltaX() {
    return 0.0;
}

protected double imageDeltaY() {
    return 0.0;
}

protected double rectDeltaX() {
    return -9.0;
}

protected double rectDeltaY() {
    return -9.0;
}

public boolean isActive() {
    return isActive;
}

public Unit(Integer health, Boolean isSpawned, boolean team, Integer damage, Boolean isDead, ArrayList<String> inventor) {
    this.health = health;
    this.baseHealth = health;
    this.isSpawned = isSpawned;
    this.team = team;
    this.damage = damage;
    this.baseDamage = damage;
    this.isDead = isDead;
    this.inventor = inventor;
    numObjects++;
}

public Unit() {
    this(100, false, true, 5, false, new ArrayList<String>(Arrays.asList("sword")));
}

public boolean getActive() {
    return isActive;
}

public static int getNumObjects() {
    return numObjects;
}

public static void setNumObjects(int num) {
    numObjects = num;
}

public Integer getHealth() {
    return health;
}

public Boolean getSpawned() {
    return isSpawned;
}

public boolean getTeam() {
    return team;
}

public Integer getDamage() {
    return damage;
}

public Boolean getDead() {
    return isDead;
}

public ArrayList<String> getInventor() {
    return inventor;
}

```

```

    }

    public static int getnumObjects() {
        return numObjects;
    }

    public javafx.scene.image.ImageView getImage() {
        return image;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    public static class HealthComparator implements Comparator<Unit> {
        @Override
        public int compare(Unit a, Unit b) {
            return Integer.compare(a.getHealth(), b.getHealth());
        }
    }

    public static class TeamComparator implements Comparator<Unit> {
        @Override
        public int compare(Unit a, Unit b) {
            return Boolean.compare(a.getTeam(), b.getTeam());
        }
    }

    public int compareTo47(Unit x) {
        int cmp = Integer.compare(this.health, x.health);
        if (cmp != 0) return cmp;
        cmp = Boolean.compare(this.team, x.team);
        if (cmp != 0) return cmp;
        cmp = this.damage.compareTo(x.damage);
        if (cmp != 0) return cmp;
        cmp = this.isSpawned.compareTo(x.isSpawned);
        if (cmp != 0) return cmp;
        cmp = this.isDead.compareTo(x.isDead);
        if (cmp != 0) return cmp;
        ArrayList<String> sortedInventor1 = new ArrayList<>(this.inventor);
        ArrayList<String> sortedInventor2 = new ArrayList<>(x.inventor);
        sortedInventor1.sort(String::compareTo);
        sortedInventor2.sort(String::compareTo);
        cmp = sortedInventor1.toString().compareTo(sortedInventor2.toString());
        if (cmp != 0) return cmp;
        return 0;
    }

    public static Comparator<Unit> comparatorFromTemplate(Unit template) {
        return (u1, u2) -> {
            if (template == null) {
                throw new IllegalArgumentException("Template cannot be null");
            }
            int cmp;
            if (template.getHealth() != null) {
                cmp = Integer.compare(u1.getHealth(), u2.getHealth());
                if (cmp != 0) return cmp;
            }
            cmp = Boolean.compare(u1.getTeam(), u2.getTeam());
            if (cmp != 0) return cmp;
            if (template.getDamage() != null) {
                cmp = u1.getDamage().compareTo(u2.getDamage());
                if (cmp != 0) return cmp;
            }
            if (template.getSpawned() != null) {
                cmp = u1.getSpawned().compareTo(u2.getSpawned());
                if (cmp != 0) return cmp;
            }
            if (template.getDead() != null) {
                cmp = u1.getDead().compareTo(u2.getDead());
                if (cmp != 0) return cmp;
            }
            if (template.getInventor() != null) {
                ArrayList<String> sortedInventor1 = new ArrayList<>(u1.getInventor());
                ArrayList<String> sortedInventor2 = new ArrayList<>(u2.getInventor());
            }
        };
    }

```

```

        sortedInventor1.sort(String::compareTo);
        sortedInventor2.sort(String::compareTo);
        cmp = sortedInventor1.toString().compareTo(sortedInventor2.toString());
        if (cmp != 0) return cmp;
    }
    return 0;
};
}

public void setHealth(Integer health) {
    this.health = health;
    if (this.health != null && this.health < 0) {
        removeUnitFromGame();
        return;
    }
    if (life != null) {
        setCoordinates();
    }
}

public void removeUnitFromGame() {
    this.isDead = true;
    if (HelloApplication.group != null) {
        if (this.labelName != null) {
            HelloApplication.group.getChildren().remove(this.labelName);
        }
        if (this.life != null) {
            HelloApplication.group.getChildren().remove(this.life);
        }
        if (this.image != null) {
            HelloApplication.group.getChildren().remove(this.image);
        }
        if (this.rectActive != null) {
            HelloApplication.group.getChildren().remove(this.rectActive);
        }
        if (this.imageMark != null) {
            HelloApplication.group.getChildren().remove(this.imageMark);
        }
        if (this.mainWeaponImage != null) {
            HelloApplication.group.getChildren().remove(this.mainWeaponImage);
        }
        if (this.getOreCountLabel() != null) {
            HelloApplication.group.getChildren().remove(this.getOreCountLabel());
        }
        if (this.getKillCountLabel() != null) {
            HelloApplication.group.getChildren().remove(this.getKillCountLabel());
        }
    }
    if (HelloApplication.units != null) {
        HelloApplication.units.remove(this);
    }
    removeUnit();
}

public void setSpawned(Boolean spawned) {
    isSpawned = spawned;
}

public void setTeam(boolean team) {
    this.team = team;
}

public void setDamage(Integer damage) {
    this.damage = damage;
}

public void setBaseHealth(Integer baseHealth) {
    this.baseHealth = baseHealth;
}

public void setBaseDamage(Integer baseDamage) {
    this.baseDamage = baseDamage;
}

public Integer getBaseHealth() {
    return baseHealth;
}

public Integer getBaseDamage() {

```

```

        return baseDamage;
    }

    public void setDead(Boolean dead) {
        isDead = dead;
    }

    public void setInventor(ArrayList<String> inventor) {
        this.inventor = inventor;
        inventoryLogic();
    }

    @Override
    public boolean equals(Object o) {
        if (o == null || getClass() != o.getClass()) return false;
        Unit unit = (Unit) o;
        return Objects.equals(health, unit.health) && Objects.equals(isSpawned, unit.isSpawned) && team == unit.team && Objects.equals(damage,
unit.damage) && Objects.equals(isDead, unit.isDead) && Objects.equals(inventor, unit.inventor);
    }

    @Override
    public int hashCode() {
        return Objects.hash(health, isSpawned, team, damage, isDead, inventor);
    }

    @Override
    public String toString() {
        return "Unit{" +
            "health=" + health +
            ", isSpawned=" + isSpawned +
            ", Team=" + team + "\"" +
            ", damage=" + damage +
            ", isDead=" + isDead +
            ", inventor=" + inventor +
            "}\n";
    }

    @Override
    public Unit clone() throws CloneNotSupportedException {
        Unit clonedUnit = (Unit) super.clone();
        ArrayList<String> clonedInventor = this.inventor;

        if (this.labelName != null) {
            clonedUnit.labelName = new Label(this.labelName.getText());
        }
        if (this.life != null) {
            clonedUnit.life = new Line();
            clonedUnit.life.setStrokeWidth(this.life.getStrokeWidth());
            clonedUnit.life.setStroke(this.life.getStroke());
        }
        if (this.image != null) {
            clonedUnit.image = new ImageView(this.image.getImage());
            clonedUnit.image.setFitWidth(this.image.getFitWidth());
            clonedUnit.image.setFitHeight(this.image.getFitHeight());
        }
        if (this.rectActive != null) {
            double rectWidth = this.rectActive.getWidth();
            double rectHeight = this.rectActive.getHeight();
            clonedUnit.rectActive = new Rectangle(rectWidth, rectHeight);
            clonedUnit.rectActive.setFill(this.rectActive.getFill());
            clonedUnit.rectActive.setStroke(this.rectActive.getStroke());
            clonedUnit.rectActive.setStrokeWidth(this.rectActive.getStrokeWidth());
        }
        if (this.imageMarkRed != null) {
            clonedUnit.imageMarkRed = new ImageView(this.imageMarkRed.getImage());
            clonedUnit.imageMarkRed.setFitWidth(this.imageMarkRed.getFitWidth());
            clonedUnit.imageMarkRed.setFitHeight(this.imageMarkRed.getFitHeight());
        }
        if (this.imageMarkGreen != null) {
            clonedUnit.imageMarkGreen = new ImageView(this.imageMarkGreen.getImage());
            clonedUnit.imageMarkGreen.setFitWidth(this.imageMarkGreen.getFitWidth());
            clonedUnit.imageMarkGreen.setFitHeight(this.imageMarkGreen.getFitHeight());
        }
        if (this.mainWeaponImage != null) {
            clonedUnit.mainWeaponImage = new ImageView(this.mainWeaponImage.getImage());
            clonedUnit.mainWeaponImage.setFitWidth(this.mainWeaponImage.getFitWidth());
            clonedUnit.mainWeaponImage.setFitHeight(this.mainWeaponImage.getFitHeight());
        }
    }

```

```

        clonedUnit.baseHealth = this.baseHealth;
        clonedUnit.baseDamage = this.baseDamage;
        clonedUnit.setInventor(clonedInventor);
        clonedUnit.isActive = false;
        clonedUnit.maxHealth = this.maxHealth;
        numObjects++;
        return clonedUnit;
    }

    public void attack() {
        boolean intersects = false;
        if (this.damage == null || this.damage <= 0) {
            return;
        }
        if (this.image == null) {
            return;
        }
        if (HelloApplication.units != null) {
            for (Unit unit : HelloApplication.units) {
                if (unit != this && unit.getTeam() != this.team && !unit.getDead() && unit.image != null) {
                    intersects = this.image.getBoundsInParent().intersects(unit.image.getBoundsInParent());
                    if (intersects) {
                        int targetHealth = unit.getHealth() == null ? 0 : unit.getHealth();
                        int newHealth = targetHealth - this.damage;
                        unit.setHealth(newHealth);
                        if (newHealth <= 0) {
                            unit.setHealth(0);
                            objectedKilled++;
                            unit.removeUnitFromGame();
                        }
                        break;
                    }
                }
            }
        }
        if (HelloApplication.buildings != null) {
            for (World world : HelloApplication.buildings) {
                if (world != null && world.getTeam() != this.team && world.getImageView() != null) {
                    intersects = this.image.getBoundsInParent().intersects(world.getImageView().getBoundsInParent());
                    if (intersects) {
                        int targetHealth = world.getHealth() == 0 ? 0 : (int) world.getHealth();
                        int newHealth = targetHealth - this.damage;
                        world.setHealth(newHealth);
                    }
                }
            }
        }
    }

    public static void removeUnit() {
        numObjects--;
    }

    public void addHealth() {
        this.health += 10;
    }

    public void addDamage() {
        this.damage += 5;
    }

    public void changeTeam() {
        this.team = !this.team;
    }

    public boolean isAlly() {
        return this.team;
    }

    public boolean isAlly(Unit x) {
        return x != null && this.team == x.team;
    }

    public void takeDamage() {
        this.health -= 10;
    }

    public void showTheStrongest(Unit unit) {
        Unit maxhealth = (this.health > unit.health) ? this : unit;
    }

```

```

    }

    public void spawnAtTeamBase() {
        if (this.getTeam()) {
            if (HelloApplication.basesA != null && !HelloApplication.basesA.isEmpty()) {
                setPosition(HelloApplication.basesA.get(0).getX(), HelloApplication.basesA.get(0).getY());
            }
        } else {
            if (HelloApplication.basesB != null && !HelloApplication.basesB.isEmpty()) {
                setPosition(HelloApplication.basesB.get(0).getX(), HelloApplication.basesB.get(0).getY());
            }
        }
    }

    public void resurrect() {
        if (HelloApplication.group == null || labelName == null || life == null || image == null) {
            return;
        }
        loadMarkImages();
        if (this.getTeam()) {
            imageMark = imageMarkGreen;
        } else {
            imageMark = imageMarkRed;
        }
        HelloApplication.group.getChildren().addAll(labelName, life, image);
        if (imageMark != null) {
            HelloApplication.group.getChildren().add(imageMark);
        }
        if (mainWeaponImage != null) {
            HelloApplication.group.getChildren().add(mainWeaponImage);
        }
        if (isActive) {
            HelloApplication.group.getChildren().add(rectActive);
        }
        setCoordinates();
        inventoryLogic();
    }

    public void loadInventoryImages() {
        try {
            if (swordImage == null && swordUrl != null) {
                swordImage = new ImageView(new Image(swordUrl.toExternalForm()));
                swordImage.setFitWidth(40);
                swordImage.setFitHeight(40);
                swordImage.setPreserveRatio(true);
            }
            if (knifeImage == null && knifeUrl != null) {
                knifeImage = new ImageView(new Image(knifeUrl.toExternalForm()));
                knifeImage.setFitWidth(40);
                knifeImage.setFitHeight(40);
                knifeImage.setPreserveRatio(true);
            }
            if (spearImage == null && spearUrl != null) {
                spearImage = new ImageView(new Image(spearUrl.toExternalForm()));
                spearImage.setFitWidth(40);
                spearImage.setFitHeight(40);
                spearImage.setPreserveRatio(true);
            }
            if (bowImage == null && bowUrl != null) {
                bowImage = new ImageView(new Image(bowUrl.toExternalForm()));
                bowImage.setFitWidth(40);
                bowImage.setFitHeight(40);
                bowImage.setPreserveRatio(true);
            }
        } catch (Exception e) {
        }
    }

    public void loadMarkImages() {
        if (imageMarkRed != null && imageMarkGreen != null) {
            return;
        }
        try {
            URL redUrl = getClass().getResource("/red.png");
            URL greenUrl = getClass().getResource("/green.png");
            if (redUrl != null) {
                imageMarkRed = new ImageView(new Image(redUrl.toExternalForm()));
                imageMarkRed.setFitWidth(20);
                imageMarkRed.setFitHeight(20);
            }
        }
    }

```

```

    }
    if (greenUrl != null) {
        imageMarkGreen = new ImageView(new Image(greenUrl.toExternalForm()));
        imageMarkGreen.setFitWidth(20);
        imageMarkGreen.setFitHeight(20);
    }
} catch (Exception e) {
}
}

public void clampBounds() {
    double maxW = HelloApplication.WORLD_WIDTH;
    double maxH = HelloApplication.WORLD_HEIGHT;
    double fitW = image != null ? image.getFitWidth() : 100.0;
    double fitH = image != null ? image.getFitHeight() : 100.0;

    if (x < 0) x = 0;
    if (x > maxW - fitW) x = maxW - fitW;
    if (y < 0) y = 0;
    if (y > maxH - fitH) y = maxH - fitH;
}

public void setCoordinates() {
    clampBounds();
    if (labelName == null || life == null || image == null) {
        return;
    }
    labelName.setLayoutX(x + labelDeltaX());
    labelName.setLayoutY(y + labelDeltaY());

    double hp = getHealth() == null ? 0.0 : Math.max(0.0, getHealth());
    double effectiveMaxHealth = maxHealth > 0.0 ? maxHealth : 100.0;
    double lifeBaseX = x + lifeDeltaX();
    double lifeBaseY = y + lifeDeltaY();
    life.setStartX(lifeBaseX);
    life.setStartY(lifeBaseY);
    life.setEndX(lifeBaseX + Math.min((hp / effectiveMaxHealth) * 100, 100));
    life.setEndY(lifeBaseY);

    image.setX(x + imageDeltaX());
    image.setY(y + imageDeltaY());

    if (rectActive != null) {
        rectActive.setX(x + rectDeltaX());
        rectActive.setY(y + rectDeltaY());
    }

    if (imageMark != null) {
        imageMark.setX(x + image.getFitWidth() - 20);
        imageMark.setY(y + image.getFitHeight() - 20);
    }
    if (mainWeaponImage != null) {
        mainWeaponImage.setX(x);
        mainWeaponImage.setY(y + 10);
    }
}

public void move(double dx, double dy) {
    x += dx;
    y += dy;
    clampBounds();
    setCoordinates();
    locateAndRotateF();
    locateAndRotateE();
}

public void locateAndRotateF() {
    if (isPushing) {
        return;
    }
    if (HelloApplication.units == null || image == null) {
        return;
    }
    try {
        isPushing = true;
        double selfCenterX = x + image.getFitWidth() / 2.0;
        double selfCenterY = y + image.getFitHeight() / 2.0;
        double minDistance = 30.0;
        double pushDistance = 50.0;

```

```

for (Unit unit : HelloApplication.units) {
    if (unit == this || unit.image == null) {
        continue;
    }
    if (unit.getTeam() != this.team) {
        continue;
    }
    double otherCenterX = unit.x + unit.image.getFitWidth() / 2.0;
    double otherCenterY = unit.y + unit.image.getFitHeight() / 2.0;
    double dx = otherCenterX - selfCenterX;
    double dy = otherCenterY - selfCenterY;
    double dist = Math.hypot(dx, dy);

    if (dist < minDistance) {
        if (dist < 0.001) {
            dx = 1.0;
            dy = 0.0;
            dist = 1.0;
        }
        double newX = selfCenterX + (dx / dist) * pushDistance;
        double newY = selfCenterY + (dy / dist) * pushDistance;
        unit.moveTo(newX - unit.image.getFitWidth() / 2.0, newY - unit.image.getFitHeight() / 2.0);
    }
}
} finally {
    isPushing = false;
}
}

public void locateAndRotateE() {
}

public void moveTo(double newX, double newY) {
    double dx = newX - x;
    double dy = newY - y;
    double dist = Math.hypot(dx, dy);

    if (dist <= moveSpeed) {
        x = newX;
        y = newY;
    } else {
        move(dx / dist * moveSpeed, dy / dist * moveSpeed);
        return;
    }
    clampBounds();
    locateAndRotateE();
    locateAndRotateF();
    setCoordinates();
}

public void setPosition(double newX, double newY) {
    x = newX;
    y = newY;
    clampBounds();
    locateAndRotateE();
    locateAndRotateF();
    setCoordinates();
}

public boolean flipActivation() {
    if (HelloApplication.group != null) {
        if (isActive) {
            HelloApplication.group.getChildren().remove(rectActive);
        } else {
            HelloApplication.group.getChildren().add(rectActive);
        }
    }
    isActive = !isActive;
    return isActive;
}

public boolean tryActivate(double mx, double my) {
    if (image.getBoundsInParent().contains(mx, my)) {
        flipActivation();
        return true;
    }
    return false;
}
}

```

```

public void updateTeamMark() {
    loadMarkImages();
    if (imageMark != null && HelloApplication.group.getChildren().contains(imageMark)) {
        HelloApplication.group.getChildren().remove(imageMark);
    }
    if (this.team) {
        imageMark = imageMarkGreen;
    } else {
        imageMark = imageMarkRed;
    }
    if (imageMark != null) {
        HelloApplication.group.getChildren().add(imageMark);
    }
    setCoordinates();
}

public void logic() {
}

protected void inventoryLogic() {
    loadInventoryImages();
    if (HelloApplication.group != null && mainWeaponImage != null && HelloApplication.group.getChildren().contains(mainWeaponImage)) {
        HelloApplication.group.getChildren().remove(mainWeaponImage);
    }
    mainWeaponImage = null;

    if (baseHealth != null) {
        this.health = baseHealth;
    }
    if (baseDamage != null) {
        this.damage = baseDamage;
    }
    this.maxHealth = baseHealth != null ? baseHealth : 100.0;

    if (this.inventor == null || this.inventor.isEmpty()) {
        setCoordinates();
        return;
    }

    String mainWeapon = this.inventor.get(0);
    int currentDamage = this.getDamage() != null ? this.getDamage() : 0;

    if (mainWeapon.equalsIgnoreCase("sword")) {
        this.damage = currentDamage + 5;
        if (swordImage != null && swordImage.getImage() != null) mainWeaponImage = new ImageView(swordImage.getImage());
    } else if (mainWeapon.equalsIgnoreCase("knife")) {
        this.damage = currentDamage + 3;
        if (knifeImage != null && knifeImage.getImage() != null) mainWeaponImage = new ImageView(knifeImage.getImage());
    } else if (mainWeapon.equalsIgnoreCase("spear")) {
        this.damage = currentDamage + 4;
        if (spearImage != null && spearImage.getImage() != null) mainWeaponImage = new ImageView(spearImage.getImage());
    } else if (mainWeapon.equalsIgnoreCase("bow")) {
        this.damage = currentDamage + 2;
        if (bowImage != null && bowImage.getImage() != null) mainWeaponImage = new ImageView(bowImage.getImage());
    }

    if (mainWeaponImage != null) {
        mainWeaponImage.setFitWidth(40);
        mainWeaponImage.setFitHeight(40);
        mainWeaponImage.setPreserveRatio(true);
        if (HelloApplication.group != null) HelloApplication.group.getChildren().add(mainWeaponImage);
    }

    for (int i = 1; i < this.inventor.size(); i++) {
        String item = this.inventor.get(i);
        if (item == null) continue;
        String it = item.trim().toLowerCase();
        if (it.equals("health potion") || it.equals("health_potion") || it.equals("health")) {
            this.maxHealth += 20;
            Integer curHp = this.getHealth();
            this.health = (curHp == null ? 20 : curHp + 20);
        } else if (it.equals("damage potion") || it.equals("damage_potion") || it.equals("damage")) {
            Integer curDmg = this.getDamage();
            this.damage = (curDmg == null ? 0 : curDmg + 5);
        }
    }
    setCoordinates();
}

```

```

protected void promotion() {
}

public javafx.scene.control.Label getOreCountLabel() {
    return null;
}

public void setOreCount(int oreCount) {
}

public javafx.scene.control.Label getKillCountLabel() {
    return null;
}

protected void logicInverse() {
}

public boolean moreThanHalf() {
    return this.health != null && this.maxHealth > 0 && this.health > this.maxHealth / 2;
}

public boolean haveSword() {
    if (this.inventor != null) {
        for (String item : this.inventor) {
            if (item != null && item.trim().equalsIgnoreCase("sword")) {
                return true;
            }
        }
    }
    return false;
}
}

```

Лістинг модуля Warrior.java

```

package org.example.laba5.Unit;

import java.util.ArrayList;
import java.util.Arrays;
import javafx.scene.control.Label;
import javafx.scene.image.ImageView;
import javafx.scene.paint.Color;
import javafx.scene.shape.Line;
import javafx.scene.shape.Rectangle;
import org.example.laba5.Base;
import org.example.laba5.HelloApplication;
import org.example.laba5.Source;
import org.example.laba5.World;

public class Warrior extends Unit {
    private static String name = "Warrior";
    private static final double MAX_HEALTH = 100.0;
    private double oreAmount = 0;
    private int activeOre = 0;
    private static final long ORE_COOLDOWN = 1000;
    private long lastOreTime = 0;
    private boolean collectingOre = false;
    private boolean deliveringOre = false;
    private static final double COMBAT_PRIORITY_RADIUS = 180.0;

    private Label oreCountLabel;
    private boolean inverseMode = false;

    @Override
    public Label getOreCountLabel() {
        return oreCountLabel;
    }

    @Override
    public void setOreCount(int oreCount) {
        this.oreAmount = oreCount;
    }

    @Override
    protected double labelDeltaX() {
        return 10.0;
    }

    public double getOre() {
        return oreAmount;
    }

    @Override
    protected double labelDeltaY() {
        return -10.0;
    }

    @Override
    protected double lifeDeltaX() {
        return 0.0;
    }

    @Override
    protected double lifeDeltaY() {
        return 10.0;
    }

    @Override
    protected double rectDeltaX() {
        return -9.0;
    }

    @Override
    protected double rectDeltaY() {
        return -9.0;
    }

    @Override
    protected double imageDeltaX() {
        return 0.0;
    }

```

```

}

@Override
protected double imageDeltaY() {
    return 0.0;
}

public static String getName() {
    return name;
}

public Warrior(Integer health, Boolean isSpawned, boolean team, Integer damage, Boolean isDead, ArrayList<String> inventor) {
    this(health, isSpawned, team, damage, isDead, inventor, 0.0, 0.0);
    initGraphics(getName(), x, y);
}

public Warrior(Integer health, Boolean isSpawned, boolean team, Integer damage, Boolean isDead,
    ArrayList<String> inventor, double startX, double startY) {
    super(health, isSpawned, team, damage, isDead, inventor);
    initGraphics(getName(), startX, startY);
}

public Warrior() {
    this((int) MAX_HEALTH, true, true, 5, false, new ArrayList<>(), 0.0, 0.0);
}

private void initGraphics(String name, double startX, double startY) {
    this.x = startX;
    this.y = startY;
    setMaxHealth(MAX_HEALTH);

    this.labelName = new Label(name);

    life = new Line();
    life.setStrokeWidth(5);
    life.setStroke(Color.LIGHTGREEN);

    image = new ImageView(HelloApplication.imgWarrior);
    image.setFitWidth(100);
    image.setFitHeight(100);

    isActive = false;
    rectActive = new Rectangle(x - 5, y - 5, 110, 110);
    rectActive.setFill(Color.TRANSPARENT);
    rectActive.setStrokeWidth(3);
    rectActive.setStroke(Color.GREEN);

    oreCountLabel = new Label();
    setCoordinates();
}

@Override
public void resurrect() {
    super.resurrect();
    if (this.getClass() == Warrior.class && oreCountLabel != null && HelloApplication.group != null) {
        if (!HelloApplication.group.getChildren().contains(oreCountLabel)) {
            HelloApplication.group.getChildren().add(oreCountLabel);
        }
    }
}

@Override
public void setCoordinates() {
    super.setCoordinates();
    if (this.getClass() == Warrior.class && oreCountLabel != null) {
        oreCountLabel.setText("Ore: " + (int) activeOre);
        oreCountLabel.setLayoutX(x + 65);
        oreCountLabel.setLayoutY(y - 10);
    }
}

public boolean isInverseMode() {
    return inverseMode;
}

public void setInverseMode(boolean inverseMode) {
    this.inverseMode = inverseMode;
}

```

```

private void doOreInverse() {
    if (HelloApplication.buildings == null) {
        return;
    }

    World sourceTarget = findBestSourceTarget();
    World baseTarget = getTeamBase();

    if (sourceTarget == null || baseTarget == null) {
        return;
    }

    long currentTime = System.currentTimeMillis();

    if (!collectingOre && !deliveringOre) {
        collectingOre = true;
    }

    if (collectingOre) {
        if (!this.isActive()) {
            double targetCenterX = baseTarget.imageView.getBoundsInParent().getCenterX();
            double targetCenterY = baseTarget.imageView.getBoundsInParent().getCenterY();
            moveTo(targetCenterX - this.image.getFitWidth() / 2, targetCenterY - this.image.getFitHeight() / 2);
        }
        if (currentTime - lastOreTime >= ORE_COOLDOWN &&
this.image.getBoundsInParent().intersects(baseTarget.imageView.getBoundsInParent())) {
            activeOre += 1;
            oreAmount += 1;
            lastOreTime = currentTime;
        }

        if (activeOre >= 10) {
            activeOre = 10;
            collectingOre = false;
            deliveringOre = true;
        }
        return;
    }

    if (deliveringOre) {
        if (!this.isActive()) {
            double targetCenterX = sourceTarget.imageView.getBoundsInParent().getCenterX();
            double targetCenterY = sourceTarget.imageView.getBoundsInParent().getCenterY();
            moveTo(targetCenterX - this.image.getFitWidth() / 2, targetCenterY - this.image.getFitHeight() / 2);
        }
        if (currentTime - lastOreTime >= ORE_COOLDOWN &&
this.image.getBoundsInParent().intersects(sourceTarget.imageView.getBoundsInParent())) {
            if (activeOre > 0) {
                sourceTarget.setOre(sourceTarget.getOre() + 1);
                activeOre -= 1;
            }
            lastOreTime = currentTime;
        }

        if (activeOre <= 0) {
            activeOre = 0;
            deliveringOre = false;
            collectingOre = true;
        }
    }
}

private void doOre() {
    if (HelloApplication.buildings == null) {
        return;
    }

    World sourceTarget = findBestSourceTarget();
    World baseTarget = getTeamBase();

    if (sourceTarget == null || baseTarget == null) {
        return;
    }

    long currentTime = System.currentTimeMillis();

    if (!collectingOre && !deliveringOre) {
        collectingOre = true;
    }
}

```

```

    if (collectingOre) {
        if (!this.isActive()) {
            double targetCenterX = sourceTarget.imageView.getBoundsInParent().getCenterX();
            double targetCenterY = sourceTarget.imageView.getBoundsInParent().getCenterY();
            moveTo(targetCenterX - this.image.getFitWidth() / 2, targetCenterY - this.image.getFitHeight() / 2);
        }
        if (currentTime - lastOreTime >= ORE_COOLDOWN &&
this.image.getBoundsInParent().intersects(sourceTarget.imageView.getBoundsInParent())) {
            activeOre += 1;
            oreAmount += 1;
            lastOreTime = currentTime;
        }

        if (activeOre >= 10) {
            activeOre = 10;
            collectingOre = false;
            deliveringOre = true;
        }
        return;
    }

    if (deliveringOre) {
        if (!this.isActive()) {
            double targetCenterX = baseTarget.imageView.getBoundsInParent().getCenterX();
            double targetCenterY = baseTarget.imageView.getBoundsInParent().getCenterY();
            moveTo(targetCenterX - this.image.getFitWidth() / 2, targetCenterY - this.image.getFitHeight() / 2);
        }
        if (currentTime - lastOreTime >= ORE_COOLDOWN &&
this.image.getBoundsInParent().intersects(baseTarget.imageView.getBoundsInParent())) {
            if (activeOre > 0) {
                baseTarget.setOre(baseTarget.getOre() + 1);
                activeOre -= 1;
            }
            lastOreTime = currentTime;
        }

        if (activeOre <= 0) {
            activeOre = 0;
            deliveringOre = false;
            collectingOre = true;
        }
    }
}

private Unit findNearbyEnemyUnit() {
    if (HelloApplication.units == null || this.image == null) {
        return null;
    }

    Unit nearest = null;
    double nearestDistance = Double.MAX_VALUE;
    double myCenterX = this.image.getBoundsInParent().getCenterX();
    double myCenterY = this.image.getBoundsInParent().getCenterY();

    for (Unit unit : HelloApplication.units) {
        if (unit == null || unit == this || unit.getDead() || unit.getTeam() == this.team || unit.image == null) {
            continue;
        }

        double unitCenterX = unit.image.getBoundsInParent().getCenterX();
        double unitCenterY = unit.image.getBoundsInParent().getCenterY();
        double dx = unitCenterX - myCenterX;
        double dy = unitCenterY - myCenterY;
        double distance = Math.sqrt(dx * dx + dy * dy);

        if (distance < COMBAT_PRIORITY_RADIUS && distance < nearestDistance) {
            nearestDistance = distance;
            nearest = unit;
        }
    }

    return nearest;
}

private World findNearbyEnemyBuilding() {
    if (HelloApplication.buildings == null || this.image == null) {
        return null;
    }
}

```

```

World nearest = null;
double nearestDistance = Double.MAX_VALUE;
double myCenterX = this.image.getBoundsInParent().getCenterX();
double myCenterY = this.image.getBoundsInParent().getCenterY();

for (World world : HelloApplication.buildings) {
    if (world == null || world.getTeam() == this.team || world.imageView == null) {
        continue;
    }

    double worldCenterX = world.imageView.getBoundsInParent().getCenterX();
    double worldCenterY = world.imageView.getBoundsInParent().getCenterY();
    double dx = worldCenterX - myCenterX;
    double dy = worldCenterY - myCenterY;
    double distance = Math.sqrt(dx * dx + dy * dy);

    if (distance < COMBAT_PRIORITY_RADIUS && distance < nearestDistance) {
        nearestDistance = distance;
        nearest = world;
    }
}

return nearest;
}

private World findBestSourceTarget() {
    if (HelloApplication.buildings == null || this.image == null) {
        return null;
    }

    World closest = null;
    double closestDistance = Double.MAX_VALUE;
    double myCenterX = this.image.getBoundsInParent().getCenterX();
    double myCenterY = this.image.getBoundsInParent().getCenterY();

    for (World build : HelloApplication.buildings) {
        if (build == null || build.getTeam() != this.team || !(build instanceof Source) || build.imageView == null) {
            continue;
        }

        if (this.image.getBoundsInParent().intersects(build.imageView.getBoundsInParent())) {
            return build;
        }

        double targetCenterX = build.imageView.getBoundsInParent().getCenterX();
        double targetCenterY = build.imageView.getBoundsInParent().getCenterY();
        double distance = Math.hypot(targetCenterX - myCenterX, targetCenterY - myCenterY);

        if (distance < closestDistance) {
            closestDistance = distance;
            closest = build;
        }
    }

    return closest;
}

private World getTeamBase() {
    if (HelloApplication.basesA == null || HelloApplication.basesB == null) {
        return null;
    }

    ArrayList<Base> bases = this.team ? HelloApplication.basesA : HelloApplication.basesB;
    if (bases == null || bases.isEmpty()) {
        return null;
    }

    Base base = bases.get(0);
    if (base == null || base.imageView == null) {
        return null;
    }

    return base;
}

@Override
protected void promotion() {
    if (HelloApplication.units == null) {

```

```

        return;
    }
    int ore = (int) this.getOre();
    if (ore >= 100) {
        this.setDead(true);
        HelloApplication.group.getChildren().removeAll(this.image, this.labelName, this.life, this.rectActive, oreCountLabel);
        this.removeUnitFromGame();
        Centurio centurio = new Centurio((int) MAX_HEALTH, true, this.team, 10, false, new ArrayList<>(this.getInventor()), this.x, this.y);
        HelloApplication.units.add(centurio);
        centurio.resurrect();
    }
}

public void attackInverse() {
    boolean intersects = false;

    if (this.getDamage() == null || this.getDamage() <= 0) {
        return;
    }
    if (this.image == null) {
        return;
    }

    if (HelloApplication.units != null) {
        for (Unit unit : HelloApplication.units) {
            if (unit != this && unit.getTeam() != this.team && !unit.getDead() && unit.image != null) {
                intersects = this.image.getBoundsInParent().intersects(unit.image.getBoundsInParent());
                if (intersects) {
                    int targetHealth = unit.getHealth() == null ? 0 : unit.getHealth();
                    int newHealth = targetHealth + this.getDamage();
                    unit.setHealth(newHealth);

                    if (newHealth <= 0) {
                        unit.setHealth(0);
                        plusObjectedKilled();
                        unit.removeUnitFromGame();
                    }
                }
                break;
            }
        }
    }

    if (HelloApplication.buildings != null) {
        for (World world : HelloApplication.buildings) {
            if (world != null && world.getTeam() != this.team && world.imageView != null) {
                intersects = this.image.getBoundsInParent().intersects(world.imageView.getBoundsInParent());
                if (intersects) {
                    int targetHealth = world.getHealth() == 0 ? 0 : (int) world.getHealth();
                    int newHealth = targetHealth + this.getDamage();
                    world.setHealth(newHealth);
                }
            }
        }
    }
}

@Override
public void logic() {
    Unit nearbyEnemyUnit = findNearbyEnemyUnit();
    World nearbyEnemyBuilding = findNearbyEnemyBuilding();

    if (!this.isActive() && (nearbyEnemyUnit != null || nearbyEnemyBuilding != null)) {
        if (nearbyEnemyBuilding != null) {
            double bCenterX = nearbyEnemyBuilding.imageView.getBoundsInParent().getCenterX();
            double bCenterY = nearbyEnemyBuilding.imageView.getBoundsInParent().getCenterY();
            double myCenterX = this.image.getBoundsInParent().getCenterX();
            double myCenterY = this.image.getBoundsInParent().getCenterY();
            double dist = Math.hypot(bCenterX - myCenterX, bCenterY - myCenterY);
            if (dist > 130.0) {
                this.moveTo(bCenterX - this.image.getFitWidth() / 2, bCenterY - this.image.getFitHeight() / 2);
            }
        } else {
            double uCenterX = nearbyEnemyUnit.image.getBoundsInParent().getCenterX();
            double uCenterY = nearbyEnemyUnit.image.getBoundsInParent().getCenterY();
            double myCenterX = this.image.getBoundsInParent().getCenterX();
            double myCenterY = this.image.getBoundsInParent().getCenterY();
            double dist = Math.hypot(uCenterX - myCenterX, uCenterY - myCenterY);
            if (dist > 70.0) {
                this.moveTo(uCenterX - this.image.getFitWidth() / 2, uCenterY - this.image.getFitHeight() / 2);
            }
        }
    }
}

```

```

    }
}

long currentTime = System.currentTimeMillis();
if (currentTime - lastAttackTime >= ATTACK_COOLDOWN) {
    attack();
    lastAttackTime = currentTime;
}
return;
}

doOre();
promotion();
}

@Override
public void logicInverse() {
    Unit nearbyEnemyUnit = findNearbyEnemyUnit();
    World nearbyEnemyBuilding = findNearbyEnemyBuilding();

    if (!this.isActive() && (nearbyEnemyUnit != null || nearbyEnemyBuilding != null)) {
        if (nearbyEnemyBuilding != null) {
            double bCenterX = nearbyEnemyBuilding.imageView.getBoundsInParent().getCenterX();
            double bCenterY = nearbyEnemyBuilding.imageView.getBoundsInParent().getCenterY();
            double myCenterX = this.imageView.getBoundsInParent().getCenterX();
            double myCenterY = this.imageView.getBoundsInParent().getCenterY();
            double dist = Math.hypot(bCenterX - myCenterX, bCenterY - myCenterY);
            if (dist > 130.0) {
                this.moveTo(bCenterX - this.imageView.getFitWidth() / 2, bCenterY - this.imageView.getFitHeight() / 2);
            }
        } else {
            double uCenterX = nearbyEnemyUnit.imageView.getBoundsInParent().getCenterX();
            double uCenterY = nearbyEnemyUnit.imageView.getBoundsInParent().getCenterY();
            double myCenterX = this.imageView.getBoundsInParent().getCenterX();
            double myCenterY = this.imageView.getBoundsInParent().getCenterY();
            double dist = Math.hypot(uCenterX - myCenterX, uCenterY - myCenterY);
            if (dist > 70.0) {
                this.moveTo(uCenterX - this.imageView.getFitWidth() / 2, uCenterY - this.imageView.getFitHeight() / 2);
            }
        }
    }

    long currentTime = System.currentTimeMillis();
    if (currentTime - lastAttackTime >= ATTACK_COOLDOWN) {
        attackInverse();
        lastAttackTime = currentTime;
    }
    return;
}

doOreInverse();
promotion();
}

@Override
public Unit clone() throws CloneNotSupportedException {
    Warrior cloned = (Warrior) super.clone();
    cloned.oreAmount = this.oreAmount;
    cloned.activeOre = this.activeOre;
    cloned.collectingOre = this.collectingOre;
    cloned.deliveringOre = this.deliveringOre;
    cloned.lastOreTime = this.lastOreTime;
    cloned.inverseMode = this.inverseMode;
    if (this.oreCountLabel != null) cloned.oreCountLabel = new Label(this.oreCountLabel.getText());
    return cloned;
}
}

```

Лістинг модуля Centurio.java

```

package org.example.laba5.Unit;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import javafx.scene.control.Label;
import javafx.scene.image.ImageView;
import javafx.scene.paint.Color;
import javafx.scene.shape.Line;
import javafx.scene.shape.Rectangle;
import org.example.laba5.Base;
import org.example.laba5.HelloApplication;
import org.example.laba5.World;

public class Centurio extends Warrior {
    private static String name = "Centurio";
    private static final double MAX_HEALTH = 120.0;
    private double healNum = 10.0;
    private static final long HEAL_COOLDOWN = 2000;
    private long lastHealTime = 0;

    private Label killCountLabel;
    private int killCount = 0;
    private HashSet<Unit> countedKills = new HashSet<>();

    public int getKillCount() {
        return killCount;
    }

    @Override
    public Label getKillCountLabel() {
        return killCountLabel;
    }

    public void setKillCount(int killCount) {
        this.killCount = killCount;
    }

    @Override
    protected double labelDeltaX() {
        return 10.0;
    }

    @Override
    protected double labelDeltaY() {
        return -10.0;
    }

    @Override
    protected double lifeDeltaX() {
        return 0.0;
    }

    @Override
    protected double lifeDeltaY() {
        return 10.0;
    }

    @Override
    protected double rectDeltaX() {
        return -9.0;
    }

    @Override
    protected double rectDeltaY() {
        return -9.0;
    }

    @Override
    protected double imageDeltaX() {
        return 0.0;
    }

    @Override
    protected double imageDeltaY() {

```

```

    return 12.0;
}

@Override
public void setCoordinates() {
    super.setCoordinates();
    if (mainWeaponImage != null) {
        mainWeaponImage.setX(x);
        mainWeaponImage.setY(y + 17);
    }
    if (this.getClass() == Centurio.class && killCountLabel != null) {
        killCountLabel.setText("Kills : " + killCount);
        killCountLabel.setLayoutX(x + 60);
        killCountLabel.setLayoutY(y - 10);
    }
}

public static String getName() {
    return name;
}

public Centurio(Integer health, Boolean isSpawned, boolean team, Integer damage, Boolean isDead, ArrayList<String> inventor) {
    this(health, isSpawned, team, damage, isDead, inventor, 0.0, 0.0);
    initGraphics(getName(), x, y);
}

public Centurio(Integer health, Boolean isSpawned, boolean team, Integer damage, Boolean isDead,
    ArrayList<String> inventor, double startX, double startY) {
    super(health, isSpawned, team, damage, isDead, inventor);
    initGraphics(getName(), startX, startY);
}

public Centurio() {
    this((int) MAX_HEALTH, true, true, 5, false, new ArrayList<>(Arrays.asList("Spear")), 0.0, 0.0);
}

private void initGraphics(String name, double startX, double startY) {
    this.x = startX;
    this.y = startY;
    setMaxHealth(MAX_HEALTH);

    this.labelName = new Label(name);

    life = new Line();
    life.setStrokeWidth(5);
    life.setStroke(Color.LIGHTGREEN);

    image = new ImageView(HelloApplication.imgCenturio);
    image.setFitWidth(100);
    image.setFitHeight(100);

    isActive = false;
    rectActive = new Rectangle(x - 5, y - 5, 110, 110);
    rectActive.setFill(Color.TRANSPARENT);
    rectActive.setStrokeWidth(3);
    rectActive.setStroke(Color.GREEN);

    killCountLabel = new Label("Kills : " + killCount);

    setCoordinates();
}

private void heal(World target) {
    if (target == null || target.imageView == null || this.image == null) {
        return;
    }
    double myCenterX = this.image.getBoundsInParent().getCenterX();
    double myCenterY = this.image.getBoundsInParent().getCenterY();
    double targetCenterX = target.imageView.getBoundsInParent().getCenterX();
    double targetCenterY = target.imageView.getBoundsInParent().getCenterY();

    boolean isIntersecting = Math.sqrt(Math.pow(targetCenterX - myCenterX, 2) + Math.pow(targetCenterY - myCenterY, 2)) < 100;

    if (isIntersecting) {
        double newHealth = Math.min(target.getHealth() + this.healNum, target.getMaxHealth());
        target.setHealth(newHealth);
    }
}

```

```

@Override
public void attack() {
    if (HelloApplication.units != null) {
        ArrayList<Unit> aliveEnemies = new ArrayList<>();
        for (Unit unit : HelloApplication.units) {
            if (unit != this && unit.getTeam() != this.team && !unit.getDead() && unit.image != null) {
                boolean intersects = this.image.getBoundsInParent().intersects(unit.image.getBoundsInParent());
                if (intersects) {
                    aliveEnemies.add(unit);
                    break;
                }
            }
        }

        super.attack();

        for (Unit enemy : aliveEnemies) {
            if (enemy.getDead() && !countedKills.contains(enemy)) {
                killCount++;
                countedKills.add(enemy);
            }
        }
    } else {
        super.attack();
    }
}

@Override
public void resurrect() {
    super.resurrect();
    if (this.getClass() == Centurio.class && killCountLabel != null && HelloApplication.group != null) {
        if (!HelloApplication.group.getChildren().contains(killCountLabel)) {
            HelloApplication.group.getChildren().add(killCountLabel);
        }
    }
}

@Override
public void removeUnitFromGame() {
    if (HelloApplication.group != null && killCountLabel != null) {
        HelloApplication.group.getChildren().remove(killCountLabel);
    }
    countedKills.clear();
    super.removeUnitFromGame();
}

@Override
public void promotion() {
    if (this.getKillCount() >= 5) {
        int pretorioHealth = 150;
        int newDamage = this.getDamage() + 2;

        Pretorio pretorio = new Pretorio(
            pretorioHealth,
            true,
            this.team,
            newDamage,
            false,
            new ArrayList<>(this.getInventor()),
            this.x,
            this.y
        );

        pretorio.setKillCount(this.getKillCount());
        HelloApplication.units.add(pretorio);
        pretorio.resurrect();
        this.removeUnitFromGame();
    }
}

@Override
public void logic() {
    World mainTarget = null;
    Unit subTarget = null;
    World healTarget = null;
    boolean goToMain = false;
    double distanceToMainTarget = Double.MAX_VALUE;
    double distanceToSubTarget = Double.MAX_VALUE;
    double distanceToHealTarget = Double.MAX_VALUE;

```

```

if (HelloApplication.units != null && !this.isActive()) {
    double myCenterX = this.image.getBoundsInParent().getCenterX();
    double myCenterY = this.image.getBoundsInParent().getCenterY();
    for (Unit unit : HelloApplication.units) {
        if (unit != this && unit.getTeam() != this.team && !unit.getDead() && unit.image != null) {
            double unitCenterX = unit.image.getBoundsInParent().getCenterX();
            double unitCenterY = unit.image.getBoundsInParent().getCenterY();
            double dx = unitCenterX - myCenterX;
            double dy = unitCenterY - myCenterY;
            double distancesub = Math.sqrt(dx * dx + dy * dy);
            if (distancesub < distanceToSubTarget) {
                distanceToSubTarget = distancesub;
                subTarget = unit;
            }
        }
    }
}

if (HelloApplication.buildings != null) {
    double myCenterX = this.image.getBoundsInParent().getCenterX();
    double myCenterY = this.image.getBoundsInParent().getCenterY();
    for (World world : HelloApplication.buildings) {
        if (world != null && world.getTeam() != this.team && world.imageView != null) {
            double worldCenterX = world.imageView.getBoundsInParent().getCenterX();
            double worldCenterY = world.imageView.getBoundsInParent().getCenterY();
            double dx = worldCenterX - myCenterX;
            double dy = worldCenterY - myCenterY;
            double distancemain = Math.sqrt(dx * dx + dy * dy);
            if (distancemain < distanceToMainTarget) {
                distanceToMainTarget = distancemain;
                mainTarget = world;
            }
        }
    }
}

if (HelloApplication.buildings != null) {
    for (World world : HelloApplication.buildings) {
        if (world != null && world.getTeam() == this.team && world.imageView != null && world.getHealth() < world.getMaxHealth()) {
            double myCenterX = this.image.getBoundsInParent().getCenterX();
            double myCenterY = this.image.getBoundsInParent().getCenterY();
            double worldCenterX = world.imageView.getBoundsInParent().getCenterX();
            double worldCenterY = world.imageView.getBoundsInParent().getCenterY();

            double dx = worldCenterX - myCenterX;
            double dy = worldCenterY - myCenterY;
            double distanceheal = Math.sqrt(dx * dx + dy * dy);
            if (distanceheal < distanceToHealTarget && world.team == this.team && world.getHealth() < world.getMaxHealth() / 2) {
                distanceToHealTarget = distanceheal;
                healTarget = world;
            }
        }
    }
}

if (!this.isActive()) {
    if (healTarget != null) {
        if (distanceToHealTarget > 80.0) {
            double targetCenterX = healTarget.imageView.getBoundsInParent().getCenterX();
            double targetCenterY = healTarget.imageView.getBoundsInParent().getCenterY();
            this.moveTo(targetCenterX - this.image.getFitWidth() / 2, targetCenterY - this.image.getFitHeight() / 2);
        }
        long currentTime = System.currentTimeMillis();
        if (currentTime - lastHealTime >= HEAL_COOLDOWN) {
            heal(healTarget);
            lastHealTime = currentTime;
        }
        if (healTarget.getHealth() >= healTarget.getMaxHealth()) {
            healTarget = null;
        }
    }
    return;
}

if (mainTarget != null && subTarget != null) {
    goToMain = distanceToMainTarget < distanceToSubTarget;
} else if (mainTarget != null) {
    goToMain = true;
} else if (subTarget != null) {
    goToMain = false;
}

```

```

    } else {
        return;
    }

    if (goToMain) {
        if (distanceToMainTarget > 130.0) {
            double targetCenterX = mainTarget.imageView.getBoundsInParent().getCenterX();
            double targetCenterY = mainTarget.imageView.getBoundsInParent().getCenterY();
            this.moveTo(targetCenterX - this.image.getFitWidth() / 2, targetCenterY - this.image.getFitHeight() / 2);
        }
    } else {
        if (distanceToSubTarget > 70.0) {
            double targetCenterX = subTarget.image.getBoundsInParent().getCenterX();
            double targetCenterY = subTarget.image.getBoundsInParent().getCenterY();
            this.moveTo(targetCenterX - this.image.getFitWidth() / 2, targetCenterY - this.image.getFitHeight() / 2);
        }
    }

    long currentTime = System.currentTimeMillis();
    if (currentTime - lastAttackTime >= ATTACK_COOLDOWN) {
        attack();
        lastAttackTime = currentTime;
    }
}
promotion();
}

@Override
public Unit clone() throws CloneNotSupportedException {
    Centurio cloned = (Centurio) super.clone();
    cloned.killCount = this.killCount;
    return cloned;
}
}

```

Лістинг модуля Pretorio.java

```

package org.example.laba5.Unit;

import java.util.ArrayList;
import java.util.Arrays;
import javafx.scene.control.Label;
import javafx.scene.image.ImageView;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.shape.Line;
import javafx.scene.shape.Rectangle;
import org.example.laba5.HelloApplication;
import org.example.laba5.World;

public class Pretorio extends Centurio {
    private static String name = "Pretorio";
    private static final double MAX_HEALTH = 150.0;
    private double healNum = 5.0;
    private Circle areaOfEffect;
    private static final long HEALAREA_COOLDOWN = 1000;
    private long lastHealAreaTime = 0;

    @Override
    protected double labelDeltaX() {
        return 10.0;
    }

    @Override
    protected double labelDeltaY() {
        return -10.0;
    }

    @Override
    protected double lifeDeltaX() {
        return 0.0;
    }

    @Override
    protected double lifeDeltaY() {
        return 10.0;
    }

    @Override
    protected double rectDeltaX() {
        return -9.0;
    }

    @Override
    protected double rectDeltaY() {
        return -9.0;
    }

    @Override
    protected double imageDeltaX() {
        return 0.0;
    }

    @Override
    protected double imageDeltaY() {
        return 12.0;
    }

    public static String getName() {
        return name;
    }

    public Pretorio(Integer health, Boolean isSpawned, boolean team, Integer damage, Boolean isDead, ArrayList<String> inventor) {
        this(health, isSpawned, team, damage, isDead, inventor, 0.0, 0.0);
        initGraphics(getName(), x, y);
    }

    public Pretorio(Integer health, Boolean isSpawned, boolean team, Integer damage, Boolean isDead,
        ArrayList<String> inventor, double startX, double startY) {
        super(health, isSpawned, team, damage, isDead, inventor);
        initGraphics(getName(), startX, startY);
    }
}

```

```

public Pretorio() {
    this((int) MAX_HEALTH, true, true, 5, false, new ArrayList<>(Arrays.asList("Sword")), 0.0, 0.0);
}

private void initGraphics(String name, double startX, double startY) {
    this.x = startX;
    this.y = startY;
    setMaxHealth(MAX_HEALTH);

    this.labelName = new Label(name);

    life = new Line();
    life.setStrokeWidth(5);
    life.setStroke(Color.LIGHTGREEN);

    image = new ImageView(HelloApplication.imgPretorio);
    image.setFitWidth(100);
    image.setFitHeight(100);

    isActive = false;
    rectActive = new Rectangle(x - 5, y - 5, 110, 110);
    rectActive.setFill(Color.TRANSPARENT);
    rectActive.setStrokeWidth(3);
    rectActive.setStroke(Color.GREEN);
    areaOfEffect = new Circle(x, y, 150);
    areaOfEffect.setFill(Color.TRANSPARENT);
    areaOfEffect.setStroke(Color.GREEN);
    areaOfEffect.setStrokeWidth(2);
    areaOfEffect.setMouseTransparent(true);
    areaOfEffect.setVisible(false);

    setCoordinates();
}

@Override
public void resurrect() {
    super.resurrect();
    if (HelloApplication.group != null && areaOfEffect != null) {
        if (!HelloApplication.group.getChildren().contains(areaOfEffect)) {
            HelloApplication.group.getChildren().add(areaOfEffect);
        }
        areaOfEffect.toBack();
        areaOfEffect.setVisible(this.isActive);
    }
    if (mainWeaponImage != null) {
        mainWeaponImage.setY(y + 17);
    }
}

@Override
public void setCoordinates() {
    super.setCoordinates();
    if (areaOfEffect != null && image != null) {
        double centerX = image.getX() + image.getFitWidth() / 2.0;
        double centerY = image.getY() + image.getFitHeight() / 2.0;
        areaOfEffect.setCenterX(centerX);
        areaOfEffect.setCenterY(centerY);
    }
    if (mainWeaponImage != null) {
        mainWeaponImage.setX(x);
        mainWeaponImage.setY(y + 17);
    }
}

@Override
public boolean flipActivation() {
    boolean activeNow = super.flipActivation();
    if (HelloApplication.group != null && areaOfEffect != null) {
        if (!HelloApplication.group.getChildren().contains(areaOfEffect)) {
            HelloApplication.group.getChildren().add(areaOfEffect);
        }
        areaOfEffect.setVisible(activeNow);
        areaOfEffect.toBack();
    }
    return activeNow;
}

private void healInArea() {

```

```

if (HelloApplication.units != null) {
    for (Unit unit : HelloApplication.units) {
        if (unit != null
            && unit != this
            && unit.getTeam() == this.team
            && unit.image != null
            && unit.getHealth() != null
            && unit.getHealth() > 0) {
            double myCenterX = this.image.getBoundsInParent().getCenterX();
            double myCenterY = this.image.getBoundsInParent().getCenterY();
            double unitCenterX = unit.image.getBoundsInParent().getCenterX();
            double unitCenterY = unit.image.getBoundsInParent().getCenterY();

            double dx = unitCenterX - myCenterX;
            double dy = unitCenterY - myCenterY;
            double distanceheal = Math.sqrt(dx * dx + dy * dy);

            if (distanceheal < 150) {
                int currentHealth = unit.getHealth();
                int allyMaxHealth = (int) (unit.maxHealth > 0 ? unit.maxHealth : 100);
                int newHealth = Math.min((int) (currentHealth + this.healNum), allyMaxHealth);
                unit.setHealth(newHealth);
            }
        }
    }
}

@Override
public void logic() {
    World mainTarget = null;
    Unit subTarget = null;
    boolean goToMain = false;
    double distanceToMainTarget = Double.MAX_VALUE;
    double distanceToSubTarget = Double.MAX_VALUE;

    if (!this.isActive()) {
        if (HelloApplication.units != null) {
            for (Unit unit : HelloApplication.units) {
                if (unit != this && unit.getTeam() != this.team && !unit.getDead() && unit.image != null) {
                    double myCenterX = this.image.getBoundsInParent().getCenterX();
                    double myCenterY = this.image.getBoundsInParent().getCenterY();
                    double unitCenterX = unit.image.getBoundsInParent().getCenterX();
                    double unitCenterY = unit.image.getBoundsInParent().getCenterY();
                    double dx = unitCenterX - myCenterX;
                    double dy = unitCenterY - myCenterY;
                    double distancesub = Math.sqrt(dx * dx + dy * dy);

                    if (distancesub < distanceToSubTarget) {
                        distanceToSubTarget = distancesub;
                        subTarget = unit;
                    }
                }
            }
        }

        if (HelloApplication.buildings != null) {
            for (World world : HelloApplication.buildings) {
                if (world != null && world.getTeam() != this.team && world.imageView != null) {
                    double myCenterX = this.image.getBoundsInParent().getCenterX();
                    double myCenterY = this.image.getBoundsInParent().getCenterY();
                    double worldCenterX = world.imageView.getBoundsInParent().getCenterX();
                    double worldCenterY = world.imageView.getBoundsInParent().getCenterY();
                    double dx = worldCenterX - myCenterX;
                    double dy = worldCenterY - myCenterY;
                    double distancemain = Math.sqrt(dx * dx + dy * dy);

                    if (distancemain < distanceToMainTarget) {
                        distanceToMainTarget = distancemain;
                        mainTarget = world;
                    }
                }
            }
        }

        if (mainTarget != null && subTarget != null) {
            if (distanceToMainTarget < distanceToSubTarget) {
                goToMain = true;
            }
        }
    }
}

```

```

    } else if (mainTarget != null) {
        goToMain = true;
    } else if (subTarget != null) {
        goToMain = false;
    }

    if (goToMain && mainTarget != null) {
        if (distanceToMainTarget > 130.0) {
            double targetCenterX = mainTarget.imageView.getBoundsInParent().getCenterX();
            double targetCenterY = mainTarget.imageView.getBoundsInParent().getCenterY();
            this.moveTo(targetCenterX - this.image.getFitWidth() / 2, targetCenterY - this.image.getFitHeight() / 2);
        }
    } else if (!goToMain && subTarget != null) {
        if (distanceToSubTarget > 70.0) {
            double targetCenterX = subTarget.image.getBoundsInParent().getCenterX();
            double targetCenterY = subTarget.image.getBoundsInParent().getCenterY();
            this.moveTo(targetCenterX - this.image.getFitWidth() / 2, targetCenterY - this.image.getFitHeight() / 2);
        }
    }

    long currentTime = System.currentTimeMillis();
    if ((mainTarget != null || subTarget != null) && currentTime - lastAttackTime >= ATTACK_COOLDOWN) {
        attack();
        lastAttackTime = currentTime;
    }

    long currentTime = System.currentTimeMillis();
    if (currentTime - lastHealAreaTime >= HEALAREA_COOLDOWN) {
        healInArea();
        lastHealAreaTime = currentTime;
    }
}
}

```

Лістинг модуля Base.java

```

package org.example.laba5;

import java.net.URL;
import java.util.ArrayList;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import org.example.laba5.Unit.Unit;

public class Base extends World {
    private static final double MAX_HEALTH = 200.0;
    private final ArrayList<Unit> unitsInside = new ArrayList<>();
    private final URL conturUrlRed = HelloApplication.class.getResource("/red.png");
    private final URL conturUrlGreen = HelloApplication.class.getResource("/green.png");

    public Base(ArrayList<Unit> units, boolean team) {
        super(units);
        this.team = team;
        loadContourImages();
    }

    public Base() {
        super();
        this.team = false;
        loadContourImages();
    }

    private void loadContourImages() {
        if (conturUrlRed != null) {
            conturImageRedImage = new Image(conturUrlRed.toExternalForm(), 30, 30, false, false);
        }
        if (conturUrlGreen != null) {
            conturImageGreenImage = new Image(conturUrlGreen.toExternalForm(), 30, 30, false, false);
        }
    }

    public ArrayList<Unit> getUnitsInside() {
        return unitsInside;
    }

    public boolean getTeam() {
        return this.team;
    }

    private boolean containsByReference(Unit candidate) {
        for (Unit u : unitsInside) {
            if (u == candidate) {
                return true;
            }
        }
        return false;
    }

    private void removeByReference(Unit candidate) {
        unitsInside.removeIf(u -> u == candidate);
    }

    public void setTeam(boolean team) {
        this.team = team;
    }

    @Override
    public void initGraphics(javafx.scene.image.Image image, String name, int numUnits, double x, double y, double maxHealth, double health) {
        super.initGraphics(image, name, numUnits, x, y, maxHealth, health);
        super.setMaxHealth(MAX_HEALTH);
    }

    @Override
    public void resurrectWorld() {
        super.resurrectWorld();
        loadContourImages();
        if (HelloApplication.group == null) {
            return;
        }
        if (this.team && conturImageGreenImage != null) {

```

```

        this.contourView = new ImageView(conturImageGreenImage);
        this.contourView.setX(x);
        this.contourView.setY(y);
        HelloApplication.group.getChildren().add(this.contourView);
    } else if (!this.team && conturImageRedImage != null) {
        this.contourView = new ImageView(conturImageRedImage);
        this.contourView.setX(x);
        this.contourView.setY(y);
        HelloApplication.group.getChildren().add(this.contourView);
    }
}

@Override
protected void intersect() {
    ArrayList<Unit> worldUnits = getUnits();
    if ((worldUnits == null || worldUnits.isEmpty()) && HelloApplication.units != null) {
        worldUnits = HelloApplication.units;
    }
    if (worldUnits == null) {
        return;
    }
    if (this.imageView == null) {
        return;
    }

    unitsInside.removeIf(unit -> unit == null || Boolean.TRUE.equals(unit.getDead()));

    for (Unit unit : new ArrayList<>(worldUnits)) {
        if (unit == null || Boolean.TRUE.equals(unit.getDead()) || unit.image == null) {
            continue;
        }
        boolean intersects = unit.image.getBoundsInParent().intersects(this.imageView.getBoundsInParent());
        if (intersects) {
            if (!containsByReference(unit)) {
                unitsInside.add(unit);
            }
        } else {
            removeByReference(unit);
        }
    }

    numUnits = unitsInside.size();
    if (numUnitsLabel != null) {
        numUnitsLabel.setText(String.valueOf(numUnits));
    }
}

@Override
public boolean isUnitInside(Unit unit) {
    return unitsInside.contains(unit);
}
}

```

ЛІСТИНГ модуля Source.java

```

package org.example.laba5;

import java.net.URL;
import java.util.ArrayList;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import org.example.laba5.Unit.Unit;

public class Source extends World {
    private static final double MAX_HEALTH = 200.0;
    private final ArrayList<Unit> unitsInside = new ArrayList<>();
    private final URL conturUrlRed = HelloApplication.class.getResource("/red.png");
    private final URL conturUrlGreen = HelloApplication.class.getResource("/green.png");

    public Source(ArrayList<Unit> units, boolean team) {
        super(units);
        this.team = team;
        loadContourImages();
    }

    public Source() {
        super();
        this.team = false;
        loadContourImages();
    }

    private void loadContourImages() {
        if (conturUrlRed != null) {
            conturImageRedImage = new Image(conturUrlRed.toExternalForm(), 20, 20, false, false);
        }
        if (conturUrlGreen != null) {
            conturImageGreenImage = new Image(conturUrlGreen.toExternalForm(), 20, 20, false, false);
        }
    }

    public ArrayList<Unit> getUnitsInside() {
        return unitsInside;
    }

    public boolean getTeam() {
        return this.team;
    }

    private boolean containsByReference(Unit candidate) {
        for (Unit u : unitsInside) {
            if (u == candidate) {
                return true;
            }
        }
        return false;
    }

    private void removeByReference(Unit candidate) {
        unitsInside.removeIf(u -> u == candidate);
    }

    public void setTeam(boolean team) {
        this.team = team;
    }

    @Override
    public void initGraphics(javafx.scene.image.Image image, String name, int numUnits, double x, double y, double maxHealth, double health) {
        super.initGraphics(image, name, numUnits, x, y, maxHealth, health);
        super.setMaxHealth(MAX_HEALTH);
    }

    @Override
    public void resurrectWorld() {
        super.resurrectWorld();
        if (this.imageView != null) {
            this.imageView.setY(y + 7);
        }
        loadContourImages();
        if (HelloApplication.group == null) {
            return;
        }
    }
}

```

```

    }
    if (this.team && conturImageGreenImage != null) {
        this.contourView = new ImageView(conturImageGreenImage);
        this.contourView.setX(x);
        this.contourView.setY(y);
        HelloApplication.group.getChildren().add(this.contourView);
    } else if (!this.team && conturImageRedImage != null) {
        this.contourView = new ImageView(conturImageRedImage);
        this.contourView.setX(x);
        this.contourView.setY(y);
        HelloApplication.group.getChildren().add(this.contourView);
    }
}

@Override
protected void intersect() {
    ArrayList<Unit> worldUnits = getUnits();
    if ((worldUnits == null || worldUnits.isEmpty()) && HelloApplication.units != null) {
        worldUnits = HelloApplication.units;
    }
    if (worldUnits == null) {
        return;
    }
    if (this.imageView == null) {
        return;
    }

    unitsInside.removeIf(unit -> unit == null || Boolean.TRUE.equals(unit.getDead()));

    for (Unit unit : new ArrayList<>(worldUnits)) {
        if (unit == null || Boolean.TRUE.equals(unit.getDead()) || unit.image == null) {
            continue;
        }
        boolean intersects = unit.image.getBoundsInParent().intersects(this.imageView.getBoundsInParent());
        if (intersects) {
            if (!containsByReference(unit)) {
                unitsInside.add(unit);
            }
        } else {
            removeByReference(unit);
        }
    }

    numUnits = unitsInside.size();
    if (numUnitsLabel != null) {
        numUnitsLabel.setText(String.valueOf(numUnits));
    }
}

@Override
public boolean isUnitInside(Unit unit) {
    return unitsInside.contains(unit);
}
}

```

Лістинг модуля Tower.java

```

package org.example.laba5;

import java.net.URL;
import java.util.ArrayList;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import org.example.laba5.Unit.Unit;

public class Tower extends World {
    private static final double MAX_HEALTH = 300.0;
    private final ArrayList<Unit> unitsInside = new ArrayList<>();
    private int healAmount = 5;
    private final URL conturUrlRed = HelloApplication.class.getResource("/red.png");
    private final URL conturUrlGreen = HelloApplication.class.getResource("/green.png");

    public Tower(ArrayList<Unit> units, boolean team) {
        super(units);
        this.team = team;
        loadContourImages();
    }

    public Tower() {
        super();
        this.team = false;
        loadContourImages();
    }

    private void loadContourImages() {
        if (conturUrlRed != null) {
            conturImageRedImage = new Image(conturUrlRed.toExternalForm(), 20, 20, false, false);
        }
        if (conturUrlGreen != null) {
            conturImageGreenImage = new Image(conturUrlGreen.toExternalForm(), 20, 20, false, false);
        }
    }

    public ArrayList<Unit> getUnitsInside() {
        return unitsInside;
    }

    public boolean getTeam() {
        return this.team;
    }

    private boolean containsByReference(Unit candidate) {
        for (Unit u : unitsInside) {
            if (u == candidate) {
                return true;
            }
        }
        return false;
    }

    private void removeByReference(Unit candidate) {
        unitsInside.removeIf(u -> u == candidate);
    }

    public void setHealAmount(int healAmount) {
        if (healAmount > 0) {
            this.healAmount = healAmount;
        }
    }

    public void setTeam(boolean team) {
        this.team = team;
    }

    @Override
    public void initGraphics(javafx.scene.image.Image image, String name, int numUnits, double x, double y, double maxHealth, double health) {
        super.initGraphics(image, name, numUnits, x, y, maxHealth, health);
        super.setMaxHealth(MAX_HEALTH);
    }

    @Override
    public void resurrectWorld() {

```

```

super.resurrectWorld();
loadContourImages();
if (HelloApplication.group == null) {
    return;
}
if (this.team && conturImageGreenImage != null) {
    this.contourView = new ImageView(conturImageGreenImage);
    this.contourView.setX(x);
    this.contourView.setY(y);
    HelloApplication.group.getChildren().add(this.contourView);
} else if (!this.team && conturImageRedImage != null) {
    this.contourView = new ImageView(conturImageRedImage);
    this.contourView.setX(x);
    this.contourView.setY(y);
    HelloApplication.group.getChildren().add(this.contourView);
}
}

@Override
protected void intersect() {
    ArrayList<Unit> worldUnits = getUnits();
    if ((worldUnits == null || worldUnits.isEmpty()) && HelloApplication.units != null) {
        worldUnits = HelloApplication.units;
    }
    if (worldUnits == null) {
        return;
    }
    if (this.imageView == null) {
        return;
    }

    unitsInside.removeIf(unit -> unit == null || Boolean.TRUE.equals(unit.getDead()));

    for (Unit unit : new ArrayList<>(worldUnits)) {
        if (unit == null || Boolean.TRUE.equals(unit.getDead()) || unit.image == null) {
            continue;
        }
        boolean intersects = unit.image.getBoundsInParent().intersects(this.imageView.getBoundsInParent());
        if (intersects) {
            if (!containsByReference(unit)) {
                unitsInside.add(unit);
            }
        } else {
            removeByReference(unit);
        }
    }

    numUnits = unitsInside.size();
    if (numUnitsLabel != null) {
        numUnitsLabel.setText(String.valueOf(numUnits));
    }
}

public void healUnits() {
    if (this.getHealth() <= 0) {
        return;
    }
    for (Unit unit : unitsInside) {
        if (unit == null || Boolean.TRUE.equals(unit.getDead())) {
            continue;
        }
        if (unit.getTeam() == this.getTeam() && unit.getHealth() != null && unit.getHealth() < unit.getMaxHealth()) {
            Integer currentHealth = unit.getHealth();
            int newHealth = (currentHealth == null ? 0 : currentHealth) + healAmount;
            unit.setHealth(newHealth);
            continue;
        }
        if (unit.getTeam() != this.getTeam()) {
            Integer currentHealth = unit.getHealth();
            int newHealth = (currentHealth == null ? 0 : currentHealth) - healAmount;
            unit.setHealth(newHealth);
        }
    }
}

@Override
public boolean isUnitInside(Unit unit) {
    return unitsInside.contains(unit);
}

```

}

Лістинг модуля World.java

```

package org.example.laba5;

import java.util.ArrayList;
import java.util.Iterator;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.shape.Line;
import org.example.laba5.Unit.Unit;
import org.example.laba5.Unit.Warrior;
import org.example.laba5.Unit.Centurio;
import org.example.laba5.Unit.Pretorio;

public class World {
    private ArrayList<Unit> units;

    public int x;
    public int y;
    public Image image;
    public String name;
    public int numUnits;
    public Label labelName;
    public Label numUnitsLabel;
    public Line life;
    public double health;
    public boolean team;

    public Image conturImageRedImage;
    public Image conturImageGreenImage;
    public ImageView contourView;
    public ImageView imageView;
    public double maxHealth;

    public double oreAmount = 0;
    public static int allyUnits = 0;
    public static int enemyUnits = 0;

    public int warriorsTeamA = 0;
    public int warriorsTeamB = 0;
    public int centaursTeamA = 0;
    public int centaursTeamB = 0;
    public int pretionsTeamA = 0;
    public int pretionsTeamB = 0;

    public double getOre() {
        return this.oreAmount;
    }

    public void setOre(double oreAmount) {
        this.oreAmount = oreAmount;
    }

    protected void setMaxHealth(double maxHealth) {
        this.maxHealth = maxHealth;
    }

    public void setHealth(double health) {
        this.health = health;
        updateLifeBar();
    }

    public double getHealth() {
        return health;
    }

    public double getMaxHealth() {
        return maxHealth;
    }

    public boolean getTeam() {
        return team;
    }

    public String getName() {
        return name;
    }

```

```

    }

    public int getX() {
        return x;
    }

    public int getY() {
        return y;
    }

    public ImageView getImageView() {
        return imageView;
    }

    protected void updateLifeBar() {
        if (life == null) {
            return;
        }

        double effectiveMaxHealth = maxHealth > 0.0 ? maxHealth : 100.0;
        double currentHealth = Math.max(0.0, Math.min(health, effectiveMaxHealth));
        double imageHeight = image != null && image.getHeight() > 0 ? image.getHeight() : 100.0;
        double barWidth = (currentHealth / effectiveMaxHealth) * imageHeight;

        double lifeBaseX = x - 5;
        double lifeBaseY = y + 5;
        life.setStartX(lifeBaseX);
        life.setStartY(lifeBaseY);
        life.setEndX(lifeBaseX);
        life.setEndY(lifeBaseY + barWidth);
    }

    public static int objects = Unit.getNumObjects();

    public World() {
        units = new ArrayList<>();
    }

    public World(ArrayList<Unit> units) {
        this.units = units;
    }

    public ArrayList<Unit> getUnits() {
        return units;
    }

    public void setUnits(ArrayList<Unit> units) {
        this.units = units;
    }

    public void printUnits() {
        if (units == null || units.isEmpty()) {
            return;
        }
        for (int i = 0; i < units.size(); i++) {
            System.out.println("Unit " + i + ":");
            System.out.println(units.get(i));
        }
    }

    public void insertUnit(int index, Unit unit) {
        if (units == null) {
            units = new ArrayList<>();
        }
        units.add(index, unit);
    }

    public void update() {
        if (units != null) {
            Iterator<Unit> it = units.iterator();
            while (it.hasNext()) {
                Unit u = it.next();
                if (u == null || Boolean.TRUE.equals(u.getDead()) || (u.getHealth() != null && u.getHealth() <= 0)) {
                    it.remove();
                    Unit.removeUnit();
                }
            }
        }
        updateUnits();
    }

```

```

}

protected void initGraphics(Image image, String name, int numUnits, double x, double y, double maxHealth, double health) {
    this.image = image;
    this.name = name;
    this.numUnits = numUnits;
    this.labelName = new Label(name);
    this.numUnitsLabel = new Label(String.valueOf(numUnits));
    this.life = new Line();
    this.x = (int) x;
    this.y = (int) y;

    setMaxHealth(maxHealth);
    setHealth(health);

    life.setStrokeWidth(5);
    life.setStroke(javafx.scene.paint.Color.GREEN);
    updateLifeBar();

    labelName.setLayoutX(x);
    labelName.setLayoutY(y - 30);
    labelName.setFont(javafx.scene.text.Font.font(20));

    numUnitsLabel.setLayoutX(x + image.getWidth() + 7);
    numUnitsLabel.setLayoutY(y);
    numUnitsLabel.setFont(javafx.scene.text.Font.font(20));
}

protected void resurrectWorld() {
    if (HelloApplication.group == null || labelName == null || life == null || image == null) {
        return;
    }
    imageView = new javafx.scene.image.ImageView(image);
    imageView.setX(x);
    imageView.setY(y);
    HelloApplication.group.getChildren().addAll(labelName, life, imageView, numUnitsLabel);
}

protected void removeBuildingFromGame() {
    if (HelloApplication.group == null) {
        return;
    }
    if (labelName != null) {
        HelloApplication.group.getChildren().remove(labelName);
    }
    if (life != null) {
        HelloApplication.group.getChildren().remove(life);
    }
    if (imageView != null) {
        HelloApplication.group.getChildren().remove(imageView);
    }
    if (numUnitsLabel != null) {
        HelloApplication.group.getChildren().remove(numUnitsLabel);
    }
    if (conturImageGreenImage != null && this.team) {
        HelloApplication.group.getChildren().remove(contourView);
    }
    if (conturImageRedImage != null && !this.team) {
        HelloApplication.group.getChildren().remove(contourView);
    }
    HelloApplication.buildings.remove(this);
    if (this.getClass().getSimpleName().equals("Base") && this.getTeam()) {
        HelloApplication.basesA.remove(this);
    } else if (this.getClass().getSimpleName().equals("Base") && !this.getTeam()) {
        HelloApplication.basesB.remove(this);
    } else if (this.getClass().getSimpleName().equals("Tower") && this.getTeam()) {
        HelloApplication.towersA.remove(this);
    } else if (this.getClass().getSimpleName().equals("Tower") && !this.getTeam()) {
        HelloApplication.towersB.remove(this);
    } else if (this.getClass().getSimpleName().equals("Source") && this.getTeam()) {
        HelloApplication.sourcesA.remove(this);
    } else if (this.getClass().getSimpleName().equals("Source") && !this.getTeam()) {
        HelloApplication.sourcesB.remove(this);
    }
}

protected void intersect() {
}

```

```

private void updateUnits() {
    warriorsTeamA = 0;
    warriorsTeamB = 0;
    centaursTeamA = 0;
    centaursTeamB = 0;
    pretionsTeamA = 0;
    pretionsTeamB = 0;
    allyUnits = 0;
    enemyUnits = 0;
    if (HelloApplication.units == null) return;

    warriorsTeamA = (int) HelloApplication.units.stream().filter(u -> u != null && u.image != null && "Warrior".equals(u.getClass().getSimpleName())
    && u.getTeam()).count();
    warriorsTeamB = (int) HelloApplication.units.stream().filter(u -> u != null && u.image != null && "Warrior".equals(u.getClass().getSimpleName())
    && !u.getTeam()).count();

    centaursTeamA = (int) HelloApplication.units.stream().filter(u -> u != null && u.image != null &&
    "Centurio".equals(u.getClass().getSimpleName()) && u.getTeam()).count();
    centaursTeamB = (int) HelloApplication.units.stream().filter(u -> u != null && u.image != null &&
    "Centurio".equals(u.getClass().getSimpleName()) && !u.getTeam()).count();

    pretionsTeamA = (int) HelloApplication.units.stream().filter(u -> u != null && u.image != null && "Pretorio".equals(u.getClass().getSimpleName())
    && u.getTeam()).count();
    pretionsTeamB = (int) HelloApplication.units.stream().filter(u -> u != null && u.image != null && "Pretorio".equals(u.getClass().getSimpleName())
    && !u.getTeam()).count();

    allyUnits = (int) HelloApplication.units.stream().filter(u -> u != null && u.image != null && u.getTeam()).count();
    enemyUnits = (int) HelloApplication.units.stream().filter(u -> u != null && u.image != null && !u.getTeam()).count();

    HelloApplication.numUnitsTeamA.setText("Team A units: " + allyUnits);
    HelloApplication.numUnitsTeamB.setText("Team B units: " + enemyUnits);
}

public void worldLogic() {
    objects = Unit.getNumObjects();
    updateUnits();

    int oreTeamA = 0;
    int oreTeamB = 0;

    if (HelloApplication.basesA != null && !HelloApplication.basesA.isEmpty()) {
        oreTeamA = (int) HelloApplication.basesA.get(0).getOre();
    }
    if (HelloApplication.basesB != null && !HelloApplication.basesB.isEmpty()) {
        oreTeamB = (int) HelloApplication.basesB.get(0).getOre();
    }

    if (oreTeamA >= 20 && warriorsTeamA < 6 && HelloApplication.basesA != null && !HelloApplication.basesA.isEmpty()) {
        HelloApplication.basesA.get(0).setOre(oreTeamA - 20);
        Unit newUnit = new Warrior();
        newUnit.setTeam(true);
        HelloApplication.units.add(newUnit);
        newUnit.setPosition(HelloApplication.basesA.get(0).x, HelloApplication.basesA.get(0).y);
        newUnit.resurrect();
    }
    if (oreTeamB >= 20 && warriorsTeamB < 6 && HelloApplication.basesB != null && !HelloApplication.basesB.isEmpty()) {
        HelloApplication.basesB.get(0).setOre(oreTeamB - 20);
        Unit newUnit = new Warrior();
        newUnit.setTeam(false);
        HelloApplication.units.add(newUnit);
        newUnit.setPosition(HelloApplication.basesB.get(0).x, HelloApplication.basesB.get(0).y);
        newUnit.resurrect();
    }
    if (oreTeamA >= 50 && centaursTeamA < 2 && HelloApplication.basesA != null && !HelloApplication.basesA.isEmpty()) {
        HelloApplication.basesA.get(0).setOre(oreTeamA - 50);
        Unit newUnit = new Centurio();
        newUnit.setTeam(true);
        HelloApplication.units.add(newUnit);
        newUnit.setPosition(HelloApplication.basesA.get(0).x, HelloApplication.basesA.get(0).y);
        newUnit.resurrect();
    }
    if (oreTeamB >= 50 && centaursTeamB < 2 && HelloApplication.basesB != null && !HelloApplication.basesB.isEmpty()) {
        HelloApplication.basesB.get(0).setOre(oreTeamB - 50);
        Unit newUnit = new Centurio();
        newUnit.setTeam(false);
        HelloApplication.units.add(newUnit);
        newUnit.setPosition(HelloApplication.basesB.get(0).x, HelloApplication.basesB.get(0).y);
        newUnit.resurrect();
    }
}

```

```

if (oreTeamA >= 70 && pretionsTeamA < 10 && HelloApplication.basesA != null && !HelloApplication.basesA.isEmpty()) {
    HelloApplication.basesA.get(0).setOre(oreTeamA - 70);
    Unit newUnit = new Pretorio();
    newUnit.setTeam(true);
    HelloApplication.units.add(newUnit);
    newUnit.setPosition(HelloApplication.basesA.get(0).x, HelloApplication.basesA.get(0).y);
    newUnit.resurrect();
}
if (oreTeamB >= 70 && pretionsTeamB < 10 && HelloApplication.basesB != null && !HelloApplication.basesB.isEmpty()) {
    HelloApplication.basesB.get(0).setOre(oreTeamB - 70);
    Unit newUnit = new Pretorio();
    newUnit.setTeam(false);
    HelloApplication.units.add(newUnit);
    newUnit.setPosition(HelloApplication.basesB.get(0).x, HelloApplication.basesB.get(0).y);
    newUnit.resurrect();
}
}

public boolean isUnitInside(Unit unit) {
    return false;
}

public void sortUnitsList(ArrayList<Unit> units) {
    if (units == null || units.isEmpty()) {
        return;
    }
    units.sort((u1, u2) -> {
        String class1 = u1 == null ? "" : u1.getClass().getSimpleName();
        String class2 = u2 == null ? "" : u2.getClass().getSimpleName();
        int classCmp = class2.compareTo(class1);
        if (classCmp != 0) {
            return classCmp;
        }

        Integer health1 = u1 == null ? null : u1.getHealth();
        Integer health2 = u2 == null ? null : u2.getHealth();
        int healthCmp = Integer.compare(health2 == null ? 0 : health2, health1 == null ? 0 : health1);
        if (healthCmp != 0) {
            return healthCmp;
        }

        Integer damage1 = u1 == null ? null : u1.getDamage();
        Integer damage2 = u2 == null ? null : u2.getDamage();
        return Integer.compare(damage2 == null ? 0 : damage2, damage1 == null ? 0 : damage1);
    });
}
}

```

Лістинг модуля SerializationDialog.java

```

package org.example.laba5;

import java.io.File;
import java.util.Optional;
import javafx.scene.control.Alert;
import javafx.scene.control.ButtonBar;
import javafx.scene.control.ButtonType;
import javafx.stage.FileChooser;
import javafx.stage.Stage;

public class SerializationDialog {

    public static void showDialog(Stage owner) {
        ButtonType btnSave = new ButtonType("Save", ButtonBar.ButtonData.LEFT);
        ButtonType btnImport = new ButtonType("Import", ButtonBar.ButtonData.LEFT);
        ButtonType btnCancel = new ButtonType("Cancel", ButtonBar.ButtonData.CANCEL_CLOSE);

        Alert alert = new Alert(Alert.AlertType.NONE, "What would you like to do?", btnSave, btnImport, btnCancel);
        alert.setTitle("Save / Import");
        alert.setHeaderText("Game Serialization (XML)");

        Optional<ButtonType> result = alert.showAndWait();
        if (result.isEmpty() || result.get() == btnCancel) return;

        boolean isSave = result.get() == btnSave;

        FileChooser fc = new FileChooser();
        fc.getExtensionFilters().add(new FileChooser.ExtensionFilter("XML Files (*.xml)", "*.xml"));
        fc.setTitle(isSave ? "Save Game as XML" : "Import Game from XML");

        File file = isSave ? fc.showSaveDialog(owner) : fc.showOpenDialog(owner);
        if (file == null) return;

        if (isSave && !file.getAbsolutePath().toLowerCase().endsWith(".xml")) {
            file = new File(file.getAbsolutePath() + ".xml");
        }

        if (isSave) {
            GameSerializer.save(file, "XML");
        } else {
            GameSerializer.load(file, "XML");
        }
    }
}

```

Лістинг модуля UnitCreationDialog.java

```

package org.example.laba5;

import java.util.ArrayList;
import java.util.Arrays;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.RadioButton;
import javafx.scene.control.Separator;
import javafx.scene.control.TextField;
import javafx.scene.control.TextFormatter;
import javafx.scene.control.ToggleGroup;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Modality;
import javafx.stage.Stage;
import org.example.laba5.Unit.Unit;
import org.example.laba5.Unit.Warrior;
import org.example.laba5.Unit.Centurio;
import org.example.laba5.Unit.Pretorio;

public class UnitCreationDialog {
    private Stage stage;
    private Unit result = null;
    private boolean confirmed = false;

    private ComboBox<String> unitTypeCombo;
    private TextField healthField;
    private TextField damageField;
    private CheckBox spawnedCheckBox;
    private RadioButton allyRadio;
    private TextField inventorField;
    private HBox oreBoxRow;
    private TextField oreField;
    private HBox killsBox;
    private TextField killsField;

    public UnitCreationDialog() {
        createDialog();
    }

    private void createDialog() {
        stage = new Stage();
        stage.initModality(Modality.APPLICATION_MODAL);
        stage.setTitle("Create Unit");
        stage.setWidth(300);
        stage.setHeight(320);
        stage.setResizable(false);

        VBox root = new VBox(8);
        root.setPadding(new Insets(12));

        HBox typeBox = new HBox(8);
        typeBox.getChildren().add(new Label("Type:"));
        unitTypeCombo = new ComboBox<>();
        unitTypeCombo.getItems().addAll("Warrior", "Centurio", "Pretorio");
        unitTypeCombo.setValue("Warrior");
        unitTypeCombo.setPrefWidth(100);
        unitTypeCombo.setOnAction(e -> updateOreVisibility());
        typeBox.getChildren().add(unitTypeCombo);

        HBox healthBox = new HBox(8);
        healthBox.getChildren().add(new Label("Health:"));
        healthField = new TextField();
        healthField.setText("100");
        healthField.setPrefWidth(100);
        healthField.setTextFormatter(new TextFormatter<>(c -> {
            if (c.getControlNewText().matches("\\d*")) return c;
            return null;
        }

```

```

    ));
    healthBox.getChildren().add(healthField);

    HBox damageBox = new HBox(8);
    damageBox.getChildren().add(new Label("Damage:"));
    damageField = new TextField();
    damageField.setText("5");
    damageField.setPrefWidth(100);
    damageField.setTextFormatter(new TextFormatter<>(c -> {
        if (c.getControlNewText().matches("\\d*")) return c;
        return null;
    }));
    damageBox.getChildren().add(damageField);

    HBox teamBox = new HBox(15);
    teamBox.getChildren().add(new Label("Team:"));
    ToggleGroup teamGroup = new ToggleGroup();
    allyRadio = new RadioButton("Ally");
    allyRadio.setToggleGroup(teamGroup);
    allyRadio.setSelected(true);
    RadioButton enemyRadio = new RadioButton("Enemy");
    enemyRadio.setToggleGroup(teamGroup);
    teamBox.getChildren().addAll(allyRadio, enemyRadio);

    spawnedCheckBox = new CheckBox("Spawned");
    spawnedCheckBox.setSelected(true);

    HBox inventorBox = new HBox(8);
    inventorBox.getChildren().add(new Label("Inventory:"));
    inventorField = new TextField();
    inventorField.setPrefWidth(150);
    inventorBox.getChildren().add(inventorField);

    oreBoxRow = new HBox(8);
    oreBoxRow.getChildren().add(new Label("Ore:"));
    oreField = new TextField();
    oreField.setText("0");
    oreField.setPrefWidth(100);
    oreField.setTextFormatter(new TextFormatter<>(c -> {
        if (c.getControlNewText().matches("\\d*")) return c;
        return null;
    }));
    oreBoxRow.getChildren().add(oreField);

    killsBox = new HBox(8);
    killsBox.getChildren().add(new Label("Kills:"));
    killsField = new TextField();
    killsField.setText("0");
    killsField.setPrefWidth(100);
    killsField.setTextFormatter(new TextFormatter<>(c -> {
        if (c.getControlNewText().matches("\\d*")) return c;
        return null;
    }));
    killsBox.getChildren().add(killsField);

    HBox buttonBox = new HBox(10);
    buttonBox.setStyle("-fx-alignment: center;");
    Button okButton = new Button("OK");
    okButton.setPrefWidth(80);
    okButton.setOnAction(e -> handleOK());
    Button cancelButton = new Button("Cancel");
    cancelButton.setPrefWidth(80);
    cancelButton.setOnAction(e -> handleCancel());
    buttonBox.getChildren().addAll(okButton, cancelButton);

    root.getChildren().addAll(
        typeBox,
        healthBox,
        damageBox,
        teamBox,
        inventorBox,
        new Separator(),
        buttonBox
    );

    updateOreVisibility();
    root.getChildren().add(root.getChildren().indexOf(buttonBox), oreBoxRow);
    root.getChildren().add(root.getChildren().indexOf(buttonBox), killsBox);

```

```

    Scene scene = new Scene(root);
    stage.setScene(scene);
}

private void updateOreVisibility() {
    boolean warriorSelected = "Warrior".equals(unitTypeCombo.getValue());
    boolean centurioSelected = "Centurio".equals(unitTypeCombo.getValue());
    oreBoxRow.setManaged(warriorSelected);
    oreBoxRow.setVisible(warriorSelected);
    killsBox.setManaged(centurioSelected);
    killsBox.setVisible(centurioSelected);
}

private void handleOK() {
    try {
        String unitType = unitTypeCombo.getValue();
        int health = Integer.parseInt(healthField.getText().trim());
        int damage = Integer.parseInt(damageField.getText().trim());
        boolean isSpawned = spawnedCheckBox.isSelected();
        boolean team = allyRadio.isSelected();

        ArrayList<String> inventor;
        String inventorInput = inventorField.getText().trim();
        if (inventorInput.isEmpty()) {
            inventor = new ArrayList<>();
        } else {
            inventor = new ArrayList<>(Arrays.asList(inventorInput.split("\\s*,\\s*")));
        }

        int ore = 0;
        if ("Warrior".equals(unitType)) {
            ore = Integer.parseInt(oreField.getText().trim());
        }

        int kills = 0;
        if ("Centurio".equals(unitType)) {
            kills = Integer.parseInt(killsField.getText().trim());
        }

        switch (unitType) {
            case "Warrior":
                result = new Warrior(health, isSpawned, team, damage, false, inventor, 100, 100);
                ((Warrior) result).setOreCount(ore);
                break;
            case "Centurio":
                result = new Centurio(health, isSpawned, team, damage, false, inventor, 100, 100);
                ((Centurio) result).setKillCount(kills);
                break;
            case "Pretorio":
                result = new Pretorio(health, isSpawned, team, damage, false, inventor, 100, 100);
                break;
        }

        confirmed = true;
        stage.close();

    } catch (NumberFormatException ex) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setTitle("Error");
        alert.setHeaderText("Invalid input");
        alert.setContentText("Health, Damage, and Ore must be numbers.");
        alert.showAndWait();
    }
}

private void handleCancel() {
    confirmed = false;
    result = null;
    stage.close();
}

public Unit showAndWait() {
    try {
        stage.showAndWait();
        return confirmed ? result : null;
    } catch (Exception e) {
        return null;
    }
}

```

}

Лістинг модуля UnitEditDialog.java

```

package org.example.laba5;

import java.util.ArrayList;
import java.util.Arrays;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.RadioButton;
import javafx.scene.control.Separator;
import javafx.scene.control.TextField;
import javafx.scene.control.TextFormatter;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Modality;
import javafx.stage.Stage;
import org.example.laba5.Unit.Unit;
import org.example.laba5.Unit.Warrior;
import org.example.laba5.Unit.Centurio;

public class UnitEditDialog {
    private Stage stage;
    private final Unit unit;
    private boolean confirmed = false;

    private TextField healthField;
    private TextField damageField;
    private TextField inventorField;
    private TextField oreField;
    private HBox oreBox;
    private TextField killsField;
    private HBox killsBox;
    private RadioButton teamButton;

    public UnitEditDialog(Unit unit) {
        this.unit = unit;
        createDialog();
    }

    private void createDialog() {
        stage = new Stage();
        stage.initModality(Modality.APPLICATION_MODAL);
        stage.setTitle("Edit Unit - " + unit.getClass().getSimpleName());
        stage.setWidth(350);
        stage.setHeight(350);
        stage.setResizable(false);

        VBox root = new VBox(10);
        root.setPadding(new Insets(15));

        Label titleLabel = new Label("Unit Type: " + unit.getClass().getSimpleName());

        HBox healthBox = new HBox(10);
        Label healthLabel = new Label("Health:");
        healthLabel.setPrefWidth(80);
        healthField = new TextField();
        healthField.setText(String.valueOf(unit.getHealth()));
        healthField.setPrefWidth(150);
        healthField.setTextFormatter(new TextFormatter<>(c -> {
            if (c.getControlNewText().matches("\\d*")) return c;
            return null;
        }));
        healthBox.getChildren().addAll(healthLabel, healthField);

        HBox damageBox = new HBox(10);
        Label damageLabel = new Label("Damage:");
        damageLabel.setPrefWidth(80);
        damageField = new TextField();
        damageField.setText(String.valueOf(unit.getDamage()));
        damageField.setPrefWidth(150);
        damageField.setTextFormatter(new TextFormatter<>(c -> {
            if (c.getControlNewText().matches("\\d*")) return c;
            return null;
        }));
    }

```

```

    ));
    damageBox.getChildren().addAll(damageLabel, damageField);

    HBox inventorBox = new HBox(10);
    Label inventorLabel = new Label("Inventory:");
    inventorLabel.setPrefWidth(80);
    inventorField = new TextField();
    ArrayList<String> inventor = unit.getInventor();
    if (inventor != null && !inventor.isEmpty()) {
        inventorField.setText(String.join(" ", inventor));
    }
    inventorField.setPrefWidth(200);
    inventorBox.getChildren().addAll(inventorLabel, inventorField);

    oreBox = new HBox(10);
    Label oreLabel = new Label("Ore:");
    oreLabel.setPrefWidth(80);
    oreField = new TextField();
    oreField.setPrefWidth(150);
    oreField.setTextFormatter(new TextFormatter<>(c -> {
        if (c.getControlNewText().matches("\\d*")) return c;
        return null;
    }));
    if (unit.getClass() == Warrior.class) {
        Warrior warrior = (Warrior) unit;
        oreField.setText(String.valueOf((int) warrior.getOre()));
    } else {
        oreField.setText("N/A");
        oreField.setEditable(false);
    }
    oreBox.getChildren().addAll(oreLabel, oreField);

    killsBox = new HBox(10);
    Label killsLabel = new Label("Kills:");
    killsLabel.setPrefWidth(80);
    killsField = new TextField();
    killsField.setPrefWidth(150);
    killsField.setTextFormatter(new TextFormatter<>(c -> {
        if (c.getControlNewText().matches("\\d*")) return c;
        return null;
    }));
    if (unit.getClass() == Centurio.class) {
        Centurio centurio = (Centurio) unit;
        killsField.setText(String.valueOf(centurio.getKillCount()));
    } else {
        killsField.setText("N/A");
        killsField.setEditable(false);
    }
    killsBox.getChildren().addAll(killsLabel, killsField);

    HBox teamBox = new HBox(10);
    Label teamLabel = new Label("Team:");
    teamLabel.setPrefWidth(80);
    teamButton = new RadioButton("Ally");
    teamButton.setSelected(unit.getTeam());

    teamButton.setPrefWidth(150);
    teamBox.getChildren().addAll(teamLabel, teamButton);

    HBox buttonBox = new HBox(10);
    buttonBox.setStyle("-fx-alignment: center;");

    Button okButton = new Button("Apply");
    okButton.setPrefWidth(80);
    okButton.setStyle("-fx-font-size: 12;");
    okButton.setOnAction(e -> handleOK());

    Button cancelButton = new Button("Cancel");
    cancelButton.setPrefWidth(80);
    cancelButton.setStyle("-fx-font-size: 12;");
    cancelButton.setOnAction(e -> handleCancel());

    buttonBox.getChildren().addAll(okButton, cancelButton);

    root.getChildren().addAll(
        titleLabel,
        new Separator(),
        healthBox,
        damageBox,

```

```

        inventorBox,
        oreBox,
        killsBox,
        teamBox,
        new Separator(),
        buttonBox
    );

    Scene scene = new Scene(root);
    stage.setScene(scene);
}

private void handleOK() {
    try {
        boolean oldTeam = unit.getTeam();

        if (!healthField.getText().trim().isEmpty()) {
            int health = Integer.parseInt(healthField.getText().trim());
            unit.setHealth(health);
            unit.setBaseHealth(health);
        }

        if (!damageField.getText().trim().isEmpty()) {
            int damage = Integer.parseInt(damageField.getText().trim());
            unit.setDamage(damage);
            unit.setBaseDamage(damage);
        }

        String inventorInput = inventorField.getText().trim();
        ArrayList<String> inventor;
        if (inventorInput.isEmpty()) {
            inventor = new ArrayList<>();
        } else {
            inventor = new ArrayList<>(Arrays.asList(inventorInput.split("\\s*,\\s*")));
        }
        unit.setInventor(inventor);

        if (unit.getClass() == Warrior.class) {
            Warrior warrior = (Warrior) unit;
            int ore = Integer.parseInt(oreField.getText().trim());
            warrior.setOreCount(ore);
        }

        if (unit.getClass() == Centurio.class) {
            Centurio centurio = (Centurio) unit;
            int kills = Integer.parseInt(killsField.getText().trim());
            centurio.setKillCount(kills);
        }

        boolean newTeam = teamButton.isSelected();
        unit.setTeam(newTeam);

        if (oldTeam != newTeam) {
            unit.updateTeamMark();
        }

        confirmed = true;
        unit.updateTeamMark();
        stage.close();
    } catch (NumberFormatException e) {
    }
}

private void handleCancel() {
    confirmed = false;
    stage.close();
}

public boolean showAndWait() {
    stage.showAndWait();
    return confirmed;
}
}

```

ЛІСТИНГ модуля UnitInventorWindow.java

```

package org.example.laba5;

import java.net.URL;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import org.example.laba5.Unit.Unit;
import org.example.laba5.Unit.Warrior;

public class UnitInventorWindow {
    private final Stage inventoryStage;
    private final Label inventoryTitleLabel;
    private final VBox inventoryItemsBox;
    private HBox oreCountBox;
    private Label oreCountField;
    private final Map<String, Image> inventoryIcons = new HashMap<>();

    public UnitInventorWindow() {
        initInventoryIcons();

        inventoryStage = new Stage();
        inventoryStage.setTitle("Unit Inventory");

        inventoryTitleLabel = new Label("No active unit");
        inventoryItemsBox = new VBox(6);

        VBox root = new VBox(10, inventoryTitleLabel, inventoryItemsBox);
        root.setPadding(new Insets(12));

        Scene inventoryScene = new Scene(root, 280, 280);
        inventoryStage.setScene(inventoryScene);
        inventoryStage.setX(1120);
        inventoryStage.setY(120);
        inventoryStage.hide();
    }

    public void updateFromUnits(List<Unit> units) {
        if (units == null) {
            return;
        }

        Unit activeUnit = units.stream().filter(u -> u != null && u.isActive()).findFirst().orElse(null);

        inventoryItemsBox.getChildren().clear();

        if (activeUnit == null) {
            inventoryTitleLabel.setText("No active unit");
            inventoryItemsBox.getChildren().add(new Label("Select a unit to view inventory"));
            if (isVisible()) {
                setVisible(false);
            }
            return;
        }

        inventoryTitleLabel.setText(activeUnit.getClass().getSimpleName() + " inventory");
        List<String> inventory = activeUnit.getInventor();
        if (inventory == null || inventory.isEmpty()) {
            inventoryItemsBox.getChildren().add(new Label("Inventory is empty"));
        } else {
            inventory.forEach(item -> {
                String itemName = item == null ? "Unknown" : item;
                String iconKey = itemName.trim().toLowerCase();
                Image icon = inventoryIcons.get(iconKey);

                HBox row = new HBox(8);
            });
        }
    }
}

```

```

        if (icon != null) {
            ImageView itemIcon = new ImageView(icon);
            itemIcon.setFitWidth(28);
            itemIcon.setFitHeight(28);
            row.getChildren().add(itemIcon);
        }
        row.getChildren().add(new Label(itemName));
        inventoryItemsBox.getChildren().add(row);
    });
}

if (activeUnit.getClass().getSimpleName().equals("Warrior")) {
    Warrior warrior = (Warrior) activeUnit;

    if (oreCountBox == null) {
        oreCountBox = new HBox(10);
        oreCountBox.setPadding(new Insets(15, 0, 0, 0));
        Label oreCountLabel = new Label("Ore count:");
        oreCountLabel.setPrefWidth(80);
        oreCountField = new Label();
        oreCountBox.getChildren().addAll(oreCountLabel, oreCountField);
    }

    oreCountField.setText(String.valueOf((int) warrior.getOre()));
    inventoryItemsBox.getChildren().add(oreCountBox);
}

}

public boolean showForActiveUnit(List<Unit> units) {
    if (!hasActiveUnit(units)) {
        setVisible(false);
        return false;
    }
    setVisible(true);
    return true;
}

public boolean hasActiveUnit(List<Unit> units) {
    if (units == null) {
        return false;
    }
    return units.stream().anyMatch(u -> u != null && u.isActive());
}

public boolean isVisible() {
    return inventoryStage != null && inventoryStage.isShowing();
}

public void setVisible(boolean visible) {
    if (inventoryStage == null) {
        return;
    }
    if (visible) {
        inventoryStage.show();
        inventoryStage.toFront();
    } else {
        inventoryStage.hide();
    }
}

private void initInventoryIcons() {
    inventoryIcons.clear();
    registerInventoryIcon("sword", "/sword.png");
    registerInventoryIcon("knife", "/knife.png");
    registerInventoryIcon("spear", "/spear.png");
    registerInventoryIcon("bow", "/bow.png");
}

private void registerInventoryIcon(String key, String resourcePath) {
    URL url = HelloApplication.class.getResource(resourcePath);
    if (url != null) {
        inventoryIcons.put(key, new Image(url.toExternalForm(), 28, 28, true, true));
    }
}
}

```

Лістинг модуля UnitSearchWindow.java

```

package org.example.laba5;

import java.util.ArrayList;
import java.util.Objects;
import java.util.List;
import java.util.Locale;
import java.util.stream.Collectors;
import java.util.stream.Stream;
import javafx.animation.KeyFrame;
import javafx.animation.Timeline;
import javafx.collections.FXCollections;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.ListCell;
import javafx.scene.control.ListView;
import javafx.scene.control.TextField;
import javafx.scene.control.TextFormatter;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import javafx.util.Duration;
import org.example.laba5.Unit.Unit;
import org.example.laba5.Unit.Warrior;
import org.example.laba5.Unit.Centurio;
import org.example.laba5.Unit.Pretorio;

public class UnitSearchWindow {
    private final Stage stage;
    private final ComboBox<String> classCombo;
    private final ComboBox<String> teamCombo;
    private final TextField healthField;
    private final TextField damageField;
    private final TextField inventorField;
    private final TextField macroField;
    private final ComboBox<String> sortCombo;
    private final ComboBox<String> countParameterCombo;
    private final ListView<Unit> resultsList;
    private final Label statusLabel;
    private final Label microobjectCountLabel;
    private final Timeline refreshTimer;
    private final World sortHelper = new World();

    public UnitSearchWindow() {
        stage = new Stage();
        stage.setTitle("Unit Search");

        classCombo = new ComboBox<>();
        classCombo.getItems().addAll("Any", "Warrior", "Centurio", "Pretorio");
        classCombo.setValue("Any");

        teamCombo = new ComboBox<>();
        teamCombo.getItems().addAll("Any", "Ally", "Enemy");
        teamCombo.setValue("Any");

        healthField = new TextField();
        healthField.setPromptText("Health");
        healthField.setTextFormatter(new TextFormatter<>(c -> c.getControlNewText().matches("\\d*") ? c : null));

        damageField = new TextField();
        damageField.setPromptText("Damage");
        damageField.setTextFormatter(new TextFormatter<>(c -> c.getControlNewText().matches("\\d*") ? c : null));

        inventorField = new TextField();
        inventorField.setPromptText("Inventory (comma separated)");

        macroField = new TextField();
        macroField.setPromptText("Macroobject name (or none)");

        sortCombo = new ComboBox<>();
        sortCombo.getItems().addAll("Class", "Health", "Damage");
        sortCombo.setValue("Class");
    }

```

```

countParameterCombo = new ComboBox<>();
countParameterCombo.getItems().addAll("moreThanHalf", "haveSword", "team (ally)");
countParameterCombo.setValue("moreThanHalf");

HBox filtersRow1 = new HBox(8, new Label("Class:"), classCombo, new Label("Team:"), teamCombo);
HBox filtersRow2 = new HBox(8, new Label("Health:"), healthField, new Label("Damage:"), damageField);
HBox filtersRow3 = new HBox(8, new Label("Inventory:"), inventorField, new Label("Macro:"), macroField,
    new Label("Sort:"), sortCombo);
Button removeAllButton = new Button("Remove all");
removeAllButton.setOnAction(e -> removeAllFiltered());
Button countMicroobjectsButton = new Button("Count microobjects");
countMicroobjectsButton.setOnAction(e -> countMicroobjectsBySelectedParameter());

resultsList = new ListView<>();
resultsList.setCellFactory(list -> new ListCell<>() {
    @Override
    protected void updateItem(Unit unit, boolean empty) {
        super.updateItem(unit, empty);
        if (empty || unit == null) {
            setText(null);
            return;
        }
        setText(buildUnitDisplay(unit));
    }
});

resultsList.setOnMouseClicked(event -> {
    if (event.getClickCount() < 1) {
        return;
    }
    Unit selected = resultsList.getSelectionModel().getSelectedItem();
    if (selected != null) {
        if (!selected.isActive()) {
            selected.flipActivation();
        }
        setVisible(false);
    }
});

statusLabel = new Label("Matches: 0");
microobjectCountLabel = new Label("Microobjects count: 0");

HBox bottomRow = new HBox(8,
    new Label("Parameter:"),
    countParameterCombo,
    countMicroobjectsButton,
    microobjectCountLabel);

VBox root = new VBox(10, filtersRow1, filtersRow2, filtersRow3, removeAllButton, statusLabel, resultsList, bottomRow);
root.setPadding(new Insets(12));

Scene scene = new Scene(root, 720, 420);
stage.setScene(scene);

classCombo.valueProperty().addListener((obs, oldVal, newVal) -> refreshList(true));
teamCombo.valueProperty().addListener((obs, oldVal, newVal) -> refreshList(true));
healthField.textProperty().addListener((obs, oldVal, newVal) -> refreshList(true));
damageField.textProperty().addListener((obs, oldVal, newVal) -> refreshList(true));
inventorField.textProperty().addListener((obs, oldVal, newVal) -> refreshList(true));
macroField.textProperty().addListener((obs, oldVal, newVal) -> refreshList(true));
sortCombo.valueProperty().addListener((obs, oldVal, newVal) -> refreshList(true));

refreshTimer = new Timeline(new KeyFrame(Duration.millis(300), e -> refreshList(false)));
refreshTimer.setCycleCount(Timeline.INDEFINITE);

stage.setOnShown(e -> {
    refreshList(true);
    refreshTimer.play();
});
stage.setOnHidden(e -> refreshTimer.stop());
}

public void show() {
    stage.show();
    stage.toFront();
}

public boolean isVisible() {

```

```

    return stage.isShowing();
}

public void setVisible(boolean visible) {
    if (visible) {
        show();
    } else {
        stage.hide();
    }
}

private void refreshList(boolean resetSelection) {
    Unit selected = resetSelection ? null : resultsList.getSelectionModel().getSelectedItem();
    ArrayList<Unit> filtered = filterUnits();
    sortFilteredUnits(filtered);

    resultsList.setItems(FXCollections.observableArrayList(filtered));
    statusLabel.setText("Matches: " + filtered.size());

    if (!resetSelection && selected != null) {
        for (Unit unit : filtered) {
            if (unit == selected) {
                resultsList.getSelectionModel().select(unit);
                break;
            }
        }
    }
}

private ArrayList<Unit> filterUnits() {
    if (HelloApplication.units == null) {
        return new ArrayList<>();
    }

    String classFilter = classCombo.getValue();
    String teamFilter = teamCombo.getValue();
    Integer healthFilter = parseIntOrNull(healthField.getText());
    Integer damageFilter = parseIntOrNull(damageField.getText());
    List<String> inventorFilter = parseInventory(inventorField.getText());
    String macroFilter = normalizeMacroFilter(macroField.getText());

    return HelloApplication.units.stream()
        .filter(u -> u != null && !Boolean.TRUE.equals(u.getDead()))
        .filter(u -> "Any".equals(classFilter) || u.getClass().getSimpleName().equals(classFilter))
        .filter(u -> "Any".equals(teamFilter) || u.getTeam() == ("Ally".equals(teamFilter)))
        .filter(u -> healthFilter == null || (u.getHealth() != null && u.getHealth().equals(healthFilter)))
        .filter(u -> damageFilter == null || (u.getDamage() != null && u.getDamage().equals(damageFilter)))
        .filter(u -> {
            if (inventorFilter.isEmpty()) return true;
            ArrayList<String> inv = u.getInventor();
            if (inv == null) return false;
            return inv.stream().filter(Objects::nonNull).map(s ->
s.trim()).toLowerCase(Locale.ROOT)).collect(Collectors.toSet()).containsAll(inventorFilter);
        })
        .filter(u -> {
            if (macroFilter == null) return true;
            if (isNoneFilter(macroFilter)) return !hasMacroMembership(u);
            return isUnitInMacro(u, macroFilter);
        })
        .collect(Collectors.toCollection(ArrayList::new));
}

private String buildUnitDisplay(Unit unit) {
    String className = unit.getClass().getSimpleName();
    Integer health = unit.getHealth();
    Integer damage = unit.getDamage();
    String team = unit.getTeam() ? "Ally" : "Enemy";
    String inventor = unit.getInventor() == null ? "[]" : unit.getInventor().toString();
    String macros = buildMacroMembership(unit);

    return className
        + " | health=" + (health == null ? "null" : health)
        + " | damage=" + (damage == null ? "null" : damage)
        + " | team=" + team
        + " | inventory=" + inventor
        + " | x=" + Math.round(unit.getX())
        + " | y=" + Math.round(unit.getY())
        + " | in=" + macros;
}

```

```

private String buildMacroMembership(Unit unit) {
    if (HelloApplication.buildings == null || HelloApplication.buildings.isEmpty()) {
        return "none";
    }
    List<String> memberships = HelloApplication.buildings.stream()
        .filter(w -> w != null && w.isUnitInside(unit))
        .map(w -> {
            String team = w.getTeam() ? "Ally" : "Enemy";
            String macroName = (w.name == null || w.name.isBlank()) ? w.getClass().getSimpleName() : w.name;
            return macroName + "(" + team + ")";
        })
        .collect(Collectors.toList());
    if (memberships.isEmpty()) return "none";
    return String.join(", ", memberships);
}

private String normalizeMacroFilter(String value) {
    if (value == null) {
        return null;
    }
    String trimmed = value.trim().toLowerCase(Locale.ROOT);
    return trimmed.isEmpty() ? null : trimmed;
}

private boolean isNoneFilter(String macroFilter) {
    return "none".equals(macroFilter) || "no".equals(macroFilter) || "empty".equals(macroFilter);
}

private boolean hasMacroMembership(Unit unit) {
    if (HelloApplication.buildings == null || HelloApplication.buildings.isEmpty()) return false;
    return HelloApplication.buildings.stream().filter(Objects::nonNull).anyMatch(w -> w.isUnitInside(unit));
}

private void sortFilteredUnits(ArrayList<Unit> filtered) {
    if (filtered == null || filtered.isEmpty()) {
        return;
    }
    String sortBy = sortCombo.getValue();
    if ("Health".equals(sortBy)) {
        filtered.sort((u1, u2) -> Integer.compare(
            u2 == null || u2.getHealth() == null ? 0 : u2.getHealth(),
            u1 == null || u1.getHealth() == null ? 0 : u1.getHealth()));
        return;
    }
    if ("Damage".equals(sortBy)) {
        filtered.sort((u1, u2) -> Integer.compare(
            u2 == null || u2.getDamage() == null ? 0 : u2.getDamage(),
            u1 == null || u1.getDamage() == null ? 0 : u1.getDamage()));
        return;
    }
    if ("Class".equals(sortBy)) {
        filtered.sort((u1, u2) -> Integer.compare(getHierarchyRank(u2), getHierarchyRank(u1)));
        return;
    }
    sortHelper.sortUnitsList(filtered);
}

private int getHierarchyRank(Unit unit) {
    if (unit == null) {
        return 0;
    }
    if (unit instanceof Pretorio) {
        return 3;
    }
    if (unit instanceof Centurio) {
        return 2;
    }
    if (unit instanceof Warrior) {
        return 1;
    }
    return 0;
}

private boolean isUnitInMacro(Unit unit, String macroFilter) {
    if (HelloApplication.buildings == null || HelloApplication.buildings.isEmpty()) return false;
    return HelloApplication.buildings.stream()
        .filter(Objects::nonNull)
        .filter(w -> w.isUnitInside(unit))

```

```

        .map(w -> (w.name == null || w.name.isBlank()) ? w.getClass().getSimpleName() : w.name)
        .map(n -> n.trim().toLowerCase(Locale.ROOT))
        .anyMatch(n -> n.contains(macroFilter));
    }

    private Integer parseIntOrNull(String value) {
        if (value == null) {
            return null;
        }
        String trimmed = value.trim();
        if (trimmed.isEmpty()) {
            return null;
        }
        try {
            return Integer.parseInt(trimmed);
        } catch (NumberFormatException ex) {
            return null;
        }
    }

    private List<String> parseInventory(String input) {
        if (input == null || input.trim().isEmpty()) return new ArrayList<>();
        return Stream.of(input.split(","))
            .filter(Objects::nonNull)
            .map(String::trim)
            .filter(s -> !s.isEmpty())
            .map(s -> s.toLowerCase(Locale.ROOT))
            .collect(Collectors.toList());
    }

    private void removeAllFiltered() {
        ArrayList<Unit> filtered = filterUnits();
        filtered.stream().filter(u -> u != null && !Boolean.TRUE.equals(u.getDead())).forEach(Unit::removeUnitFromGame);
        refreshList(true);
    }

    private void countMicroobjectsBySelectedParameter() {
        if (HelloApplication.units == null || HelloApplication.units.isEmpty()) {
            microobjectCountLabel.setText("Microobjects count: 0");
            return;
        }

        String parameter = countParameterCombo.getValue();
        long count = HelloApplication.units.stream().filter(u -> u != null && !Boolean.TRUE.equals(u.getDead())).filter(u -> {
            switch (parameter) {
                case "moreThanHalf":
                    return u.moreThanHalf();
                case "haveSword":
                    return u.haveSword();
                case "team (ally)":
                    return u.getTeam();
                default:
                    return false;
            }
        }).count();

        microobjectCountLabel.setText("Microobjects count: " + count);
    }
}

```

ЛІСТИНГ модуля GameSerializer.java

```

package org.example.laba5;

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.Arrays;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import javafx.application.Platform;
import javafx.scene.control.Alert;
import javafx.scene.image.Image;
import org.example.laba5.Unit.Centurio;
import org.example.laba5.Unit.Unit;
import org.example.laba5.Unit.Warrior;
import org.example.laba5.Unit.Pretorio;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;

public class GameSerializer {

    public static void save(File file, String format) {
        try {
            switch (format.toUpperCase()) {
                case "TEXT" -> saveText(file);
                case "BINARY" -> saveBinary(file);
                case "XML" -> saveXml(file);
                default -> throw new IllegalArgumentException("Unknown format: " + format);
            }
            showInfo("Save successful", "Game saved to:\n" + file.getAbsolutePath());
        } catch (Exception e) {
            showError("Save failed", e.getMessage());
            e.printStackTrace();
        }
    }

    public static void load(File file, String format) {
        try {
            clearGame();
            switch (format.toUpperCase()) {
                case "TEXT" -> loadText(file);
                case "BINARY" -> loadBinary(file);
                case "XML" -> loadXml(file);
                default -> throw new IllegalArgumentException("Unknown format: " + format);
            }
            bringUnitsToFront();
            showInfo("Load successful", "Game loaded from:\n" + file.getAbsolutePath());
        } catch (Exception e) {
            showError("Load failed", e.getMessage());
            e.printStackTrace();
        }
    }

    private static void saveText(File file) throws IOException {
        try (PrintWriter pw = new PrintWriter(new OutputStreamWriter(new FileOutputStream(file), StandardCharsets.UTF_8))) {
            for (Unit u : HelloApplication.units) {
                if (u == null) continue;
                pw.println("[UNIT]");
                pw.println("type=" + u.getClass().getSimpleName());
                pw.println("x=" + u.x);
                pw.println("y=" + u.y);
                pw.println("health=" + u.getHealth());
                pw.println("damage=" + u.getDamage());
                pw.println("maxHealth=" + u.getMaxHealth());
                pw.println("team=" + u.getTeam());
                pw.println("isDead=" + u.getDead());
                pw.println("isSpawned=" + u.getSpawned());
                pw.println("inventor=" + String.join(",", u.getInventor()));
            }
        }
    }
}

```

```

        pw.println("/UNIT");
    }
    for (World w : HelloApplication.buildings) {
        if (w == null) continue;
        pw.println("[BUILDING]");
        pw.println("type=" + w.getClass().getSimpleName());
        pw.println("name=" + w.name);
        pw.println("x=" + w.x);
        pw.println("y=" + w.y);
        pw.println("health=" + w.getHealth());
        pw.println("maxHealth=" + w.getMaxHealth());
        pw.println("team=" + w.getTeam());
        pw.println("ore=" + w.getOre());
        pw.println("/BUILDING");
    }
}

private static void loadText(File file) throws IOException {
    try (BufferedReader br = new BufferedReader(new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8))) {
        String line;
        boolean inUnit = false, inBuilding = false;
        String type = null, name = null;
        double x = 0, y = 0, maxHealth = 100, health = 100, ore = 0;
        int damage = 5;
        boolean team = true, isDead = false, isSpawned = false;
        ArrayList<String> inventor = new ArrayList<>();

        while ((line = br.readLine()) != null) {
            line = line.trim();
            if (line.equals("[UNIT]")) {
                inUnit = true;
                type = null; x = 0; y = 0; maxHealth = 100; health = 100;
                damage = 5; team = true; isDead = false; isSpawned = false;
                inventor = new ArrayList<>();
            } else if (line.equals("/UNIT")) {
                inUnit = false;
                spawnLoadedUnit(type, x, y, (int) health, damage, (int) maxHealth, team, isDead, isSpawned, inventor);
            } else if (line.equals("[BUILDING]")) {
                inBuilding = true;
                type = null; name = ""; x = 0; y = 0; maxHealth = 200; health = 200; ore = 0; team = true;
            } else if (line.equals("/BUILDING")) {
                inBuilding = false;
                spawnLoadedBuilding(type, name, x, y, maxHealth, health, team, ore);
            } else if (inUnit || inBuilding) {
                int eq = line.indexOf('=');
                if (eq < 0) continue;
                String key = line.substring(0, eq).trim();
                String val = line.substring(eq + 1).trim();
                switch (key) {
                    case "type" -> type = val;
                    case "name" -> name = val;
                    case "x" -> x = Double.parseDouble(val);
                    case "y" -> y = Double.parseDouble(val);
                    case "health" -> health = Double.parseDouble(val);
                    case "maxHealth" -> maxHealth = Double.parseDouble(val);
                    case "damage" -> damage = Integer.parseInt(val);
                    case "team" -> team = Boolean.parseBoolean(val);
                    case "isDead" -> isDead = Boolean.parseBoolean(val);
                    case "isSpawned" -> isSpawned = Boolean.parseBoolean(val);
                    case "ore" -> ore = Double.parseDouble(val);
                    case "inventor" -> {
                        inventor = new ArrayList<>();
                        if (!val.isEmpty()) inventor.addAll(Arrays.asList(val.split(",")));
                    }
                }
            }
        }
    }
}

private static final int MAGIC = 0xCAFEBADE;

private static void saveBinary(File file) throws IOException {
    try (DataOutputStream dos = new DataOutputStream(new BufferedOutputStream(new FileOutputStream(file)))) {
        dos.writeInt(MAGIC);
        dos.writeInt(HelloApplication.units.size());
        for (Unit u : HelloApplication.units) {
            if (u == null) { dos.writeUTF("null"); continue; }

```

```

        dos.writeUTF(u.getClass().getSimpleName());
        dos.writeDouble(u.x);
        dos.writeDouble(u.y);
        dos.writeInt(u.getHealth() != null ? u.getHealth() : 0);
        dos.writeInt(u.getDamage() != null ? u.getDamage() : 0);
        dos.writeDouble(u.getMaxHealth());
        dos.writeBoolean(u.getTeam());
        dos.writeBoolean(Boolean.TRUE.equals(u.getDead()));
        dos.writeBoolean(Boolean.TRUE.equals(u.getSpawned()));
        ArrayList<String> inv = u.getInventor();
        dos.writeInt(inv == null ? 0 : inv.size());
        if (inv != null) for (String s : inv) dos.writeUTF(s);
    }
    dos.writeInt(HelloApplication.buildings.size());
    for (World w : HelloApplication.buildings) {
        if (w == null) { dos.writeUTF("null"); continue; }
        dos.writeUTF(w.getClass().getSimpleName());
        dos.writeUTF(w.name != null ? w.name : "");
        dos.writeDouble(w.x);
        dos.writeDouble(w.y);
        dos.writeDouble(w.getHealth());
        dos.writeDouble(w.getMaxHealth());
        dos.writeBoolean(w.getTeam());
        dos.writeDouble(w.getOre());
    }
}

private static void loadBinary(File file) throws IOException {
    try (DataInputStream dis = new DataInputStream(new BufferedInputStream(new FileInputStream(file)))) {
        int magic = dis.readInt();
        if (magic != MAGIC) throw new IOException("Invalid binary save file (bad magic number).");
        int unitCount = dis.readInt();
        for (int i = 0; i < unitCount; i++) {
            String type = dis.readUTF();
            if ("null".equals(type)) continue;
            double ux = dis.readDouble();
            double uy = dis.readDouble();
            int health = dis.readInt();
            int damage = dis.readInt();
            double maxHp = dis.readDouble();
            boolean team = dis.readBoolean();
            boolean isDead = dis.readBoolean();
            boolean spawned = dis.readBoolean();
            int invSize = dis.readInt();
            ArrayList<String> inv = new ArrayList<>();
            for (int j = 0; j < invSize; j++) inv.add(dis.readUTF());
            spawnLoadedUnit(type, ux, uy, health, damage, (int) maxHp, team, isDead, spawned, inv);
        }
        int buildCount = dis.readInt();
        for (int i = 0; i < buildCount; i++) {
            String type = dis.readUTF();
            if ("null".equals(type)) continue;
            String name = dis.readUTF();
            double bx = dis.readDouble();
            double by = dis.readDouble();
            double health = dis.readDouble();
            double maxHp = dis.readDouble();
            boolean team = dis.readBoolean();
            double ore = dis.readDouble();
            spawnLoadedBuilding(type, name, bx, by, maxHp, health, team, ore);
        }
    }
}

private static void saveXml(File file) throws Exception {
    DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
    DocumentBuilder db = dbf.newDocumentBuilder();
    Document doc = db.newDocument();

    Element root = doc.createElement("GameState");
    doc.appendChild(root);

    Element unitsEl = doc.createElement("Units");
    root.appendChild(unitsEl);
    for (Unit u : HelloApplication.units) {
        if (u == null) continue;
        Element el = doc.createElement("Unit");
        el.setAttribute("type", u.getClass().getSimpleName());
    }
}

```

```

        el.setAttribute("x",    String.valueOf(u.x));
        el.setAttribute("y",    String.valueOf(u.y));
        el.setAttribute("health", String.valueOf(u.getHealth()));
        el.setAttribute("damage", String.valueOf(u.getDamage()));
        el.setAttribute("maxHealth", String.valueOf(u.getMaxHealth()));
        el.setAttribute("team", String.valueOf(u.getTeam()));
        el.setAttribute("isDead", String.valueOf(Boolean.TRUE.equals(u.getDead())));
        el.setAttribute("isSpawned", String.valueOf(Boolean.TRUE.equals(u.getSpawned())));
        el.setAttribute("inventor", String.join(", ", u.getInventor()));
        unitsEl.appendChild(el);
    }

    Element buildingsEl = doc.createElement("Buildings");
    root.appendChild(buildingsEl);
    for (World w : HelloApplication.buildings) {
        if (w == null) continue;
        Element el = doc.createElement("Building");
        el.setAttribute("type", w.getClass().getSimpleName());
        el.setAttribute("name", w.name != null ? w.name : "");
        el.setAttribute("x",    String.valueOf(w.x));
        el.setAttribute("y",    String.valueOf(w.y));
        el.setAttribute("health", String.valueOf(w.getHealth()));
        el.setAttribute("maxHealth", String.valueOf(w.getMaxHealth()));
        el.setAttribute("team", String.valueOf(w.getTeam()));
        el.setAttribute("ore", String.valueOf(w.getOre()));
        buildingsEl.appendChild(el);
    }

    Transformer t = TransformerFactory.newInstance().newTransformer();
    t.setOutputProperty(OutputKeys.INDENT, "yes");
    t.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "2");
    t.transform(new DOMSource(doc), new StreamResult(file));
}

private static void loadXml(File file) throws Exception {
    DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
    DocumentBuilder db = dbf.newDocumentBuilder();
    Document doc = db.parse(file);
    doc.getDocumentElement().normalize();

    NodeList unitNodes = doc.getElementsByTagName("Unit");
    for (int i = 0; i < unitNodes.getLength(); i++) {
        Element el = (Element) unitNodes.item(i);
        String type = el.getAttribute("type");
        double x = Double.parseDouble(el.getAttribute("x"));
        double y = Double.parseDouble(el.getAttribute("y"));
        int health = Integer.parseInt(el.getAttribute("health"));
        int damage = Integer.parseInt(el.getAttribute("damage"));
        double maxHealth = Double.parseDouble(el.getAttribute("maxHealth"));
        boolean team = Boolean.parseBoolean(el.getAttribute("team"));
        boolean isDead = Boolean.parseBoolean(el.getAttribute("isDead"));
        boolean spawned = Boolean.parseBoolean(el.getAttribute("isSpawned"));
        String invStr = el.getAttribute("inventor");
        ArrayList<String> inv = new ArrayList<>();
        if (!invStr.isEmpty()) inv.addAll(Arrays.asList(invStr.split(",")));
        spawnLoadedUnit(type, x, y, health, damage, (int) maxHealth, team, isDead, spawned, inv);
    }

    NodeList buildNodes = doc.getElementsByTagName("Building");
    for (int i = 0; i < buildNodes.getLength(); i++) {
        Element el = (Element) buildNodes.item(i);
        String type = el.getAttribute("type");
        String name = el.getAttribute("name");
        double x = Double.parseDouble(el.getAttribute("x"));
        double y = Double.parseDouble(el.getAttribute("y"));
        double health = Double.parseDouble(el.getAttribute("health"));
        double maxHealth = Double.parseDouble(el.getAttribute("maxHealth"));
        boolean team = Boolean.parseBoolean(el.getAttribute("team"));
        double ore = Double.parseDouble(el.getAttribute("ore"));
        spawnLoadedBuilding(type, name, x, y, maxHealth, health, team, ore);
    }
}

private static void spawnLoadedUnit(String type, double x, double y,
    int health, int damage, int maxHealth,
    boolean team, boolean isDead, boolean spawned,
    ArrayList<String> inventor) {
    Unit unit = switch (type) {
        case "Warrior" -> new Warrior(health, spawned, team, damage, isDead, inventor, x, y);
    }
}

```

```

        case "Centurio" -> new Centurio(health, spawned, team, damage, isDead, inventor, x, y);
        case "Pretorio" -> new Pretorio(health, spawned, team, damage, isDead, inventor, x, y);
        default -> {
            System.err.println("GameSerializer: unknown unit type: " + type);
            yield null;
        }
    };
    if (unit == null) return;
    unit.setTeam(team);
    unit.setDead(isDead);
    unit.setSpawned(spawned);
    unit.setBaseHealth(health);
    unit.setBaseDamage(damage);
    unit.setDamage(damage);
    unit.setMaxHealth(maxHealth);
    unit.setInventor(inventor);
    HelloApplication.units.add(unit);
    unit.setPosition(x, y);
    unit.resurrect();
}

private static void spawnLoadedBuilding(String type, String name,
                                         double x, double y,
                                         double maxHealth, double health,
                                         boolean team, double ore) {
    Image imgBase = HelloApplication.imgBase;
    Image imgTower = HelloApplication.imgTower;
    Image imgSource = HelloApplication.imgSource;

    World world = switch (type) {
        case "Base" -> {
            Base b = new Base();
            b.setTeam(team);
            b.initGraphics(imgBase, name, 0, x, y, maxHealth, health);
            yield b;
        }
        case "Tower" -> {
            Tower t = new Tower();
            t.setTeam(team);
            t.initGraphics(imgTower, name, 0, x, y, maxHealth, health);
            yield t;
        }
        case "Source" -> {
            Source s = new Source();
            s.setTeam(team);
            s.initGraphics(imgSource, name, 0, x, y, maxHealth, health);
            yield s;
        }
        default -> {
            System.err.println("GameSerializer: unknown building type: " + type);
            yield null;
        }
    };
    if (world == null) return;
    world.setOre(ore);
    world.resurrectWorld();
    HelloApplication.buildings.add(world);
    if (world instanceof Base b) {
        if (team) HelloApplication.basesA.add(b);
        else HelloApplication.basesB.add(b);
    } else if (world instanceof Tower t) {
        if (team) HelloApplication.towersA.add(t);
        else HelloApplication.towersB.add(t);
    } else if (world instanceof Source s) {
        if (team) HelloApplication.sourcesA.add(s);
        else HelloApplication.sourcesB.add(s);
    }
}

private static void bringUnitsToFront() {
    for (Unit u : HelloApplication.units) {
        if (u == null) continue;
        if (u.mainWeaponImage != null) u.mainWeaponImageToFront();
        if (u.imageMark != null) u.imageMarkToFront();
        if (u.life != null) u.lifeToFront();
        if (u.labelName != null) u.labelNameToFront();
        if (u.rectActive != null) u.rectActiveToFront();
        if (u.image != null) u.imageToFront();
        if (u.getOreCountLabel() != null) u.getOreCountLabel().ToFront();
    }
}

```

```

        if (u.getKillCountLabel() != null) u.getKillCountLabel().toFront();
    }
}

private static void clearGame() {
    for (Unit u : new ArrayList<>(HelloApplication.units)) {
        if (u != null) u.removeUnitFromGame();
    }
    HelloApplication.units.clear();
    for (World w : new ArrayList<>(HelloApplication.buildings)) {
        if (w != null) w.removeBuildingFromGame();
    }
    HelloApplication.buildings.clear();
    HelloApplication.basesA.clear();
    HelloApplication.basesB.clear();
    HelloApplication.towersA.clear();
    HelloApplication.towersB.clear();
    HelloApplication.sourcesA.clear();
    HelloApplication.sourcesB.clear();
}

private static void showInfo(String title, String msg) {
    Platform.runLater() -> {
        Alert a = new Alert(Alert.AlertType.INFORMATION);
        a.setTitle(title);
        a.setHeaderText(null);
        a.setContentText(msg);
        a.showAndWait();
    });
}

private static void showError(String title, String msg) {
    Platform.runLater() -> {
        Alert a = new Alert(Alert.AlertType.ERROR);
        a.setTitle(title);
        a.setHeaderText(null);
        a.setContentText(msg != null ? msg : "Unknown error");
        a.showAndWait();
    });
}
}

```

Лістинг модуля Launcher.java

```
package org.example.laba5;

import javafx.application.Application;

public class Launcher {
    public static void main(String[] args) {
        Application.launch(HelloApplication.class, args);
    }
}
```

Лістинг модуля MiniMapOverlay.java

```

package org.example.laba5;

import java.util.ArrayList;
import java.util.Collections;
import java.util.IdentityHashMap;
import java.util.Iterator;
import java.util.Set;
import java.util.function.Consumer;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.layout.Background;
import javafx.scene.layout.BackgroundFill;
import javafx.scene.layout.CornerRadii;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Polygon;
import javafx.scene.shape.Rectangle;
import org.example.laba5.Unit.Unit;
import org.example.laba5.Unit.Centurio;
import org.example.laba5.Unit.Pretorio;

public class MiniMapOverlay {
    private static final double SCALE = 0.08;
    private static final double MARGIN = 12.0;

    private final Pane root;
    private final Pane buildingLayer;
    private final Pane unitLayer;
    private final Rectangle background;
    private final Rectangle border;
    private final Rectangle viewportRect;
    private final Consumer<double[]> onNavigate;

    private final IdentityHashMap<Unit, Polygon> unitMarkers = new IdentityHashMap<>();
    private final IdentityHashMap<World, Polygon> buildingMarkers = new IdentityHashMap<>();

    private double worldWidth;
    private double worldHeight;
    private double cameraX;
    private double cameraY;
    private double viewportWidth;
    private double viewportHeight;

    public MiniMapOverlay(double initialWorldWidth, double initialWorldHeight, double viewportWidth, double viewportHeight, Consumer<double[]>
onNavigate) {
        this.worldWidth = Math.max(1.0, initialWorldWidth);
        this.worldHeight = Math.max(1.0, initialWorldHeight);
        this.viewportWidth = Math.max(1.0, viewportWidth);
        this.viewportHeight = Math.max(1.0, viewportHeight);
        this.onNavigate = onNavigate;

        double mapWidth = this.worldWidth * SCALE;
        double mapHeight = this.worldHeight * SCALE;

        root = new Pane();
        root.setPrefSize(mapWidth, mapHeight + 20);
        root.setMinSize(mapWidth, mapHeight + 20);
        root.setMaxSize(mapWidth, mapHeight + 20);
        root.setBackground(new Background(new BackgroundFill(Color.TRANSPARENT, CornerRadii.EMPTY, Insets.EMPTY)));

        background = new Rectangle(0, 20, mapWidth, mapHeight);
        background.setFill(Color.rgb(14, 22, 30, 0.85));
        background.setArcWidth(12);
        background.setArcHeight(12);

        border = new Rectangle(0, 20, mapWidth, mapHeight);
        border.setFill(Color.TRANSPARENT);
        border.setStroke(Color.rgb(120, 200, 255, 0.9));
        border.setStrokeWidth(2.0);
        border.setArcWidth(12);
        border.setArcHeight(12);

        viewportRect = new Rectangle();

```

```

viewportRect.setFill(Color.color(0.2, 0.9, 1.0, 0.14));
viewportRect.setStroke(Color.web("#7ef0ff"));
viewportRect.setStrokeWidth(1.6);
viewportRect.setArcWidth(8);
viewportRect.setArcHeight(8);

buildingLayer = new Pane();
buildingLayer.setLayoutY(20);
buildingLayer.setPickOnBounds(false);

unitLayer = new Pane();
unitLayer.setLayoutY(20);
unitLayer.setPickOnBounds(false);

root.getChildren().addAll(background, buildingLayer, unitLayer, viewportRect, border);

root.setOnMouseClicked(event -> {
    double mx = clamp(event.getX(), 0, background.getWidth());
    double my = clamp(event.getY() - 20, 0, background.getHeight());
    double targetX = miniToWorldX(mx);
    double targetY = miniToWorldY(my);
    if (onNavigate != null) {
        onNavigate.accept(new double[] { targetX, targetY });
    }
});
}

public Pane getPane() {
    return root;
}

public void bindToScene(Scene scene) {
    if (scene == null) {
        return;
    }
    positionInCorner(scene);
    scene.widthProperty().addListener((obs, oldVal, newVal) -> positionInCorner(scene));
    scene.heightProperty().addListener((obs, oldVal, newVal) -> positionInCorner(scene));
}

public void toggleVisible() {
    root.setVisible(!root.isVisible());
}

public void update(ArrayList<Unit> units, ArrayList<World> buildings) {
    ArrayList<Unit> unitsSnapshot = units == null ? new ArrayList<>() : new ArrayList<>(units);
    ArrayList<World> buildingsSnapshot = buildings == null ? new ArrayList<>() : new ArrayList<>(buildings);

    recomputeWorldBounds(unitsSnapshot, buildingsSnapshot);
    updateBuildings(buildingsSnapshot);
    updateUnits(unitsSnapshot);
    updateViewportRect();
}

public void setCameraPosition(double cameraX, double cameraY) {
    this.cameraX = cameraX;
    this.cameraY = cameraY;
    updateViewportRect();
}

public void setViewportSize(double width, double height) {
    this.viewportWidth = Math.max(1.0, width);
    this.viewportHeight = Math.max(1.0, height);
    updateViewportRect();
}

public void navigateToWorldPoint(double targetX, double targetY) {
    double desiredX = targetX - viewportWidth / 2.0;
    double desiredY = targetY - viewportHeight / 2.0;
    double maxX = Math.max(0.0, worldWidth - viewportWidth);
    double maxY = Math.max(0.0, worldHeight - viewportHeight);
    setCameraPosition(clamp(desiredX, 0.0, maxX), clamp(desiredY, 0.0, maxY));
}

private void positionInCorner(Scene scene) {
    root.setLayoutX(scene.getWidth() - root.getPrefWidth() - MARGIN);
    root.setLayoutY(MARGIN);
}

```

```

private void recomputeWorldBounds(ArrayList<Unit> units, ArrayList<World> buildings) {
    double maxX = worldWidth;
    double maxY = worldHeight;

    for (World world : buildings) {
        if (world == null) {
            continue;
        }
        double w = world.image == null ? 200.0 : world.image.getWidth();
        double h = world.image == null ? 200.0 : world.image.getHeight();
        maxX = Math.max(maxX, world.x + w + 60.0);
        maxY = Math.max(maxY, world.y + h + 60.0);
    }

    for (Unit unit : units) {
        if (unit == null) {
            continue;
        }
        double uw = unit.getImage() == null ? 100.0 : unit.getImage().getFitWidth();
        double uh = unit.getImage() == null ? 100.0 : unit.getImage().getFitHeight();
        maxX = Math.max(maxX, unit.x + uw + 40.0);
        maxY = Math.max(maxY, unit.y + uh + 40.0);
    }

    worldWidth = Math.max(1.0, maxX);
    worldHeight = Math.max(1.0, maxY);
}

private void updateBuildings(ArrayList<World> buildings) {
    Set<World> alive = Collections.newSetFromMap(new IdentityHashMap<>());

    for (World world : buildings) {
        if (world == null) {
            continue;
        }
        alive.add(world);

        Polygon marker = buildingMarkers.get(world);
        if (marker == null) {
            marker = new Polygon();
            marker.setOpacity(0.8);
            buildingMarkers.put(world, marker);
            buildingLayer.getChildren().add(marker);
        }

        double size = getBuildingMarkerSize(world);
        updateBuildingMarkerShape(marker, world, size);
        marker.setFill(world.getTeam() ? Color.web("#6fe3a4") : Color.web("#d46a6a"));
        marker.setStroke(Color.color(0, 0, 0.45));
        marker.setStrokeWidth(1.0);
        marker.setLayoutX(worldToMiniX(world.x) + size * 0.5);
        marker.setLayoutY(worldToMiniY(world.y) + size * 0.5);
    }

    Iterator<World> it = buildingMarkers.keySet().iterator();
    while (it.hasNext()) {
        World world = it.next();
        if (!alive.contains(world)) {
            Polygon marker = buildingMarkers.get(world);
            buildingLayer.getChildren().remove(marker);
            it.remove();
        }
    }
}

private void updateUnits(ArrayList<Unit> units) {
    Set<Unit> alive = Collections.newSetFromMap(new IdentityHashMap<>());

    for (Unit unit : units) {
        if (unit == null || Boolean.TRUE.equals(unit.getDead()) || unit.getImage() == null) {
            continue;
        }
        alive.add(unit);

        Polygon marker = unitMarkers.get(unit);
        if (marker == null) {
            marker = new Polygon();
            unitMarkers.put(unit, marker);
            unitLayer.getChildren().add(marker);
        }
    }
}

```

```

    }

    double size = getUnitMarkerRadius(unit) * 2.0;
    updateUnitMarkerShape(marker, size);
    marker.setLayoutX(worldToMiniX(unit.x + 10));
    marker.setLayoutY(worldToMiniY(unit.y + 10));
    marker.setFill(unit.getTeam() ? Color.web("#5fe2b0") : Color.web("#ff7a7a"));
    marker.setStroke(unit.isActive() ? Color.web("#ffd166") : Color.TRANSPARENT);
    marker.setStrokeWidth(unit.isActive() ? 1.2 : 0.0);
    marker.setOpacity(0.92);
}

Iterator<Unit> it = unitMarkers.keySet().iterator();
while (it.hasNext()) {
    Unit unit = it.next();
    if (!alive.contains(unit)) {
        Polygon marker = unitMarkers.get(unit);
        unitLayer.getChildren().remove(marker);
        it.remove();
    }
}

private double worldToMiniX(double worldX) {
    return clamp((worldX / worldWidth) * background.getWidth(), 0, background.getWidth());
}

private double worldToMiniY(double worldY) {
    return clamp((worldY / worldHeight) * background.getHeight(), 0, background.getHeight());
}

private double miniToWorldX(double miniX) {
    return (miniX / background.getWidth()) * worldWidth;
}

private double miniToWorldY(double miniY) {
    return (miniY / background.getHeight()) * worldHeight;
}

private double getUnitMarkerRadius(Unit unit) {
    if (unit instanceof Pretorio) {
        return 3.5;
    }
    if (unit instanceof Centurio) {
        return 3.0;
    }
    return 2.6;
}

private double getBuildingMarkerSize(World world) {
    if (world instanceof Base) {
        return 8.0;
    }
    if (world instanceof Tower) {
        return 6.0;
    }
    return 5.6;
}

private void updateUnitMarkerShape(Polygon marker, double size) {
    double half = size * 0.5;
    marker.getPoints().setAll(
        0.0, -half,
        half, 0.0,
        0.0, half,
        -half, 0.0
    );
}

private void updateBuildingMarkerShape(Polygon marker, World world, double size) {
    double half = size * 0.5;
    if (world instanceof Base) {
        marker.getPoints().setAll(
            -half, -half,
            half, -half,
            half, half,
            -half, half
        );
    } else if (world instanceof Tower) {

```

```

        marker.getPoints().setAll(
            0.0, -half,
            half, half,
            -half, half
        );
    } else {
        marker.getPoints().setAll(
            0.0, -half,
            half, 0.0,
            0.0, half,
            -half, 0.0
        );
    }
}

private void updateViewportRect() {
    viewportRect.setVisible(true);
    double viewportMiniWidth = viewportWidth / worldWidth * background.getWidth();
    double viewportMiniHeight = viewportHeight / worldHeight * background.getHeight();
    viewportRect.setX(worldToMiniX(cameraX));
    viewportRect.setY(20 + worldToMiniY(cameraY));
    viewportRect.setWidth(viewportMiniWidth);
    viewportRect.setHeight(viewportMiniHeight);
}

private double clamp(double value, double min, double max) {
    return Math.max(min, Math.min(max, value));
}
}

```

Лістинг модуля module-info.java

```
module org.example.laba5 {  
    requires javafx.controls;  
    requires javafx.fxml;  
    requires java.xml;  
  
    opens org.example.laba5 to javafx.fxml;  
    opens org.example.laba5.Unit to javafx.fxml;  
    exports org.example.laba5;  
    exports org.example.laba5.Unit;  
}
```